

Fundamente Linux

de Paul Cobbaut

traducere din limba engleză de I. C.

Linux Fundamentals

Paul Cobbaut
lt-2.0

Publicat Joi 01 Aug 2013 01:00:39 CEST

Rezumat

Această carte este menită să fie utilizată într-un curs condus de un instructor. Pentru analiză, intenția este ca această carte să fie citită lângă un computer Linux funcțional astfel încât să faceți imediat fiecare subiect, practicând fiecare comandă.

Această carte se adresează administratorilor novici de sistem Linux (și poate fi interesantă și folositoare pentru utilizatorii casnici care vor să știe un pic mai mult despre sistemul lor Linux). Oricum, această carte nu este făcută să fie o introducere a aplicațiilor desktop Linux ca editoare de text, browsere, clienți e-mail, aplicații multimedia sau office.

Mai multe informații și fișiere gratuite .pdf sînt disponibile la <http://linux-training.be>.

Contactați autorul:

- Paul Cobbaut: paul.cobbaut@gmail.com, <http://www.linkedin.com/in/cobbaut>

Contribuitori ai proiectului Linux Training sînt:

- Serge van Ginderachter: serge@ginsys.eu, construirea scripturilor și setarea infrastructurii
- Ywein Van den Brande: ywein@crealaw.eu, licență și secțiuni legale
- Hendrik De Vloed: hendrik.devloed@ugent.be, pentru scriptul buildheader.pl

Dorim de asemeni să mulțumim recenzenților noștri:

- Wouter Verhelst: wo@uter.be, <http://grep.be>
- Geert Goossens: mail.goossens.geert@gmail.com, <http://www.linkedin.com/in/geertgoossens>
- Elie De Brauer: elie@de-brauer.be, <http://www.de-brauer.be>
- Christophe Vandeplas: christophe@vandeplas.com, <http://christophe.vandeplas.com>
- Bert Desmet: bert@devnox.be, <http://blog.bdesmet.be>
- Rich Yonts: richyonts@gmail.com,

Copyright 2007-2013 Netsec BVBA, Paul Cobbaut

Permisea este acordată de a copia, distribui și/sau modifica acest document în termenii **GNU Free Documentation License**, Versiunea 1.3 sau a oricărei versiuni mai târzii publicată de Free Software Foundation; cu absența Secțiunilor Invariante, fără Texte Copertă Față, și fără Texte Copertă Spate. O copie a licenței este inclusă în secțiunea intitulată 'GNU Free Documentation License'.

CUPRINS

I. introducere în Linux	.6
1. istorie Linux.	.7
2. distribuții.	.9
3. licențe.	11
4. obținerea Linux acasă.	15
II. primii pași în linia de comandă.	26
5. paginile man.	.27
6. lucrând cu directoare.	30
7. lucrând cu fișiere.	.39
8. lucrând cu conținutul fișierelor.	.48
9. arborele de fișier Linux.	.56
III. extindere shell.	.75
10. comenzi și argumente.	.76
11. operatori de control.	.87
12. variabile.	.94
13. istorie shell.	106
14. fișiere globulare.	.112
IV. conduite și comenzi.	119
15. redirectare și conduite.	120
16. filtre.	129
17. utilitare de bază Unix	142
V. vi.	.150
18. introducere în vi.	.151
VI. scripting.	159
19. introducere în scripting.	.160
20. loop-uri script.	167
21. parametri scripting.	175
22. mai mult scripting.	.183
VII. management utilizator local.	191
23. utilizatori.	192
24. grupuri.	210
VIII. securitatea fișierului.	216
25. permisiuni fișier standard.	.217
26. permisiuni fișier avansate.	.226
27. liste control acces.	232
28. legături fișier.	236

IX. Apendice.	243
A. certificări.	.244
B. setări tastatură.	246
C. hardware.	248
D. Licență (în limba română).	.253
E. Licență (în limba engleză).	260

Lista Tabelelor

18.1.	intrînd în modul comandă.	.152
18.2.	schimbare în modul insert.	152
18.3.	înlocuiește și șterge.	152
18.4.	undo și repetă.	.152
18.5.	taie, copie și lipește o linie.	.153
18.6.	taie, copie și lipește linii.	.153
18.7.	început și sfîrșit linie.	.153
18.8.	îmbinare două linii.	153
18.9.	cuvinte.	154
18.10.	salvare și ieșire vi.	.154
18.11.	căutare.	154
18.12.	înlocuiri.	155
18.13.	citire fișiere și introducere.	155
18.14.	memorii intermediare text.	155
18.15.	fișiere multiple.	.155
18.16.	prescurtări.	156
23.1.	mediu de utilizator Debian.	.209
23.2.	mediu de utilizator Red Hat.	209
25.1.	fișiere speciale Unix.	219
25.2.	permisiuni de fișere standard Unix.	.219
25.3.	poziția permisiunilor de fișier Unix.	.220
25.4.	permisiuni octale.	222

Partea I. introducere în Linux

Capitolul 1. istorie Linux

Conținut

1.1 istorie Linux.8

Acest capitol descrie istoria Unix-ului și poziția Linux-ului.

Dacă sînteți nerăbdători să începeți să lucrați cu Linux fără această bla bla bla asupra istoriei, distribuției, și licențe, atunci săriți direct la **Partea II, Capitolul 6, lucrînd cu directoare pagina 30.**

1.1 istorie linux

Toate sistemele de operare moderne își au rădăcina în 1969 când **Dennis Ritchie** și **Ken Thompson** au dezvoltat limbajul C și sistemul de operare **Unix** la AT&T Bell Labs. Ei au împărțit codul lor sursă (da, exista sursă deschisă în anii 70') cu restul lumii, incluzând hippioții din Berkely California. În 1975, când AT&T a început să vândă Unix în mod comercial, aproape jumătate din codul sursă a fost scris de alții. Hippioții nu au fost fericiți că o companie comercială a vândut software pe care ei l-au scris; consecințele (legale) ale luptei s-au sfârșit existind două versiuni de **Unix** în anii șaptezeci: Unix-ul oficial AT&T și liberul Unix **BSD**.

În anii 80' multe companii au început să dezvolte propriul lor Unix: IBM a creat AIX, Sun SunOS (Solaris-ul de mai târziu), HP HP-UX și aproape o duzină de alte companii au făcut la fel. Rezultatul a fost o încurcătură de dialecte Unix și o duzină de căi diferite pentru a face același lucru. Și aici ia naștere prima rădăcină reală a **Linux-ului**, când **Richard Stallman** și-a propus să sfârșească această eră a separării Unix când toată lumea reinventa roata prin a începe proiectul **GNU** (GNU Nu este Unix). Scopul lui a fost să facă un sistem de operare care să fie disponibil fără costuri tuturor, și unde oricine ar putea lucra împreună (ca în anii 70'). Multe dintre utilitățile liniei de comandă pe care le utilizați astăzi pe **Linux** sau Solaris sînt utilitare GNU.

Anii 90' au început cu **Linus Torvalds**, un student finlandez vorbitor al limbii suedeze, care a cumpărat un computer 386 și a scris un nou kernel POSIX corespunzător. El a făcut disponibil codul sursă online, crezînd că acesta nu va susține niciodată decît hardware 386. Multă lume a îmbrățișat combinația acestui kernel cu utilitățile GNU, și restul, după cum se spune, este istorie.

Astăzi mai mult de 90% din supercomputere (incluzînd topul 10 complet), mai mult de jumătate dintre toate smartphone-urile, multe milioane de computere desktop, în jur de 70% dintre toate serverele web, un mare bloc de computere tabletă, și mai multe dispozitive (dvd-playere, mașini de spălat, modemuri dsl, rutere, ...) execută **Linux**. Este de departe sistemul de operare cel mai utilizat în mod obișnuit din lume.

Versiunea de kernel Linux 3.2 a fost lansat în ianuarie 2012. Codul lui sursă a crescut cu aproape două sute de mii de linii (comparat cu versiunea 3.1) datorită contribuțiilor a peste 4000 de dezvoltatori plătiți de aproape 200 de companii comerciale incluzînd Red Hat, Intel, Broadcom, Texas Instruments, IBM, Novell, Qualcomm, Samsung, Nokia, Oracle, Google și chiar Microsoft.

http://en.wikipedia.org/wiki/Dennis_Ritchie

http://en.wikipedia.org/wiki/Richard_Stallman

http://en.wikipedia.org/wiki/Linus_Torvalds

<http://kernel.org>

<http://lwn.net/Articles/472852/>

<http://www.linuxfoundation.org/>

<http://en.wikipedia.org/wiki/Linux>

<http://www.levenez.com/unix/> (un poster imens cu istorie Unix)

Capitolul 2. distribuții

Conținut

2.1. Red Hat.	9
2.2. Ubuntu.	9
2.3. Debian.	9
2.4. altă distribuție.	9
2.5. pe care să o aleg?.	10

Acest capitol dă o privire de ansamblu a distribuțiilor Linux curente.

O **distribuție** Linux este o colecție de software (sursă deschisă de obicei) deasupra unui kernel Linux. O distribuție (sau pe scurt, distro) poate să înmănuncheze software server, utilitare de management de sistem, documentație și multe aplicații desktop într-un **repozitoriu software central sigur**. O distribuție urmărește să înzestreze un aspect comun, un management software ușor și sigur și deseori un scop operațional specific.

Să aruncăm o privire asupra unor distribuții populare.

2.1. Red Hat

Red Hat este o companie comercială Linux de un miliard de dolari care pune mult efort în dezvoltarea Linux. Deține sute de specialiști Linux și sînt cunoscuți pentru sprijinul lor excelent. Ei dau produsele lor (Red Hat Enterprise Linux și Fedora) gratuit. În timp ce **Red Hat Enterprise Linux** (RHEL) este bine testat înainte de lansare, **Fedora** este o distribuție cu actualizări mai rapide dar fără suport.

2.2. Ubuntu

Canonical a început în 2004 să trimită compact disk-uri gratuite cu **Ubuntu** Linux în 2004 și a devenit repede popular pentru utilizatori (mulți schimbînd Microsoft Windows cu Ubuntu). Canonical vrea ca Ubuntu să fie un desktop grafic Linux ușor de folosit fără nevoia de a vedea vreodată o linie de comandă. Bineînțeles ei vor deasemeni să facă profit vînzînd sprijin pentru Ubuntu.

2.3. Debian

Nu există nici o companie în spatele **Debian**. În schimb există sute de dezvoltatori bine organizați care aleg un Debian Project Leader la fiecare doi ani. Debian este privit ca unul dintre cele mai stabile distribuții Linux. Este de asemeni baza fiecărei lansări de Ubuntu. Debian vine în trei versiuni: stabil, testare și instabil. Fiecare lansare Debian este numită după un personaj din filmul Toy Story.

2.4. altă distribuție

Distribuții ca CentOS, Oracle Enterprise Linux și Scientific Linux sînt bazate pe Red Hat Enterprise Linux și împart multe din aceleași principii de bază, tehnici directe și sisteme administrative. Linux Mint, Edubuntu și multe alte distribuții numite *buntu sînt bazate pe Ubuntu și astfel împart multe cu Debian. Există sute de alte distribuții Linux.

2.5. pe care să o aleg?

Cînd sînteți nou în Linux în 2012, alegeți ultimul Ubuntu sau Fedora. Dacă vreți doar să practicați linia de comandă Linux atunci instalați un Ubuntu server și/sau un CentOS server (fără interfață grafică).

redhat.com
ubuntu.com
debian.org
centos.org
distrowatch.com

Capitolul 3. licențe

Conținut

3.1. despre licențe software.	12
3.2. software și freeware ale domeniului public.	12
3.3. Free Software sau Open Source Software.	12
3.4. licența GNU General Public License.	13
3.5. folosind software GPLv3.	13
3.6. licența BSD.	13
3.7. alte licențe.	14
3.8. combinații de licențe software.	14

Acest capitol explică pe scurt diferitele licențe folosite pentru distribuirea software-ului sistemelor de operare.

Multe mulțumiri lui **Ywein Van den Brande** pentru scrierea a unei mari părți a acestui capitol.

Ywein este un avocat al apărării, co-autor a **Cărții Internaționale a Legii FOSS** și autor al **Praktijkboek Informaticarecht** (în limba olandeză).



<http://ifosslawbook.org>

<http://www.crealaw.eu>

3.1. despre licențe software

Există două paradigme predominante software: **Free and Open Source Software (FOSS)** și **software proprietar**. Criteriul de diferențiere între aceste două aproprieri este bazat pe controlul asupra software-ului. Cu **software proprietar**, controlul tinde să se încline mai mult spre vânzător, în timp ce cu **Free and Open Source Software** el tinde să fie mai înclinat spre utilizatorul final. Dar chiar dacă paradigmele diferă, ele folosesc aceleași **legi de copyright** pentru a ajunge și a pune în practică scopurile lor. Dintr-o perspectivă legală, **Software-ul liber și cu sursă deschisă (FOSS)** poate fi considerat ca software pentru care utilizatorii în general primesc mai multe drepturi prin intermediul licenței decât ar avea cu **licența software proprietar**, totuși principiile mecanismelor licenței sînt la fel.

Teoria legală susține că autorul FOSS, în mod contrar autorului de **software de domeniu public**, nu a renunțat în nici un caz la drepturile muncii sale. FOSS susține drepturile de autor (**copyright**) pentru a impune condițiile licenței FOSS. Condițiile licenței FOSS trebuie să fie respectate de către utilizator în același fel precum condițiile licenței proprietare. Întotdeauna verificați licența cu atenție înainte de a utiliza software third party.

Exemple de software proprietar sînt **AIX** de la IBM, **HP-UX** de la HP și **Oracle Database 11g**. Nu sînteți autorizat să instalați sau să folosiți acest software fără a plăti o taxă de licență. Nu sînteți autorizați să distribuiți copii și nu sînteți autorizați să modificați cod sursă închis.

3.2. software de domeniu public și freeware

Software care este original în sensul în care este o creație intelectuală a autorului beneficiază de protecție **copyright**. Software non-original nu intră în considerare pentru protecție **copyright** și poate, în principiu, să fie utilizat în mod liber.

Software de domeniul public este considerat ca software pentru care autorul a renunțat la toate drepturile și pentru care nimeni nu-i este îngăduit să îi impună nici un drept. Acest software poate fi utilizat, reprodus sau executat în mod liber, fără permisiune sau plata unei taxe.

Freeware nu este software de domeniu public sau FOSS. Este software proprietar pe care îl puteți utiliza fără a plăti un cost al licenței. Oricum, deseori, termenii stricți ai licenței trebuie să fie respectați.

Exemple de freeware sînt **Adobe Reader**, **Skype** și **Command and Conquer: Tiberian Sun** (acest joc a fost vîndut ca proprietar în 1999 și este disponibil din 2011 ca freeware).

3.3. Free Software sau Open Source Software

Ambele mișcări **Free Software** (se traduce cu **vrije software** în limba olandeză și cu **Logiciel Libre** în limba franceză) și mișcarea **Open Source Software** urmăresc în mod larg scopuri similare și îmbracă licențe software similare. Dar din punct de vedere istoric, au existat percepții de diferențiere datorită diferitelor puncte de vedere. În timp ce mișcarea **Free Software** se focalizează asupra drepturilor (cele patru libertăți) pe care Free Software le dă utilizatorilor lor, mișcarea **Open Source Software** subliniază Definiția Open Source și avantajele dezvoltării software-ului peer-to-peer.

Recent, termenul software liber și cu sursă deschisă sau FOSS a răsărit ca o

alternativă neutră. O variantă mai puțin utilizată este free/libre/open source software (FLOSS), care folosește termenul **libre** pentru a clarifica înțelesul de liber ca **libertate** versus **fără taxă**.

Exemple de software liber sînt **gcc**, **MySQL** și **gimp**.

Informații detaliate despre cele **patru libertăți** pot fi găsite aici:

<http://www.gnu.org/philosophy/free-sw.html>

Definiția open source poate fi găsită la:

<http://www.opensource.org/docs/osd>

Definiția de mai sus este bazată pe **Directivele Debian Free Software** disponibile aici:

http://www.debian.org/social_contract#guidelines

3.4. licența GNU General Public License

Cu mult și mai mult software este lansat sub **GNU GPL** (în 2006 Java a fost lansat sub GPL). Această licență (versiunea 2 și versiunea 3) este licența principală îmbrățișată de Free Software Foundation. Caracteristica ei principală este principiul **copyleft**. Aceasta înseamnă că oricine în lanțul utilizatorilor consecutivi, în schimbul dreptului de utilizare care este stabilit, trebuie să distribuie îmbunătățirile pe care le face software-ului și lucrărilor lui derivate sub aceleași condiții pentru alți utilizatori, dacă el alege să distribuie astfel de îmbunătățiri sau lucrări derivate. Cu alte cuvinte, software care încorporează software GNU GPL, trebuie să fie distribuit la rîndul lui ca software GNU GPL (sau compatibil, vezi mai jos). Nu este posibil a incorpora părți software protejate prin copyright ale GNU GPL într-o lucrare licențiată cu software proprietar. GPL a fost sprijinit în instanță judecătorească.

3.5. folosind software GPLv3

Puteți folosi **software GPLv3** aproape fără nici o condiționare. Dacă folosiți numai software nici nu trebuie să acceptați termenii GPLv3. Oricum, orice altă utilizare – cum ar fi modificarea sau distribuirea de software – implică acceptare. În cazul în care folosiți software intern (incluzînd o rețea), puteți modifica software fără a fi obligat să distribuiți modificarea. Puteți angaja firme pentru a lucra asupra software-ului în mod exclusiv doar pentru dumneavoastră și sub directarea și controlul dumneavoastră. Dar dacă modificați software și îl folosiți altfel decît intern, aceasta va fi considerată o distribuție. Trebuie să distribuiți modificările dumneavoastră sub GPLv3 (principiul copyleft). Mai multe obligații se aplică dacă distribuiți software GPLv3. Verificați licența GPLv3 cu atenție.

Dacă creați produs cu software GPLv3: GPLv3 nu se aplică în mod automat produsului.

3.6. licența BSD

Există cîteva versiuni ale licenței originale Berkeley Distribution License. Cea mai comună este licența celor trei clauze („New BSD License” sau „Modified BSD License”).

Aceasta este o licență free software permisivă. Licența pune restricții minimale

asupra modalității cum software-ul poate fi redistribuit. Ea este în contrast cu licențele copyleft cum ar fi GPLv3 discutată mai sus, care are un mecanism copyleft.

Diferența este mai puțin importantă numai când folosiți software-ul, dar intervine când începeți să redistribuiți copii a software-ului sau propriile dumneavoastră versiuni modificate.

3.7. alte licențe

FOSS sau nu, există multe tipuri de licențe asupra software-ului. Ar trebui să le citiți și să le înțelegeți înainte de a utiliza orice software.

3.8. combinații de licențe software

Când folosiți mai multe surse sau vreți să redistribuiți software sub o altă licență, trebuie să verificați dacă toate celelalte licențe sînt compatibile. Unele licențe FOSS (precum BSD) sînt compatibile cu licențele proprietare, dar cele mai multe nu sînt. Dacă întîlniți o incompatibilitate a licențelor, trebuie să contactați autorul pentru a negocia diferite condiții ale licenței sau să vă abțineți să utilizați software incompatibil.

Capitolul 4. obținerea Linux acasă

Conținut

4.1. downloadați o imagine Linux CD.	16
4.2. downloadați VirtualBox.	16
4.3. creați o mașină virtuală.	17
4.4. atașați imaginea CD.	22
4.5. instalați Linux.	25

Această carte presupune că aveți acces la un computer Linux funcțional. Multe companii au unul sau mai multe servere Linux, dacă sînteți deja intrat în sistem pe el, atunci sînteți gata (treceți peste acest capitol la următorul).

O altă opțiune este să puneți un CD Ubuntu Linux într-un computer cu (sau fără) Microsoft Windows și să urmați instalarea. Ubuntu va micșora (sau crea) partiții și va seta un meniu la butare pentru a alege Windows sau Linux.

Dacă nu aveți acces la un computer Linux în acest moment, sau dacă sînteți neștiutor sau nesigur în ceea ce privește instalarea Linux pe computerul dumneavoastră, atunci acest capitol propune o a treia opțiune: instalarea Linux pe o mașină virtuală.

Instalarea pe o mașină virtuală (dată de **VirtualBox**) este ușoară și sigură. Chiar și atunci cînd faceți greșeli și stricați totul pe mașina virtuală Linux, atunci nimic nu este atins pe computerul real.

Acest capitol arată pași simpli și capturi de ecran pentru a obține un server Ubuntu funcțional într-o mașină virtuală VirtualBox. Pașii sînt foarte asemănători ca instalarea Fedora sau CentOS sau chiar Debian, și dacă vă place puteți de asemenea folosi VMWare în loc de VirtualBox.

4.1. downloadați o imagine Linux CD

Începeți prin a downloada o imagine Linux CD (un fișier .iso) de la o distribuție la alegerea dumneavoastră de pe Internet. Aveți grijă la selecția arhitecturii corecte cpu a computerului dumneavoastră; alegeți **i386** dacă nu știți sigur. A alege greșit tipul cpu (ca x86_64 când aveți un Pentium vechi) va eșua aproape imediat să buteze CD-ul.



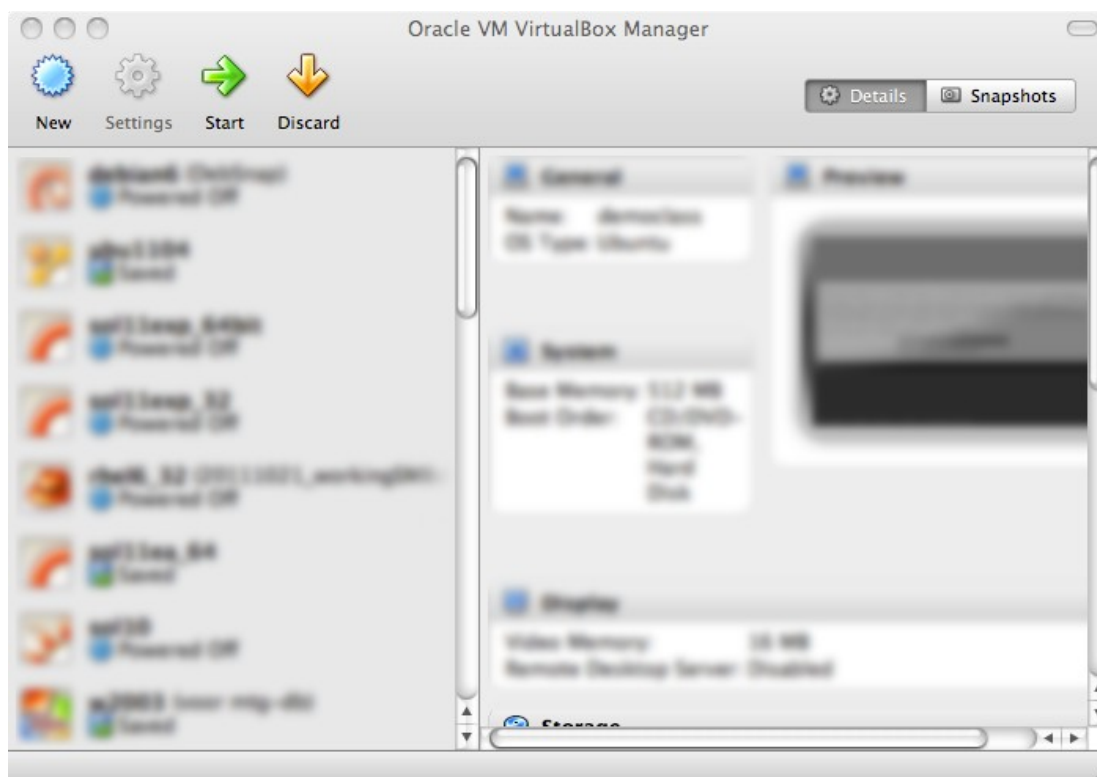
4.2. downloadați VirtualBox

Pasul doi (când fișierul .iso s-a terminat de downloadat) este să downloadați VirtualBox. Dacă aveți Microsoft Windows pe computer atunci downloadați și instalați VirtualBox pentru Windows!



4.3. creați o mașină virtuală

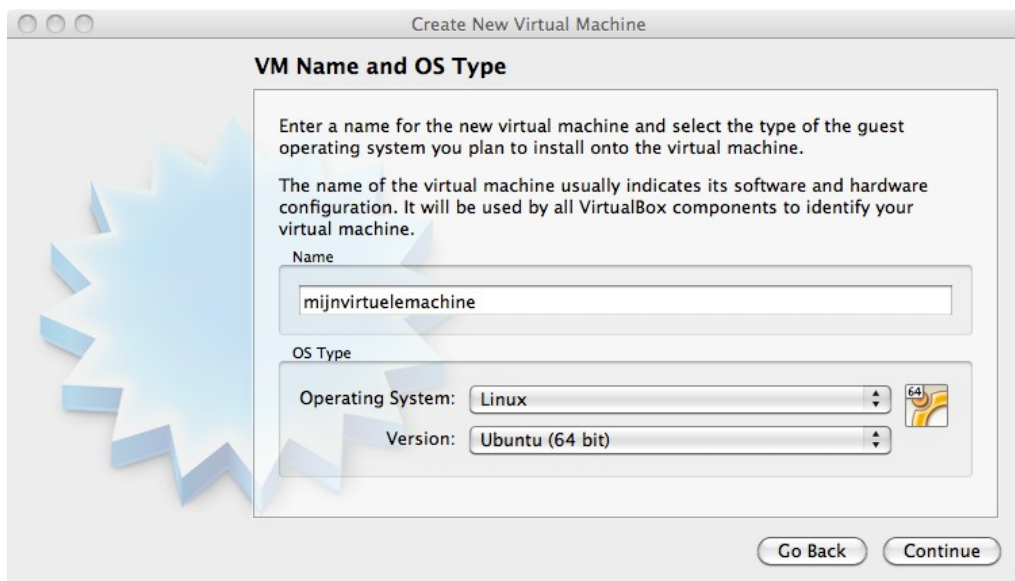
Acum deschideți VirtualBox. Contrar capturii de ecran de dedesubt, panoul din stînga ar trebui să fie gol.



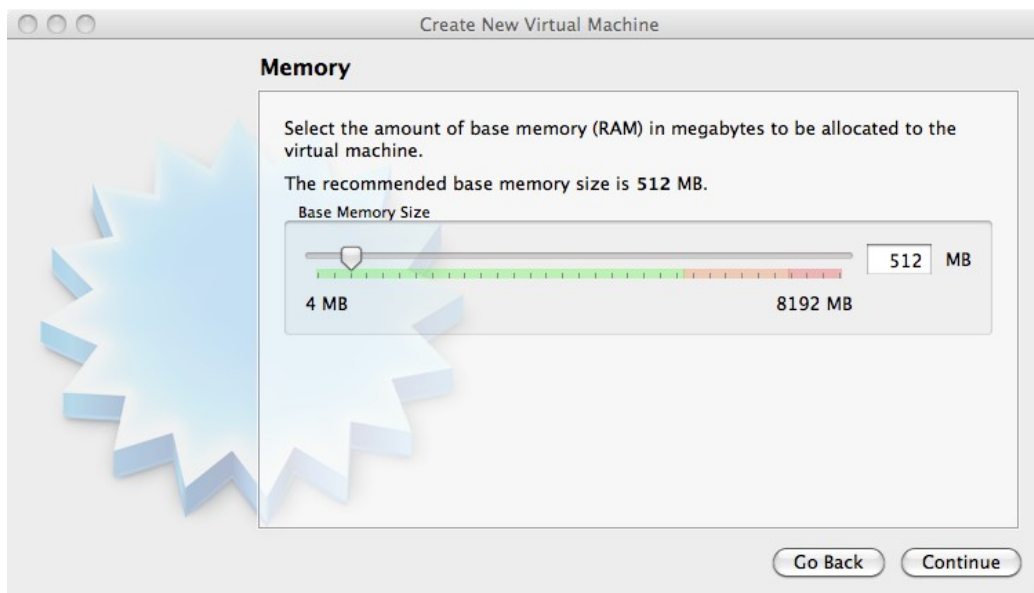
Click **New** pentru a crea o mașină virtuală nouă. Vă vom însoți de-a lungul acestui proces. Capturile de ecran de dedesubt sînt luate de pe un Mac OS X; ele vor fi ușor diferite dacă executați Microsoft Windows.



Dați un nume mașinii virtuale (și eventual selectați 32-bit sau 64-bit).



Dați mașinii virtuale memorie (512MB dacă aveți 2GB sau mai mult, altfel selectați 256MB).



Selecțați pentru a crea un disk nou (rețineți, acesta va fi un disk virtual).



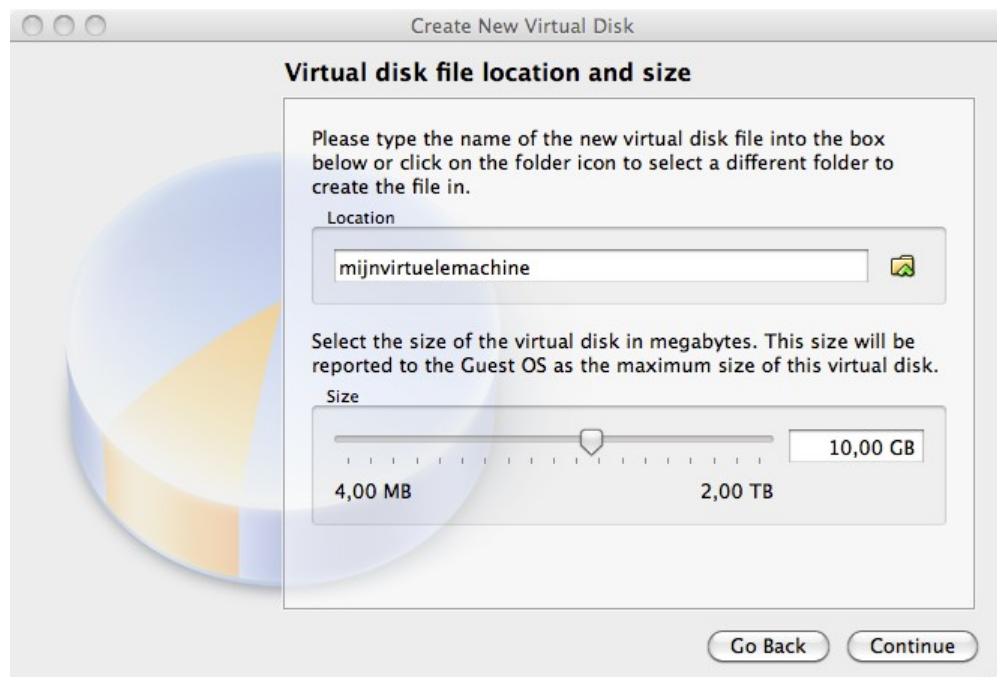
Dacă obțineți întrebarea de dedesubt, alegeți vdi.



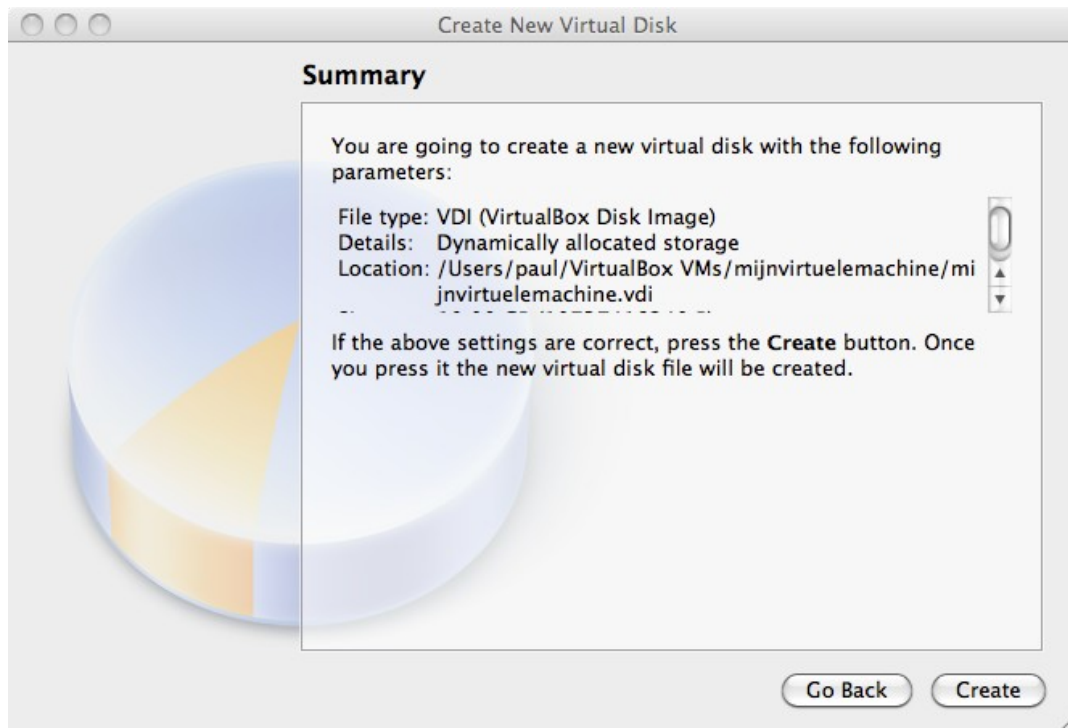
Alegeți **alocat dinamic** (mărimea fixă este folosită doar în producție sau pe hardware foarte vechi, încet).



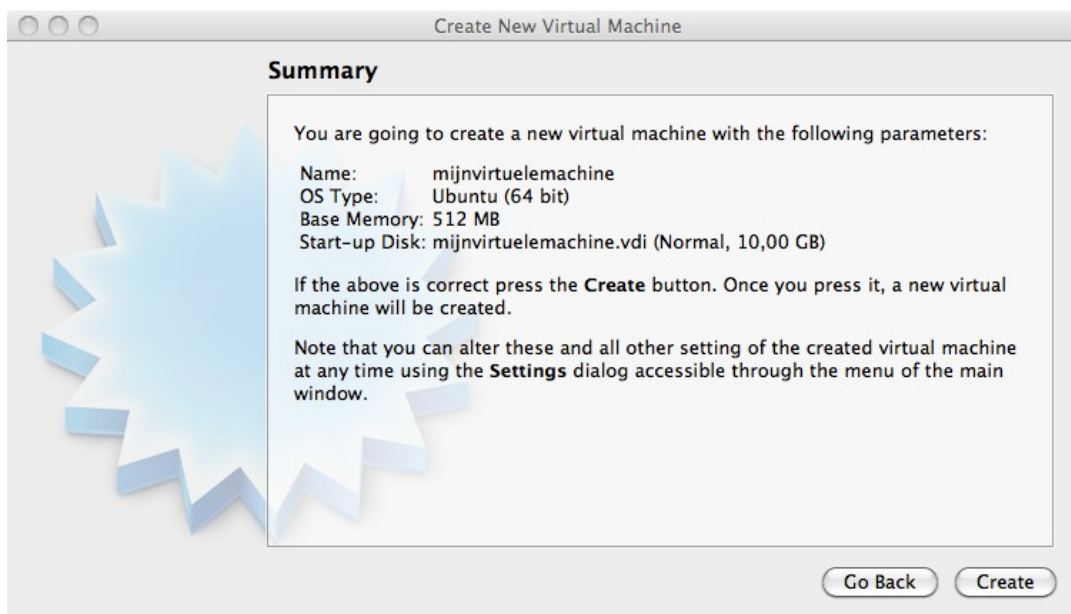
Alegeți între 10 GB și 16 GB ca mărime a disk-ului.



Click **create** pentru a crea disk-ul virtual.

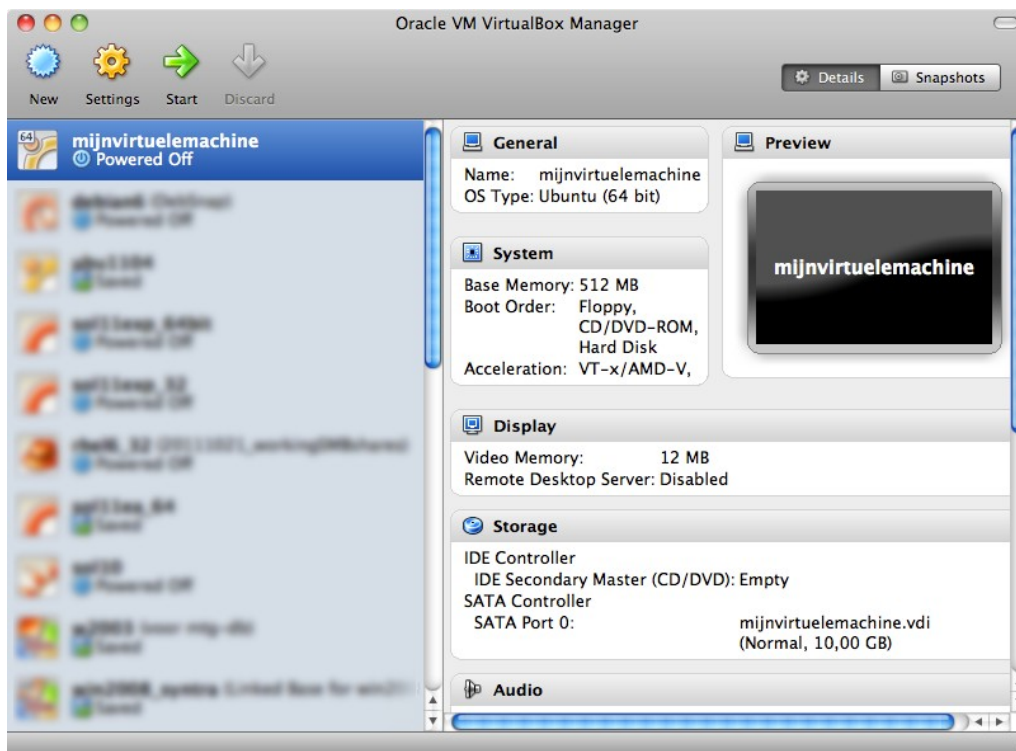


Click **create** pentru a crea mașina virtuală.

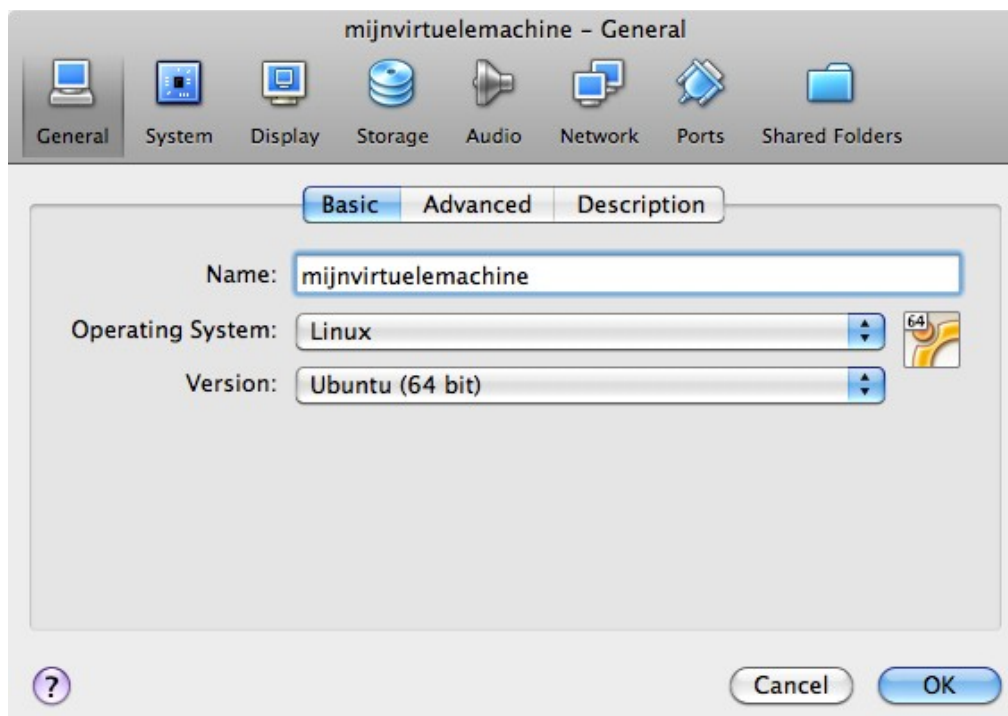


4.4. ataşați imaginea CD

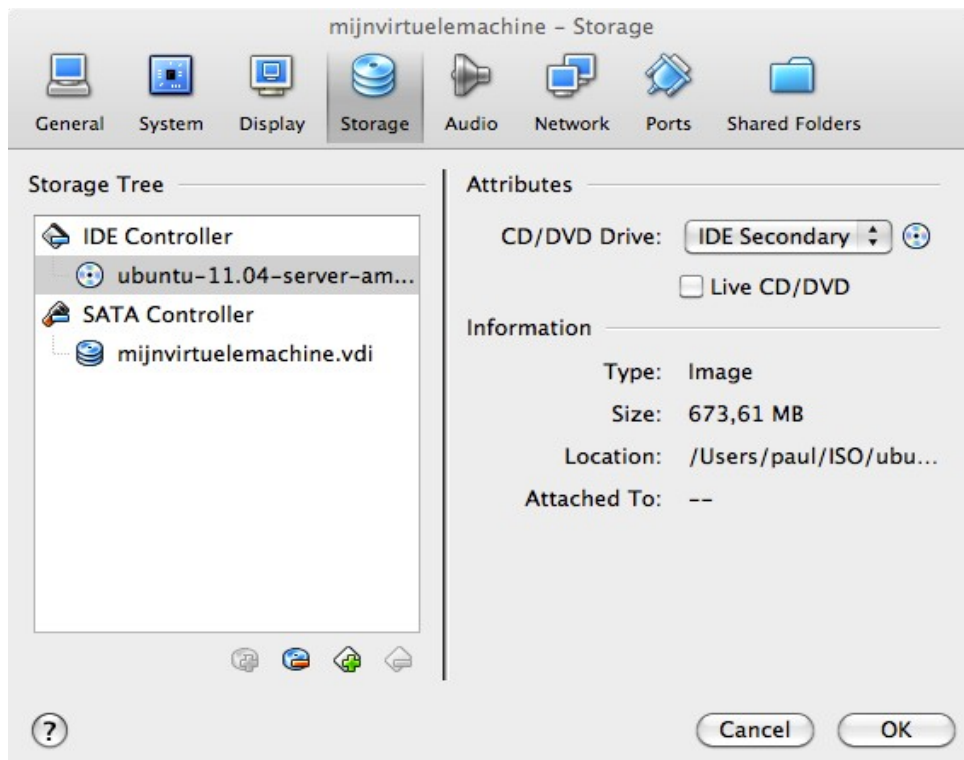
Înainte de a da start computerului virtual, haideți să aruncăm o privire la anumite setări (click **Settings**).



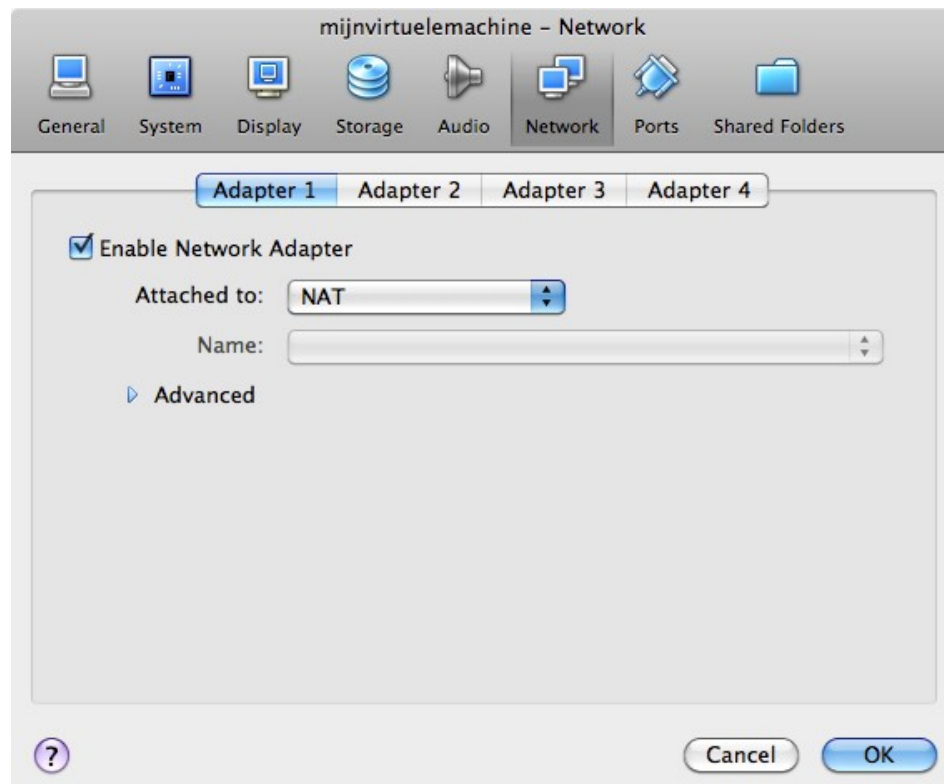
Nu vă faceți griji dacă ecranul arată altfel, trebuie să găsiți doar butonul pe care scrie **Storage**.



23



Poate fi folositor să vă setați adaptorul de rețea la bridge în loc de NAT. Bridged de obicei vă va conecta computerul virtual la Internet.



4.5. instalați Linux

Mașina virtuală este gata acum de start. Când i se dă o alegere la butare, selectați **install** și urmați instrucțiunile de pe ecran. Când instalarea este terminată, puteți intra în computer și să începeți să exersați Linux!

Partea II. primii pași în linia de comandă

Capitolul 5. paginile man

Conținut

5.1.	comanda \$man.	.28
5.2.	man \$configfile.	28
5.3.	man \$daemon.	28
5.4.	man -k (apropos).	.28
5.5.	whatis.	.28
5.6.	whereis.	28
5.7.	secțiunile man.	.29
5.8.	man \$section \$file.	.29
5.9.	man man.	29
5.10.	mandb.	29

Acest capitol va explica folosirea paginilor **man** (numite și **pagini manual**) pe computerul dumneavoastră Unix sau Linux.

Veți învăța comanda **man** împreună cu comenzi ca **whereis**, **whatis** și **mandb**.

Cele mai multe fișiere și comenzi Unix au pagini foarte bune de manual pentru a explica folosirea lor. Paginile de manual sînt de asemeni folosite cînd utilizați multiple versiuni de Unix sau cîteva distribuții Linux pentru că opțiunile și parametrii uneori variază.

5.1. comanda \$man

Tastați **man** urmat de o comandă (pentru care doriți ajutor) și începeți să citiți. Apăsati **q** pentru a renunța la pagina de manual. Uneori paginile de manual conțin exemple (spre sfârșit).

```
paul@laika:~$ man whois
Reformatting whois(1), please wait...
```

5.2. man \$configfile

Cele mai multe **fișiere de configurare** au propriul lor manual.

```
paul@laika:~$ man syslog.conf
Reformatting syslog.conf(5), please wait...
```

5.3. man \$daemon

Asta e deasemeni adevărat pentru mulți **daemoni** (programe background) pe sistem.

```
paul@laika:~$ man syslogd
Reformatting syslogd(8), please wait...
```

5.4. man -k (apropos)

man -k (sau **apropos**) arată o listă de pagini manual conținând un șir.

```
paul@laika:~$ man -k syslog
lm-syslog-setup (8)      - configure laptop mode to switch syslog.conf ...
logger (1)              - a shell command interface to the syslog(3) ...
syslog-facility (8)     - Setup and remove LOCALx facility for sysklogd
syslog.conf (5)         - syslogd(8) configuration file
syslogd (8)             - Linux system logging utilities.
syslogd-listfiles (8)   - list system logfiles
```

5.5. whatis

Pentru a vedea doar o descriere a unei pagini de manual folosiți **whatis** urmat de un șir.

```
paul@u810:~$ whatis route
route (8)              - show / manipulate the IP routing table
```

5.6. whereis

Localizarea unei pagini de manual poate fi dezvăluită cu **whereis**.

```
paul@laika:~$ whereis -m whois
whois: /usr/share/man/man1/whois.1.gz
```

Acest fișier este citit direct de **man**.

```
paul@laika:~$ man /usr/share/man/man1/whois.1.gz
```

5.7. secțiunile man

Ați observat deja numerele dintre parantezele rotunde. **man man** vă va explica că acestea sînt numere secțiune. Programele executabile și comenzile terminal stau în secțiunea întâi.

- 1 Executable programs or shell commands
- 2 System calls (functions provided by the kernel)
- 3 Library calls (functions within program libraries)
- 4 Special files (usually found in /dev)
- 5 File formats and conventions eg /etc/passwd
- 6 Games
- 7 Miscellaneous (including macro packages and conventions), e.g. man(7)
- 8 System administration commands (usually only for root)
- 9 Kernel routines [Non standard]

5.8. man \$section \$file

De aceea, cînd faceți referire la pagina manual a comenzii **passwd** (1), o veți vedea scrisă ca **passwd(1)**; cînd faceți referire la fișierul **passwd**, îl veți vedea scris ca **passwd(5)**. Captura de ecran explică cum să deschideți pagina de manual în secțiunea corectă.

```
[paul@RHEL52 ~]$ man passwd      # deschide primul manual găsit
[paul@RHEL52 ~]$ man 5 passwd    # deschide o pagină din secțiunea 5
```

5.9. man man

Dacă vreți să știți mai multe despre **man**, atunci citiți Manualul Fantastic (RTFM).

Din păcate, paginile manual nu au răspunsul la toate...

```
paul@laika:~$ man woman
No manual entry for woman
```

5.10 mandb

Dacă ar trebui să fiți convins că o pagină manual există, dar nu o puteți accesa, atunci încercați comanda **mandb**.

```
root@laika:~# mandb
0 man subdirectories contained newer manual pages.
0 manual pages were added.
0 stray cats were added.
0 old database entries were purged.
```

Capitolul 6. lucrînd cu directoare

Conținut

6.1. pwd.	31
6.2. cd.	31
6.3. căi absolute și relative.	32
6.4. completarea căii.	33
6.5. ls.	33
6.6. mkdir.	34
6.7. rmdir.	35
6.8. practică: lucrînd cu directoare.	36
6.9. soluție: lucrînd cu directoare.	37

Pentru a explora **arborele fișier** Linux, veți avea nevoie de niște utilitare de bază.

Acest capitol este o mică trecere în revistă a celor mai comune comenzi pentru a lucra cu directoare: **pwd**, **cd**, **ls**, **mkdir**, **rmdir**. Aceste comenzi sînt disponibile pe orice sistem Linux (sau Unix).

În acest capitol se vorbește de asemeni despre căile **absolute** și **relative** și **completarea căii** în shell-ul **bash**.

6.1. pwd

Semnul **sînteți aici** poate fi afișat cu comanda **pwd** (afișează directorul curent). Haideți, încercați: deschideți o interfață de line de comandă (ca gnome-terminal, konsole, xterm, sau un tty) și tastați **pwd**. Utilitarul afișează **directorul curent**.

```
paul@laika:~$ pwd
/home/paul
```

6.2. cd

Puteți schimba directorul curent cu comanda **cd** (schimbă directorul).

```
paul@laika$ cd /etc
paul@laika$ pwd
/etc
paul@laika$ cd /bin
paul@laika$ pwd
/bin
paul@laika$ cd /home/paul/
paul@laika$ pwd
/home/paul
```

cd ~

Puteți reuși să faceți un truc cu **cd**. Doar tastînd **cd** fără un director țintă, vă va pune în directorul home. Tastînd **cd ~** are același efect.

```
paul@laika$ cd /etc
paul@laika$ pwd
/etc
paul@laika$ cd
paul@laika$ pwd
/home/paul
paul@laika$ cd ~
paul@laika$ pwd
/home/paul
```

cd ..

Pentru a merge la **directorul părinte** (sau în cel de deasupra directorului curent în directorul arbore), tastați **cd..**

```
paul@laika$ pwd
/usr/share/games
paul@laika$ cd ..
paul@laika$ pwd
/usr/share
```

*Pentru a rămîne în directorul curent, tastați **cd .**;-) Vom vedea ajutorul caracterului **.** reprezentînd directorul curent mai tîrziu.*

cd -

O altă scurtătură folositoare cu **cd** e să tastați doar **cd -** pentru a merge în directorul de dinainte.

```
paul@laika$ pwd
/home/paul
paul@laika$ cd /etc
paul@laika$ pwd
/etc
paul@laika$ cd -
/home/paul
paul@laika$ cd -
/etc
```

6.3. căi absolute și relative

Ar trebui să vă dați seama de **căile absolute și relative** din fișierul arbore. Când tastați o cale care începe cu un **slash (/)**, atunci **rădăcina** fișierului arbore este asumată. Dacă nu începeți calea cu un slash, atunci directorul curent este punctul de start asumat.

Captura de ecran de mai jos afișează directorul curent **/home/paul**. Din interiorul acestui director, trebuie să tastați **cd /home** în loc de **cd home** pentru a merge în directorul **/home**.

```
paul@laika$ pwd
/home/paul
paul@laika$ cd home
bash: cd: home: No such file or directory
paul@laika$ cd /home
paul@laika$ pwd
/home
```

Când sînteți în interiorul **/home**, trebuie să tastați **cd paul** în loc de **cd /paul** pentru a intra în subdirectorul **paul** al directorului curent **/home**.

```
paul@laika$ pwd
/home
paul@laika$ cd /paul
bash: cd: /paul: No such file or directory
paul@laika$ cd paul
paul@laika$ pwd
/home/paul
```

În caz că directorul curent este **directorul rădăcină /**, atunci și **cd /home** și **cd home** vă va duce în directorul **/home**.


```
paul@laika$ pwd
/
paul@laika$ cd home
paul@laika$ pwd
/home
paul@laika$ cd /
paul@laika$ cd /home
paul@laika$ pwd
/home
```

Aceasta a fost ultima captură de ecran cu instrucțiunile **pwd**. De acum înainte, directorul curent va fi deseori afișat în prompt. Mai târziu în această carte vom explica cum variabila shell **\$PS1** poate fi configurată pentru a afișa asta.

6.4. completarea căii

Tasta tab vă poate ajuta să tastați o cale fără erori. Tastînd **cd /et** urmată de **tasta tab** va mări linia de comandă la **cd /etc**. Cînd tastați **cd /Et** urmată de **tasta tab**, nu se va întîmpla nimic pentru că ați tastat **calea** greșită (litera mare E).

Veți avea nevoie de mai puține apăsări de taste folosind **tasta tab**, și veți fi sigur că **calea** tastată este corectă!

6.5. ls

Puteți lista conținutul unui director cu **ls**.

```
paul@pasha:~$ ls
allfiles.txt dmesg.txt httpd.conf stuff summer.txt
paul@pasha:~$
```

ls -a

O opțiune frecventă cu **ls** este **-a** pentru a afișa toate fișierele. A vedea toate fișierele înseamnă inclusiv **fișierele ascunse**. Cînd un nume de fișier pe un sistem de fișiere Unix începe cu un punct, este considerat un fișier ascuns și nu apare în afișările fișierelor normale.

```
paul@pasha:~$ ls
allfiles.txt dmesg.txt httpd.conf stuff summer.txt
paul@pasha:~$ ls -a
.    allfiles.txt .bash_profile dmesg.txt .lessht stuff
..   .bash_history .bashrc httpd.conf .ssh  summer.txt
paul@pasha:~$
```

ls -l

De multe ori veți folosi opțiuni cu **ls** pentru a afișa conținutul directorului în diferite formate sau să afișați diferite părți ale directorului. Tastînd doar **ls** vă dă o listă a fișierelor în director. Tastînd **ls -l** (aceasta este litera L, nu numărul 1) vă dă o listă lungă.

```
paul@pasha:~$ ls -l
total 23992
-rw-r--r-- 1 paul paul 24506857 2006-03-30 22:53 allfiles.txt
-rw-r--r-- 1 paul paul 14744 2006-09-27 11:45 dmesg.txt
-rw-r--r-- 1 paul paul 8189 2006-03-31 14:01 httpd.conf
drwxr-xr-x 2 paul paul 4096 2007-01-08 12:22 stuff
-rw-r--r-- 1 paul paul 0 2006-03-30 22:45 summer.txt
```

ls -lh

O altă opțiune frecvent utilizată a ls este **-h**. Ea arată numerele (mărimile fișierelor) într-un format mai ușor lizibil. De asemeni mai jos este arătată variația privind modalitatea pe care o puteți da comenzii ls. Vom explica detaliile afișărilor mai târziu în această carte.

```
paul@pasha:~$ ls -l -h
total 24M
-rw-r--r-- 1 paul paul 24M 2006-03-30 22:53 allfiles.txt
-rw-r--r-- 1 paul paul 15K 2006-09-27 11:45 dmesg.txt
-rw-r--r-- 1 paul paul 8.0K 2006-03-31 14:01 httpd.conf
drwxr-xr-x 2 paul paul 4.0K 2007-01-08 12:22 stuff
-rw-r--r-- 1 paul paul 0 2006-03-30 22:45 summer.txt
paul@pasha:~$ ls -lh
total 24M
-rw-r--r-- 1 paul paul 24M 2006-03-30 22:53 allfiles.txt
-rw-r--r-- 1 paul paul 15K 2006-09-27 11:45 dmesg.txt
-rw-r--r-- 1 paul paul 8.0K 2006-03-31 14:01 httpd.conf
drwxr-xr-x 2 paul paul 4.0K 2007-01-08 12:22 stuff
-rw-r--r-- 1 paul paul 0 2006-03-30 22:45 summer.txt
paul@pasha:~$ ls -hl
total 24M
-rw-r--r-- 1 paul paul 24M 2006-03-30 22:53 allfiles.txt
-rw-r--r-- 1 paul paul 15K 2006-09-27 11:45 dmesg.txt
-rw-r--r-- 1 paul paul 8.0K 2006-03-31 14:01 httpd.conf
drwxr-xr-x 2 paul paul 4.0K 2007-01-08 12:22 stuff
-rw-r--r-- 1 paul paul 0 2006-03-30 22:45 summer.txt
paul@pasha:~$ ls -h -l
total 24M
-rw-r--r-- 1 paul paul 24M 2006-03-30 22:53 allfiles.txt
-rw-r--r-- 1 paul paul 15K 2006-09-27 11:45 dmesg.txt
-rw-r--r-- 1 paul paul 8.0K 2006-03-31 14:01 httpd.conf
drwxr-xr-x 2 paul paul 4.0K 2007-01-08 12:22 stuff
-rw-r--r-- 1 paul paul 0 2006-03-30 22:45 summer.txt
```

6.6. mkdir

Plimbându-ne prin sistemul de fișiere Unix e amuzant, dar este și mai amuzant a crea propriile directoare cu **mkdir**. Trebuie să dați cel puțin un parametru lui **mkdir**, numele noului director pentru a fi creat. Gândiți-vă înainte să tastați un /.

```

paul@laika:~$ mkdir MyDir
paul@laika:~$ cd MyDir
paul@laika:~/MyDir$ ls -al
total 8
drwxr-xr-x 2 paul paul 4096 2007-01-10 21:13 .
drwxr-xr-x 39 paul paul 4096 2007-01-10 21:13 ..
paul@laika:~/MyDir$ mkdir stuff
paul@laika:~/MyDir$ mkdir otherstuff
paul@laika:~/MyDir$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 2007-01-10 21:14 otherstuff
drwxr-xr-x 2 paul paul 4096 2007-01-10 21:14 stuff
paul@laika:~/MyDir$

```

mkdir -p

Când este dată opțiunea **-p**, atunci **mkdir** va crea directoare părinte cum este necesar.

```

paul@laika:~$ mkdir -p MyDir2/MySubdir2/ThreeDeep
paul@laika:~$ ls MyDir2
MySubdir2
paul@laika:~$ ls MyDir2/MySubdir2
ThreeDeep
paul@laika:~$ ls MyDir2/MySubdir2/ThreeDeep/

```

6.7. rmdir

Când un director este gol, puteți folosi **rmdir** pentru a șterge directorul.

```

paul@laika:~/MyDir$ rmdir otherstuff
paul@laika:~/MyDir$ ls
stuff
paul@laika:~/MyDir$ cd ..
paul@laika:~$ rmdir MyDir
rmdir: MyDir/: Directory not empty
paul@laika:~$ rmdir MyDir/stuff
paul@laika:~$ rmdir MyDir

```

rmdir -p

Similar cu opțiunea **mkdir -p**, puteți folosi de asemeni **rmdir** pentru a șterge recursiv directoare.

```

paul@laika:~$ mkdir -p dir/subdir/subdir2
paul@laika:~$ rmdir -p dir/subdir/subdir2
paul@laika:~$

```

6.8. practică: lucrînd cu directoare

1. Afişaţi directorul curent.
2. Intraţi în directorul /etc.
3. Acum intraţi în directorul home folosind doar trei apăsări de taste.
4. Intraţi în directorul /boot/grub folosind doar 11 apăsări de taste.
5. Intraţi în directorul părinte al directorului curent.
6. Intraţi în directorul root.
7. Listaţi conţinutul directorului root.
8. Listaţi o listă lungă a directorului root.
9. Staţi unde sînteţi şi listaţi conţinutul /etc.
10. Staţi unde sînteţi, şi listaţi conţinutul /bin şi /sbin.
11. Staţi unde sînteţi, şi listaţi conţinutul ~ .
12. Listaţi toate fişierele (incluzînd fişierele ascunse) din directorul home.
13. Listaţi fişierele din /boot în format lizibil.
14. Creaţi un director testdir în directorul home.
15. Mergeţi în directorul /etc, staţi unde sînteţi şi creaţi directorul newdir în directorul home.
16. Creaţi cu o singură comandă directoarele ~/dir1/dir2/dir3 (dir3 este un subdirector din dir2, şi dir2 este un subdirector din dir1).
17. Ştergeţi directorul testdir.
18. Dacă timpul permite (sau dacă aşteptaţi ca ceilalţi studenţi să termine această practică), folosiţi şi înţelegeţi **pushd** şi **popd**. Folosiţi pagina de manual **bash** pentru a afla informaţii despre aceste comenzi.

6.9. soluție: lucrînd cu directoare

1. Afișați directorul curent.

```
pwd
```

2. Intrați în directorul /etc.

```
cd /etc
```

3. Acum intrați în directorul home folosind doar trei apăsări de taste.

```
cd (și tasta enter)
```

4. Intrați în directorul /boot/grub folosind doar 11 apăsări de taste.

```
cd /boot/grub (folosiți tasta tab)
```

5. Intrați în directorul părinte a directorului curent.

```
cd .. (cu spațiu între cd și ..)
```

6. Intrați în directorul root.

```
cd /
```

7. Listați conținutul directorului root.

```
ls
```

8. Listați o listă lungă a directorului root.

```
ls -l -h
```

9. Stați unde sînteți și listați conținutul /etc.

```
ls /etc
```

10. Stați unde sînteți, și listați conținutul /bin și /sbin.

```
ls /bin /sbin
```

11. Stați unde sînteți, și listați conținutul ~ .

```
ls ~
```

12. Listați toate fișierele (incluzînd fișierele ascunse) din directorul home.

```
ls -al ~
```

13. Listați fișierele din /boot în format lizibil.

```
ls -lh /boot
```

14. Creați un director testdir în directorul home.

```
mkdir ~/testdir
```

15. Mergeți în directorul /etc, stați unde sînteți și creați un director newdir în directorul home.

```
cd /etc ; mkdir ~/newdir
```

16. Creați cu o singură comandă directoarele ~/dir1/dir2/dir3 (dir3 este un subdirector din dir2, și dir2 este un subdirector din dir1).

```
mkdir -p ~/dir1/dir2/dir3
```

17. Ștergeți directorul testdir.

```
rmdir testdir
```

18. Dacă timpul permite (sau dacă așteptați ca ceilalți studenți să termine această practică), folosiți și înțelegeți **pushd** și **popd**. Folosiți pagina de manual bash pentru a afla informații despre aceste comenzi.

```
man bash
```

Shell-ul bash are două comenzi interne numite pushd și popd. Ambele comenzi lucrează cu un depozit comun a directoarelor de dinainte. pushd adaugă un director depozitului și trece pe un nou director curent, popd șterge un director din depozit și setează directorul curent.

```
paul@laika:/etc$ cd /bin
paul@laika:/bin$ pushd /lib
/lib /bin
paul@laika:/lib$ pushd /proc
/proc /lib /bin
paul@laika:/proc$
paul@laika:/proc$ popd
/lib /bin
paul@laika:/lib$
paul@laika:/lib$
paul@laika:/lib$ popd
/bin
paul@laika:/bin$
```

Capitolul 7. lucrînd cu fişiere

Conţinut

7.1.	toate fişierele sînt senzitive.	.40
7.2.	totul este un fişier.	.40
7.3.	file.	.40
7.4.	touch.	41
7.5.	rm.	.41
7.6.	cp.	.42
7.7.	mv.	.43
7.8.	rename.	.43
7.9.	practică: lucrînd cu fişiere.	.45
7.10.	soluţie: lucrînd cu fişiere.	46

În acest capitol vom învăţa cum să recunoaştem, să creăm, să copiem şi să mutăm fişiere folosind comenzi ca **file**, **touch**, **rm**, **cp**, **mv**, şi **rename**.

7.1. toate fişierele sînt senzitive

Linux e **senzitiv**, asta înseamnă că **FILE1** este diferit față de **file1**, și **/etc/hosts** este diferit față de **/etc/Hosts** (ultimul nu există pe un computer Linux tipic).

Această captură de ecran arată diferența dintre două fişiere, unul cu litera mare **W**, altul cu litera mică **w**.

```
paul@laika:~/Linux$ ls
winter.txt Winter.txt
paul@laika:~/Linux$ cat winter.txt
It is cold.
paul@laika:~/Linux$ cat Winter.txt
It is very cold!
```

7.2. totul este un fişier

Un **director** este un tip special de **fişier**, dar este totuși un **fişier** (senzitiv!). Chiar și o fereastră de terminal (**/dev/pts/4**) sau un hard-disk (**/dev/sdb**) este reprezentat undeva în **sistemul de fişiere** ca un **fişier**. Va deveni clar de-a lungul acestui curs că totul pe Linux este un **fişier**.

7.3. file

Utilitarul **file** determină tipul de fişier. Linux nu folosește extensii pentru a determina tipul de fişier. Editorul de text nu face diferența dacă un fişier se termină în **.TXT** sau **.DOC**. Ca administrator de sistem, ar trebui să folosiți comanda **file** pentru a determina tipul de fişier. Aici sînt niște exemple pe un sistem Linux tipic.

```
paul@laika:~$ file pic33.png
pic33.png: PNG image data, 3840 x 1200, 8-bit/color RGBA, non-interlaced
paul@laika:~$ file /etc/passwd
/etc/passwd: ASCII text
paul@laika:~$ file HelloWorld.c
HelloWorld.c: ASCII C program text
```

Comanda **file** folosește un fişier magic care conține tipare pentru a recunoaște tipuri de fişiere. Fişierul magic este localizat în **/usr/share/file/magic**. Tastați **man 5 magic** pentru mai multe informații.

Este interesant de subliniat comanda **file -s** pentru fişiere speciale ca acelea în **/dev** și **/proc**.

```
root@debian6~# file /dev/sda
/dev/sda: block special
root@debian6~# file -s /dev/sda
/dev/sda: x86 boot sector; partition 1: ID=0x83, active, starthead...
root@debian6~# file /proc/cpuinfo
/proc/cpuinfo: empty
root@debian6~# file -s /proc/cpuinfo
/proc/cpuinfo: ASCII C++ program text
```


7.4. touch

O modalitate ușoară de a crea un fișier este cu **touch**. (Vom vedea multe alte modalități de a crea fișiere mai târziu în această carte).

```
paul@laika:~/test$ touch file1
paul@laika:~/test$ touch file2
paul@laika:~/test$ touch file555
paul@laika:~/test$ ls -l
total 0
-rw-r--r-- 1 paul paul 0 2007-01-10 21:40 file1
-rw-r--r-- 1 paul paul 0 2007-01-10 21:40 file2
-rw-r--r-- 1 paul paul 0 2007-01-10 21:40 file555
```

touch -t

Desigur, touch poate face mai mult decît să creeze fișiere. Puteți determina ce anume privind următoarea captură de ecran? Dacă nu, căutați manualul pentru touch.

```
paul@laika:~/test$ touch -t 200505050000 SinkoDeMayo
paul@laika:~/test$ touch -t 130207111630 BigBattle
paul@laika:~/test$ ls -l
total 0
-rw-r--r-- 1 paul paul 0 1302-07-11 16:30 BigBattle
-rw-r--r-- 1 paul paul 0 2005-05-05 00:00 SinkoDeMayo
```

7.5. rm

Cînd nu mai aveți nevoie de un fișier, folosiți **rm** pentru a-l șterge. Contrar unor interfețe grafice de utilizator, linia de comandă în general nu are un *coș de gunoi* sau *recipient de gunoi* pentru a recupera fișiere. Cînd folosiți **rm** pentru a șterge un fișier, fișierul nu mai există. De aceea, aveți grijă cînd ștergeți fișiere!

```
paul@laika:~/test$ ls
BigBattle SinkoDeMayo
paul@laika:~/test$ rm BigBattle
paul@laika:~/test$ ls
SinkoDeMayo
```

rm -i

Pentru a vă preveni să nu ștergeți accidental un fișier, puteți tasta **rm -i**.

```
paul@laika:~/Linux$ touch brel.txt
paul@laika:~/Linux$ rm -i brel.txt
rm: remove regular empty file `brel.txt'? y
paul@laika:~/Linux$
```

rm -rf

Prin default, **rm -r** nu va șterge directoare care nu sînt goale. Oricum **rm** acceptă cîteva opțiuni care vă va permite să ștergeți orice director. Comanda **rm -rf** este faimoasă pentru că va șterge orice (dacă aveți permisiunea să faceți asta). Cînd

sînteți intrat în sistem ca root, fiți foarte atenți cu comanda **rm -rf** (**f** înseamnă **force** și **r** înseamnă **recursiv**) pentru că a fi root implică permisiuni care nu vi se aplică. Ați putea șterge în mod literal întregul sistem de fișier din greșeală.

```
paul@laika:~$ ls test
SinkoDeMayo
paul@laika:~$ rm test
rm: cannot remove `test': Is a directory
paul@laika:~$ rm -rf test
paul@laika:~$ ls test
ls: test: No such file or directory
```

7.6. cp

Pentru a copia un fișier, folosiți **cp** cu o sursă și un argument țintă. Dacă ținta este un director, atunci fișierele sursă sînt copiate la acel director țintă.

```
paul@laika:~/test$ touch FileA
paul@laika:~/test$ ls
FileA
paul@laika:~/test$ cp FileA FileB
paul@laika:~/test$ ls
FileA FileB
paul@laika:~/test$ mkdir MyDir
paul@laika:~/test$ ls
FileA FileB MyDir
paul@laika:~/test$ cp FileA MyDir/
paul@laika:~/test$ ls MyDir/ls
FileA
```

cp -r

Pentru a copia directoare întregi, folosiți **cp -r** (opțiunea **-r** forțează copierea **recursivă** a tuturor fișierelor în toate subdirectoarele).

```
paul@laika:~/test$ ls
FileA FileB MyDir
paul@laika:~/test$ ls MyDir/
FileA
paul@laika:~/test$ cp -r MyDir MyDirB
paul@laika:~/test$ ls
FileA FileB MyDir MyDirB
paul@laika:~/test$ ls MyDirB
FileA
```

cp fișiere multiple către un director

Puteți folosi de asemeni cp pentru a copia fișiere multiple într-un director. În acest caz, ultimul argument (ținta) trebuie să fie un director.

```
cp -r file1 file2 dir1/file3 dir1/file55 dir2
```

cp -i

Pentru a preveni cp să suprascrie fișiere existente, folosiți opțiunea -i (de la interactiv).

```
paul@laika:~/test$ cp fire water
paul@laika:~/test$ cp -i fire water
cp: overwrite `water'? no
paul@laika:~/test$
```

cp -p

Pentru a păstra permisiunile și formatul timpului din fișierele sursă, folosiți **cp -p**

```
paul@laika:~/perms$ cp file* cp
paul@laika:~/perms$ cp -p file* cpp
paul@laika:~/perms$ ll *
-rwx----- 1 paul paul 0 2008-08-25 13:26 file33
-rwxr-x--- 1 paul paul 0 2008-08-25 13:26 file42
```

```
cp:
total 0
-rwx----- 1 paul paul 0 2008-08-25 13:34 file33
-rwxr-x--- 1 paul paul 0 2008-08-25 13:34 file42
```

```
cpp:
total 0
-rwx----- 1 paul paul 0 2008-08-25 13:26 file33
-rwxr-x--- 1 paul paul 0 2008-08-25 13:26 file42
```

7.7. mv

Folosiți **mv** pentru a redenumi un fișier sau să mutați fișierul în alt director.

```
paul@laika:~/test$ touch file100
paul@laika:~/test$ ls
file100
paul@laika:~/test$ mv file100 ABC.txt
paul@laika:~/test$ ls
ABC.txt
paul@laika:~/test$
```

Cînd trebuie să redenumiți doar un singur fișier atunci **mv** este comanda preferată de utilizat.

7.8. rename

Comanda **rename** poate fi de asemeni utilizată dar are o sintaxă mai complexă pentru a autoriza redenumirea a mai multor fișiere odată. Mai jos sînt două exemple, primul schimbă toate terminațiile .txt în .png pentru toate fișierele care se termină în .txt. Al doilea exemplu schimbă toate terminațiile fișierelor cu literă mare ABC în literă mică pentru toate fișierele care se termină în .png. Următoarea sintaxă va merge pe Debian și Ubuntu (înainte de Ubuntu 7.10).

```
paul@laika:~/test$ ls
123.txt ABC.txt
paul@laika:~/test$ rename 's/txt/png/' *.txt
paul@laika:~/test$ ls
123.png ABC.png
paul@laika:~/test$ rename 's/ABC/abc/' *.png
paul@laika:~/test$ ls
123.png abc.png
paul@laika:~/test$
```

Pe Red Hat Enterprise Linux (și multe alte distribuții ca Ubuntu 8.04), sintaxa `rename` este un pic diferită. Primul exemplu de mai jos redenumeste toate fișierele `*.conf` înlocuind orice terminație `.conf` cu `.bak`. Al doilea exemplu redenumeste toate (*) fișierele înlocuind `one` cu `ONE`.

```
[paul@RHEL4a test]$ ls
one.conf two.conf
[paul@RHEL4a test]$ rename conf bak *.conf
[paul@RHEL4a test]$ ls
one.bak two.bak
[paul@RHEL4a test]$ rename one ONE *
[paul@RHEL4a test]$ ls
ONE.bak two.bak
[paul@RHEL4a test]$
```

7.9. practică: lucrând cu fișiere

1. Listați fișierele din directorul /bin.
2. Afișați tipul de fișier al /bin/cat, /etc/passwd și /usr/bin/passwd.
- 3a. Downloadați wolf.jpg și LinuxFun.pdf de la <http://linux-training.be> (wget <http://linux-training.be/files/studentfiles/wolf.jpg> și wget <http://linux-training.be/files/books/LinuxFun.pdf>)
- 3b. Afișați tipul de fișier wolf.jpg și LinuxFun.pdf.
- 3c. Redenumiți wolf.jpg în wolf.pdf (folosiți mv).
- 3d. Afișați tipul de fișier al wolf.pdf și LinuxFun.pdf.
4. Creați un director ~/touched și intrați în el.
5. Creați fișierele today.txt și yesterday.txt în touched.
6. Schimbați data yesterday.txt pentru a se potrivi cu data de ieri.
7. Copiați yesterday.txt în copy.yesterday.txt.
8. Redenumiți copy.yesterday.txt în kim.
9. Creați un director numit ~/testbackup și copiați toate fișierele din ~/touched în el.
10. Folosiți o singură comandă pentru a șterge directorul ~/testbackup și toate fișierele din el.
11. Creați un director ~/etcbackup și copiați toate fișierele *.conf din /etc în el. Ați inclus toate subdirectoarele din /etc?
12. Folosiți rename pentru a redenumi toate fișierele *.conf în *.backup. (dacă aveți mai mult de o distribuție disponibilă, încercați-le în toate!)

7.10. soluție: lucrând cu fișiere

1. Listați fișierele în directorul /bin.

```
ls /bin
```

2. Afișați tipul de fișier al /bin/cat, /etc/passwd și /usr/bin/passwd.

```
file /bin/cat /etc/passwd /usr/bin/passwd
```

3a. Downloadați wolf.jpg și LinuxFun.pdf de la <http://linux-training.be> (wget <http://linux-training.be/files/studentfiles/wolf.jpg> și wget <http://linux-training.be/files/books/LinuxFun.pdf>)

```
wget http://linux-training.be/files/studentfiles/wolf.jpg
```

```
wget http://linux-training.be/files/studentfiles/wolf.png
```

```
wget http://linux-training.be/files/books/LinuxFun.pdf
```

3b. Afișați tipul de fișier a wolf.jpg și LinuxFun.pdf.

```
file wolf.jpg LinuxFun.pdf
```

3c. Redenumiți wolf.jpg în wolf.pdf (folosiți mv).

```
mv wolf.jpg wolf.pdf
```

3d. Afișați tipul de fișier al wolf.pdf și LinuxFun.pdf.

```
file wolf.pdf LinuxFun.pdf
```

4. Creați un director ~/touched și intrați în el.

```
mkdir ~/touched ; cd ~/touched
```

5. Creați fișierele today.txt și yesterday.txt în touched.

```
touch today.txt yesterday.txt
```

6. Schimbați data yesterday.txt pentru a se potrivi cu data de ieri.

```
touch -t 200810251405 yesterday.txt (substituiți 20081025 cu data de ieri)
```

7. Copiați yesterday.txt în copy.yesterday.txt.

```
cp yesterday.txt copy.yesterday.txt
```

8. Redenumiți copy.yesterday.txt în kim.

```
mv copy.yesterday.txt kim
```

9. Creați un director numit ~/testbackup și copiați toate fișierele din ~/touched în el.

```
mkdir ~/testbackup ; cp -r ~/touched ~/testbackup/
```

10. Folosiți o singură comandă pentru a șterge directorul ~/testbackup și toate fișierele din el.

```
rm -rf ~/testbackup
```

11. Creați un director ~/etcbackup și copiați toate fișierele *.conf din /etc în el. Ați inclus toate subdirectoarele din /etc?

```
cp -r /etc/*.conf ~/etcbackup
```

Doar fișierele *.conf care sînt în mod direct în /etc/ sînt copiate.

12. Folosiți rename pentru a redenumi toate fișierele *.conf în *.backup. (dacă aveți mai mult de o distribuție disponibilă, încercați-le în toate!)

Pe RHEL: touch 1.conf 2.conf ; rename conf backup *.conf

Pe Debian: touch 1.conf 2.conf ; rename 's/conf/backup/' *.conf

Capitolul 8. lucrînd cu conținutul fișierelor

Conținut

8.1. head.	49
8.2. tail.	49
8.3. cat.	50
8.4. tac.	51
8.5. more și less.	51
8.6. strings.	51
8.7. practică: conținutul fișierelor.	53
8.8. soluție: conținutul fișierelor.	54

În acest capitol vom arunca o privire la conținutul **fișierelor text** cu **head**, **tail**, **cat**, **tac**, **more**, **less** și **șiruri (strings)**.

Vom obține de asemeni o întrezărire a posibilităților utilităreilor precum **cat** pe linia de comandă.

8.1. head

Puteți folosi **head** pentru a afișa primele zece linii ale unui fișier.

```
paul@laika:~$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
paul@laika:~$
```

Comanda head poate de asemenea să afișeze primele linii n a unui fișier.

```
paul@laika:~$ head -4 /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
```

Comanda head poate de asemenea să afișeze primul bit n.

```
paul@laika:~$ head -c4 /etc/passwd
rootpaul@laika:~$
```

8.2. tail

Similar cu comanda head, comanda **tail** va afișa ultimele 10 rînduri ale unui fișier.

```
paul@laika:~$ tail /etc/services
vboxd      20012/udp
binkp      24554/tcp    # binkp fidonet protocol
asp        27374/tcp    # Address Search Protocol
asp        27374/udp
csync2     30865/tcp    # cluster synchronization tool
dirproxy   57000/tcp    # Detachable IRC Proxy
tfido      60177/tcp    # fidonet EMSI over telnet
fido       60179/tcp    # fidonet EMSI over TCP

# Local services
paul@laika:~$
```

Puteți să dați comenzii **tail** numărul de linii pe care vreți să le vedeți.

```
$ tail -3 count.txt
six
seven
eight
```

Comanda **tail** are alte opțiuni folositoare, unele dintre ele le vom folosi de-a lungul acestui curs.

8.3. cat

Comanda **cat** este una dintre cele mai des folosite utilitare. Tot ceea ce face este să copieze intrarea standard către ieșirea standard. În combinație cu shell-ul această comandă poate fi foarte puternică și diversificată. Unele exemple vă vor da o întrezărire a posibilităților. Primul exemplu este simplu, puteți folosi **cat** pentru a afișa un fișier pe ecran. Dacă fișierul este mai lung decât ecranul, ea va afișa conținutul la sfârșit.

```
paul@laika:~$ cat /etc/resolv.conf
nameserver 194.7.1.4
paul@laika:~$
```

concatenate

cat este prescurtarea de la **concatenate**. Una dintre utilizările de bază a **cat** este să concateneze fișierele într-un fișier mai mare (sau complet).

```
paul@laika:~$ echo one > part1
paul@laika:~$ echo two > part2
paul@laika:~$ echo three > part3
paul@laika:~$ cat part1 part2 part3
one
two
three
paul@laika:~$
```

creați fișiere

Puteți folosi **cat** pentru a crea fișiere text simple. Tastați comanda **cat > winter.txt** cum este arătat în captura de ecran de mai jos. Apoi tastați una sau mai multe linii, terminând fiecare linie cu tasta enter. După ultima linie, tastați și țineți apăsată tasta Control (Ctrl) și apăsați d.

```
paul@laika:~/test$ cat > winter.txt
It is very cold today!
paul@laika:~/test$ cat winter.txt
It is very cold today!
paul@laika:~/test$
```

Combinația **Ctrl + tasta d** va trimite un semnal **EOF** (End of File) către procesul care se execută terminând comanda **cat**.

marcator de sfârșit personalizat

Puteți folosi un marcator de sfârșit pentru **cat** cu **<<** cum este arătat mai jos în acest screenshot. Această construcție se numește o **directivă here** și va termina comanda **cat**.

```
paul@laika:~/test$ cat > hot.txt <<stop
> It is hot today!
> Yes it is summer.
> stop
paul@laika:~/test$ cat hot.txt
It is hot today!
Yes it is summer.
paul@laika:~/test$
```

copiați fișiere

În al treilea exemplu puteți vedea că **cat** poate fi folosită pentru a copia fișiere. Vă vom explica în detaliu ce se întâmplă aici în capitolul bash shell.

```
paul@laika:~/test$ cat winter.txt
It is very cold today!
paul@laika:~/test$ cat winter.txt > cold.txt
paul@laika:~/test$ cat cold.txt
It is very cold today!
paul@laika:~/test$
```

8.4. tac

Doar un exemplu va arăta scopul lui **tac** (contrariul lui **cat**).

```
paul@laika:~/test$ cat count
one
two
three
four
paul@laika:~/test$ tac count
four
three
two
one
paul@laika:~/test$
```

8.5. more și less

Comanda **more** este folosită pentru a afișa fișiere care ocupă mai mult de un singur ecran. **more** vă va permite să vedeți conținutul fișierului pagină cu pagină. Folosiți tasta spațiu pentru a vedea pagina următoare, sau **q** pentru a renunța. Unii oameni preferă comanda **less** în locul comenzii **more**.

8.6. strings

Cu comanda **strings** puteți afișa șiruri lizibile ascii găsite în fișiere (binare). Acest exemplu localizează binarul **ls** apoi afișează șirurile lizibile în fișierul binar (ieșirea este scurtată).

```
paul@laika:~$ which ls
/bin/ls
paul@laika:~$ strings /bin/ls
/lib/ld-linux.so.2
librt.so.1
__gmon_start__
_Jv_RegisterClasses
clock_gettime
libacl.so.1
...
```

8.7. practică: conținutul fișierelor

1. Afișați primele 12 linii a **/etc/services**.
2. Afișați ultima linie a **/etc/passwd**.
3. Folosiți **cat** pentru a crea un fișier numit **count.txt** care arată astfel:

One
Two
Three
Four
Five
4. Folosiți **cp** pentru a face un backup a acestui fișier la **cnt.txt**.
5. Folosiți **cat** pentru a face un backup a acestui fișier la **catcnt.txt**.
6. Afișați **catcnt.txt**, dar cu toate liniile în ordine inversă (ultima linie prima).
7. Folosiți **more** pentru a afișa **/var/log/messages**.
8. Afișați șirul de caractere în clar din comanda **/usr/bin/passwd**.
9. Folosiți **ls** pentru a găsi cel mai mare fișier din **/etc**.
10. Deschideți două ferestre de terminal și asigurați-vă că sînteți în același director în ambele. Tastați **echo this is the first line > tailing.txt** în primul terminal, apoi tastați **tail -f tailing.txt** în al doilea terminal. Acum întoarceți-vă în primul terminal și tastați **echo This is another line >> tailing.txt** (observați dublul semn >>), verificați dacă **tail -f** în al doilea terminal arată ambele linii. Opriți **tail -f** cu **Ctrl-C**.
11. Folosiți **cat** pentru a crea un fișier numit **tailing.txt** care are conținutul lui **tailing.txt** urmat de conținutul lui **/etc/passwd**.
12. Folosiți **cat** pentru a crea un fișier numit **tailing.txt** care are conținutul lui **tailing.txt** precedat de conținutul lui **/etc/passwd**.

8.8. soluție: conținutul fișierelor

1. Afișați primele 12 linii a **/etc/services**.

```
head -12 /etc/services
```

2. Afișați ultima linie a **/etc/passwd**.

```
tail -1 /etc/passwd
```

3. Folosiți **cat** pentru a crea un fișier numit **count.txt** care arată astfel:

```
cat > count.txt
One
Two
Three
Four
Five (urmat de Ctrl-d)
```

4. Folosiți **cp** pentru a face un backup a acestui fișier la **cnt.txt**.

```
cp count.txt cnt.txt
```

5. Folosiți **cat** pentru a face un backup a acestui fișier la **catcnt.txt**.

```
cat count.txt > catcnt.txt
```

6. Afișați **catcnt.txt**, dar cu toate liniile în ordine inversă (ultima linie prima).

```
tac catcnt.txt
```

7. Folosiți **more** pentru a afișa **/var/log/messages**.

```
more /var/log/messages
```

8. Afișați șirul de caractere în clar din comanda **/usr/bin/passwd**.

```
strings /usr/bin/passwd
```

9. Folosiți **ls** pentru a găsi cel mai mare fișier din **/etc**.

```
ls -lrS /etc
```

10. Deschideți două ferestre de terminal și asigurați-vă că sînteți în același director în ambele. Tastați **echo this is the first line > tailing.txt** în primul terminal, apoi tastați **tail -f tailing.txt** în al doilea terminal. Acum întoarceți-vă în primul terminal și tastați **echo This is another line >> tailing.txt**, (observați dublul semn >>), verificați dacă **tail -f** în al doilea terminal arată ambele linii. Opriți **tail -f** cu **Ctrl-C**.

11. Folosiți **cat** pentru a crea un fișier numit **tailing.txt** care are conținutul lui **tailing.txt** urmat de conținutul lui **/etc/passwd**.

```
cat /etc/passwd >> tailing.txt
```

12. Folosiți **cat** pentru a crea un fișier numit **tailing.txt** care are conținutul lui **tailing.txt** precedat de conținutul lui **/etc/passwd**.

```
mv tailing.txt tmp.txt ; cat /etc/passwd tmp.txt > tailing.txt
```

Capitolul 9. arborele de fișier Linux

Conținut

9.1.	ierarhia sistemului de fișier standard.	57
9.2.	man hier.	57
9.3.	directorul root /.	57
9.4.	directoare binare.	57
9.5.	directoare de configurare.	59
9.6.	directoare de date.	61
9.7.	directoare în memorie.	63
9.8.	resurse de sistem Unix /usr.	67
9.9.	date variabile /var.	68
9.10.	practică: sistemul de fișier arborescent.	71
9.11.	soluție: sistemul de fișier arborescent.	73

Acest capitol aruncă o privire la cele mai comune directoare în **arborele de fișier Linux**. Arată de asemeni că pe Unix totul este un fișier.

9.1. ierarhia sistemului de fișier standard

Multe distribuții Linux urmăresc parțial **Standardul de Fișier-Sistem Ierarhic. FHS (Filesystem Hierarchy Standard)** poate să contureze mai multe sisteme de fișier arborescente Unix/Linux să se conformeze mai bine în viitor. **FHS** este disponibil online la <http://www.pathname.com/fhs/> unde citim: „Standardul de fișier de sistem ierarhic a fost făcut pentru a fi utilizat de dezvoltatori de distribuții Unix, dezvoltatori de pachete, și de implementatori de sistem. Oricum, este în primul rând făcut să fie o referință și nu este un tutorial despre cum să se aranjeze un fișier de sistem Unix sau o ierarhie a directoarelor”.

9.2. man hier

Există unele diferențe în sistemele de fișiere între **distribuțiile Linux**. Pentru ajutor despre mașină, tastați **man hier** pentru a afla informații despre ierarhia sistemului de fișier. Acest manual va explica structura de directoare pe computer.

9.3. directorul root /

Toate sistemele Linux au o structură de directoare care începe la **directorul rădăcină**. Directorul rădăcină este reprezentat de un **forward slash**, ca acesta: **/**. Tot ceea ce există pe sistemul Linux poate fi găsit dedesubtul acestui director root. Să ne uităm puțin la conținutul directorului root.

```
[paul@RHELv4u3 ~]$ ls /  
bin dev home media mnt proc sbin  srv tftpboot usr  
boot etc lib  misc  opt root selinux sys tmp      var
```

9.4. directoare binare

Binarele sînt fișiere care conțin cod sursă compilat (sau cod mașină). Binarele pot fi **executate** pe computer. Uneori binarele sînt numite **executabile**.

/bin

Directorul **/bin** conține **binare** pentru a fi folosite de către toți utilizatorii. Potrivit FHS directorul **/bin** ar trebui să conțină **/bin/cat** și **/bin/date** (printre altele).

În screenshot-ul de mai jos veți vedea comenzi comune Unix/Linux precum cat, cp, cpio, date, dd, echo, grep, și așa mai departe. Multe dintre acestea vor fi discutate în această carte.

```
paul@laika:~$ ls /bin
archdetect      egrep           mt              setupcon
autopartition   false          mt-gnu          sh
bash            fgconsole      mv              sh.distrib
bunzip2          fgrep           nano            sleep
bzipcat          fuser           nc              stralign
bzipcmp          fusermount      nc.traditional stty
bzdiff           get_mountoptions netcat           su
bzegrep          grep            netstat         sync
bzeze            gunzip           ntfs-3g         sysfs
bzfgrep          gzexe           ntfs-3g.probe  tailf
bzgrep           gzip            parted_devices tar
bzip2            hostname        parted_server  tempfile
bzip2recover     hw-detect       partman         touch
bzless           ip              partman-commit true
bzmore           kbd_mode        perform_recipe ulockmgr
cat              kill            pidof           umount
...
```

alte directoare /bin

Puteți găsi un **subdirector** **/bin** în multe alte directoare. Un utilizator numit **serena** poate pune propriile ei programe în **/home/serena/bin**.

Unele aplicații, deseori când sînt instalate direct din sursă se vor pune pe ele însele în **/opt**. O instalare **samba server** poate utiliza **/opt/samba/bin** pentru a-și pune spre păstrare binarele.

/sbin

/sbin conține binare pentru a configura sistemul de operare. Multe din **binarele de sistem** cer privilegiu **root** pentru a face anumite sarcini.

Mai jos există o captură de ecran care conține **binare de sistem** pentru a schimba adresa ip, pentru a partiționa un disk și pentru a crea un fișier de sistem ext4.

```
paul@ubu1010:~$ ls -l /sbin/ifconfig /sbin/fdisk /sbin/mkfs.ext4
-rwxr-xr-x 1 root root 97172 2011-02-02 09:56 /sbin/fdisk
-rwxr-xr-x 1 root root 65708 2010-07-02 09:27 /sbin/ifconfig
-rwxr-xr-x 5 root root 55140 2010-08-18 18:01 /sbin/mkfs.ext4
```

/lib

Binarele găsite în **/bin** și **/sbin** deseori folosesc **biblioteci partajate** localizate în **/lib**. Mai jos este un conținut parțial al **/lib**.

```
paul@laika:~$ ls /lib/libc*
/lib/libc-2.5.so      /lib/libcfont.so.0.0.0  /lib/libcom_err.so.2.1
/lib/libcap.so.1      /lib/libcidn-2.5.so     /lib/libconsole.so.0
/lib/libcap.so.1.10   /lib/libcidn.so.1       /lib/libconsole.so.0.0.0
/lib/libcfont.so.0    /lib/libcom_err.so.2    /lib/libcrypt-2.5.so
```

/lib/modules

În mod tipic, **kernelul Linux** încarcă modulele kernel din **/lib/modules/\$kernel-version/**. Despre acest director se discută în detaliu în capitolul kernel Linux.

/lib32 și /lib64

În mod curent sîntem în tranziție între sistemele **32-bit** și **64-bit**. De aceea, puteți întâlni directoare numite **/lib32** și **/lib64** care clarifică mărimea registru utilizată în timpul compilării bibliotecilor. Un computer pe 64-bit poate avea unele binare și biblioteci pe 32-bit pentru compabilitate cu aplicațiile. Această captură de ecran folosește utilitarul **file** pentru a demonstra diferența.

```
paul@laika:~$ file /lib32/libc-2.5.so
/lib32/libc-2.5.so: ELF 32-bit LSB shared object, Intel 80386, \
version 1 (SYSV), for GNU/Linux 2.6.0, stripped
paul@laika:~$ file /lib64/libcap.so.1.10
/lib64/libcap.so.1.10: ELF 64-bit LSB shared object, AMD x86-64, \
version 1 (SYSV), stripped
```

ELF (Executable and Linkable Format) este utilizat în aproape toate sistemele de operare asemănătoare cu Unix de cînd se folosește **System V**.

/opt

Scopul lui **/opt** este de a stoca software **opțional**. În multe cazuri acest software este din afara repozitoriului distribuției. Puteți găsi un director **/opt** gol în multe distribuții.

Un pachet mare poate să instaleze toate fișierele lui în subdirectoarele **/bin**, **/lib**, **/etc** în interiorul lui **/opt/\$numepachet/**. Dacă de exemplu pachetul se numește **wp**, atunci el se instalează în **/opt/wp**, punînd binarele în **/opt/wp/bin** și paginile de manual în **/opt/wp/man**.

9.5. directoare de configurare

/boot

Directorul **/boot** conține toate fișierele necesare pentru a buta computerul. Aceste fișiere nu se schimbă des. Pe sistemele Linux veți găsi în mod tipic directorul **/boot/grub** aici. **/boot/grub** conține **/boot/grub/grub.cfg** (sistemele mai vechi pot încă avea **/boot/grub/grub.conf**) definind meniul boot care este afișat înainte ca kernel-ul să pornească.

/etc

Toate **fișierele de configurație** specific-mașină ar trebui să fie localizate în **/etc**. În mod istoric **/etc** însemna **etcetera**, azi oamenii folosesc deseori termenul acronic **Editable Text Configuration**.

De multe ori numele unor fișiere de configurare este același cu aplicația, daemon, sau protocol, cu **.conf** adăugat ca extensie.

```

paul@laika:~$ ls /etc/*.conf
/etc/adduser.conf          /etc/ld.so.conf           /etc/scrollkeeper.conf
/etc/brltty.conf           /etc/lftp.conf            /etc/sysctl.conf
/etc/ccertificates.conf    /etc/libao.conf           /etc/syslog.conf
/etc/cvs-cron.conf         /etc/logrotate.conf       /etc/ucf.conf
/etc/ddclient.conf         /etc/ltrace.conf          /etc/uniconf.conf
/etc/debconf.conf          /etc/mke2fs.conf          /etc/updatedb.conf
/etc/deluser.conf          /etc/netscsid.conf        /etc/usplash.conf
/etc/fdmount.conf          /etc/nsswitch.conf        /etc/uswsusp.conf
/etc/hdparm.conf           /etc/pam.conf             /etc/vnc.conf
/etc/host.conf             /etc/pnm2ppa.conf         /etc/wodim.conf
/etc/inetd.conf            /etc/povray.conf          /etc/wvdial.conf
/etc/kernel-img.conf       /etc/resolv.conf
paul@laika:~$

```

Există mai multe lucruri de aflat în **/etc**.

/etc/init.d/

Multe dintre distribuțiile Unix/Linux au un director **/etc/init.d** care conține script-uri pentru a porni și opri **daemoni**. Acest director ar putea dispărea în timp ce Linux migrează spre sisteme care înlocuiesc vechea cale **init** de a porni toți **daemonii**.

/etc/X11/

Afișajul grafic (cunoscut și ca **Sistemul X Window** sau doar **X**) este condus de software de la fundația X.org. Fișierul de configurație pentru afișajul grafic este **/etc/X11/xorg.conf**.

/etc/skel/

Directorul **schelet** **/etc/skel** este copiat în directorul home al unui utilizator nou creat. În mod uzual el conține fișiere ascunse ca script-uri **.bashrc**.

/etc/sysconfig/

Acest director, care nu este menționat în FHS, conține multe fișiere configurabile **Red Hat Enterprise Linux**. Vom discuta unele dintre ele în mod detaliat. Captura de ecran de mai jos se află în directorul **/etc/sysconfig** din RHELv4u4, cu toate instalate.

```
paul@RHELv4u4:~$ ls /etc/sysconfig/
```

apmd	firstboot	irda	network	saslauthd
apm-scripts	grub	irqbalance	networking	selinux
authconfig	hidd	keyboard	ntpd	spamassassin
autofs	httpd	kudzu	openib.conf	squid
bluetooth	hwconf	lm_sensors	pand	syslog
clock	il8n	mouse	pcmcia	sys-config-sec
console	init	mouse.B	pgsql	sys-config-users
crond	installinfo	named	prelink	sys-logviewer
desktop	ipmi	netdump	rawdevices	tux
diskdump	iptables	netdump_id_dsa	rhn	vncservers
dund	iptables-cfg	netdump_id_dsa.p	samba	xinetd

```
paul@RHELv4u4:~$
```

Fișierul **/etc/sysconfig/firstboot** îi dictează Agentului de Setup Red Hat să nu ruleze în timpul butării. Dacă vreți să executați Agentul de Setup la următorul reboot, atunci ștergeți acest fișier, și executați **chkconfig -level 5 firstboot on**. Agentul de Setări Red Hat vă permite să instalați cele mai noi programe, să creați un cont de utilizator, să vă alăturați Rețelei Red Hat, și mult mai multe. Abia apoi acesta va crea fișierul **/etc/sysconfig/firstboot** din nou.

```
paul@RHELv4u4:~$ cat /etc/sysconfig/firstboot
RUN_FIRSTBOOT=NO
```

Fișierul **/etc/sysconfig/harddisks** conține unii parametri pentru a ajusta hard-disk-urile. Fișierul este explanatoriu.

Puteți vedea hardware detectat de **kudzu** în **/etc/sysconfig/hwconf**. Kudzu este software de la Red Hat pentru descoperirea și configurarea hardware-ului în mod automat.

Tipul de tastatură și limba folosită sînt setate în fișierul **/etc/sysconfig/keyboard**. Pentru mai multe informații despre tastatură și tabelul tastelor, verificați pagina de manual **keymaps(5)**, **dumpkeys(1)**, **loadkeys(1)** și directorul **/lib/kbd/keymaps**.

```
root@RHELv4u4:/etc/sysconfig# cat keyboard
KEYBOARDTYPE="pc"
KEYTABLE="us"
```

Vom discuta fișierele de rețea din acest director în capitolul despre rețele.

9.6. directoare de date

/home

Utilizatorii pot stoca sau proiecta date sub **/home**. Este comun (dar nu obligatoriu după FHS) să numim directorul home după numele utilizatorului în formatul **/home/\$USERNAME**. De exemplu:

```
paul@ubu606:~$ ls /home
geert annik sandra paul tom
```

În afară de a da fiecărui utilizator (sau fiecărui proiect sau grup) o locație pentru a stoca fișiere personale, directorul `home` al unui utilizator de asemenea servește ca o locație pentru a stoca profilul utilizatorului. Un profil de utilizator tipic Unix conține multe fișiere ascunse (fișiere a căror nume încep cu un punct). Fișierele ascunse de profil utilizator conțin setări specifice pentru acel utilizator.

```
paul@ubu606:~$ ls -d /home/paul/.*
```

/home/paul/.	/home/paul/.bash_profile	/home/paul/.ssh
/home/paul/..	/home/paul/.bashrc	/home/paul/.viminfo
/home/paul/.bash_history	/home/paul/.lessht	

/root

Pe multe sisteme, **/root** este locația default pentru date personale și profil ale **utilizatorului root**. Dacă nu există prin default, atunci unii administratori îl creează.

/srv

Puteți folosi **/srv** pentru date care sînt **deservite de sistem**. FHS permite alocarea datelor cvs, rsync, ftp și www în această locație. FHS aprobă de asemenea numiri administrative în **/srv**, ca **/srv/project55/ftp** și **/srv/sales/www**.

Pe Solaris Sun (sau Solaris Oracle) **/export** este folosit pentru acest scop.

/media

Directorul **/media** servește ca un punct de montare pentru **dispozitivele demontabile media** precum CD-ROM, camere digitale, și diferite dispozitive USB atașate. Pentru că **/media** este mai curînd nou în lumea Unix, puteți întîlni foarte bine sisteme care rulează fără acest director. Solaris 9 nu îl are, Solaris 10 îl are. Astăzi cele mai multe distribuții Linux montează toate mediile demontabile în **/media**.

```
paul@debian5:~$ ls /media/
```

cdrom	cdrom0	usbdisk
-------	--------	---------

/mnt

Directorul **/mnt** ar trebui să fie gol și ar trebui să fie utilizat doar pentru puncte de montare temporare (după FHS).

Administratorii Unix și Linux obișnuiau să creeze multe directoare aici, ca **/mnt/ceva/**. Veți întîlni probabil multe sisteme cu mai mult de un singur director creat și/sau montat în interiorul **/mnt** pentru a fi folosit pentru diferite sisteme de fișiere locale sau la distanță.

/tmp

Aplicațiile și utilizatorii ar trebui să folosească **/tmp** pentru a stoca date temporare cînd este necesar. Datele stocate în **/tmp** pot folosi fie spațiu pe disk fie RAM. Ambele sînt administrate de sistemul de operare. Nu utilizați niciodată **/tmp** pentru a stoca date care sînt importante sau pe care vreți să le arhivați.

9.7. directoare în memorie

/dev

Fișierele dispozitiv în **/dev** apar ca fiind fișiere obișnuite, dar nu sînt de fapt localizate pe hard-disk. Directorul **/dev** este populat cu fișiere în timp ce kernel-ul recunoaște hardware.

dispozitive fizice comune

Hardware comun ca dispozitive hard-disk sînt reprezentate de fișiere dispozitiv în **/dev**. Mai jos e o captură de ecran a fișierelor de dispozitiv SATA pe un laptop și apoi dispozitive IDE pe un desktop. (Înțelesul detaliat al acestor dispozitive va fi discutat mai tîrziu.)

```
#
# SATA or SCSI or USB
#
paul@laika:~$ ls /dev/sd*
/dev/sda /dev/sda1 /dev/sda2 /dev/sda3 /dev/sdb /dev/sdb1 /dev/sdb2

#
# IDE or ATAPI
#
paul@barry:~$ ls /dev/hd*
/dev/hda /dev/hda1 /dev/hda2 /dev/hdb /dev/hdb1 /dev/hdb2 /dev/hdc
```

În afară că reprezintă hardware fizic, unele fișiere dispozitiv sînt speciale. Aceste dispozitive speciale pot fi foarte utile.

/dev/tty și /dev/pts

De exemplu, **/dev/tty1** reprezintă un terminal sau o consolă atașată sistemului. (Nu vă spargeți capul cu terminologia exactă a ceea ce înseamnă 'terminal' sau 'consolă', ceea ce vrem să spunem aici este o interfață de linie de comandă.) Când tastezi comenzi într-un terminal care este parte a interfeței grafice ca Gnome sau KDE, atunci terminalul va fi reprezentat ca **/dev/pts/1** (1 poate fi un alt număr).

/dev/null

Pe Linux veți găsi un alt dispozitiv special ca **/dev/null** care poate fi considerat o gaură neagră; are spațiu nelimitat, dar nimic nu poate fi recuperat din el. Din punct de vedere tehnic vorbind, orice este scris în **/dev/null** va fi anulat. **/dev/null** poate fi folositor pentru a anula ieșiri nedorite de la comenzi. */dev/null nu este o locație bună pentru a stoca backupuri ;-).*

conversația /proc cu kernelul

/proc este un alt director special, apărînd ca fiind un fișier obișnuit, dar nu ocupă spațiu pe disk. Este de fapt o vedere asupra kernel-ului, sau mai bine, a ceea ce kernel-ul aranjează, și este o modalitate de a interacționa cu el în mod direct. **/proc** este un sistem de fișiere proc.

```
paul@RHELv4u4:~$ mount -t proc
none on /proc type proc (rw)
```

Cînd afișați directorul /proc veți vedea multe numere (pe orice Unix) și unele fișiere interesante (pe Linux).

```
mul@laika:~$ ls /proc
1      1285 1403 16    27    46    8      fs      pagetypeinfo
10     1294 1419 1607 28     47    810    interrupts partitions
103    1298 1423 1625 29     5     868    iomem    sched_debug
1034   13    1426 168   3      503   879    ioports  schedstat
1055   1303 1433 169   373   507   9      irq      scsi
107    1307 1434 17    374   514   918    kallsyms self
112    1314 1456 170   389   515   acpi    kcore    slabinfo
1185   1328 1457 171   391   56    buddyinfo keys     softirqs
1186   1332 1461 172   399   6     bus     key-users stat
1199   1339 1484 173   4      604   cgroups kmsg     swaps
12     1349 1488 174   403   614   cmdline kpagecgroup sys
1202   1373 15    175   407   615   config.gz kpagecount sysrq-trigger
1205   1384 1502 18    408   627   consoles kpageflags sysvipc
1245   1385 1510 19    411   649   cpuinfo  latency_stats thread-self
1247   1386 1515 2     418   7     crypto  loadavg   timer_list
1264   1391 1520 20    42    709   devices locks     timer_stats
1265   1393 1527 21    43    710   diskstats meminfo   tty
1269   1394 1535 22    431   781   dma      misc      uptime
1271   1395 1553 23    432   783   driver   modules   version
1275   1396 1567 24    439   784   execdomains mounts    vmallocinfo
1276   1398 1572 25    44    786   fb       mtrr      vmstat
1283   14    1593 26    45    796   filesystems net       zoneinfo
```

Să investigăm proprietățile fișierelor din interiorul **/proc**. Data și timpul vor afișa data și timpul curent indicînd fișierele care sînt aduse la zi constant (o vedere asupra kernel-ului).

```
paul@RHELv4u4:~$ date
Mon Jan 29 18:06:32 EST 2007
paul@RHELv4u4:~$ ls -al /proc/cpuinfo
-r--r--r-- 1 root root 0 Jan 29 18:06 /proc/cpuinfo
paul@RHELv4u4:~$
paul@RHELv4u4:~$ ...time passes...
paul@RHELv4u4:~$
paul@RHELv4u4:~$ date
Mon Jan 29 18:10:00 EST 2007
paul@RHELv4u4:~$ ls -al /proc/cpuinfo
-r--r--r-- 1 root root 0 Jan 29 18:10 /proc/cpuinfo
```

Multe fișiere în /proc au 0 biți, totuși ele conțin date – uneori o mulțime de date. Puteți vedea asta executînd **cat** pe fișiere ca **/proc/cpuinfo**, care conține informații despre CPU.


```

paul@RHELv4u4:~$ file /proc/cpuinfo
/proc/cpuinfo: empty
paul@RHELv4u4:~$ cat /proc/cpuinfo
processor       : 0
vendor_id      : AuthenticAMD
cpu family     : 15
model          : 43
model name     : AMD Athlon(tm) 64 X2 Dual Core Processor 4600+
stepping       : 1
cpu MHz        : 2398.628
cache size     : 512 KB
fdiv_bug       : no
hlt_bug        : no
f00f_bug       : no
coma_bug       : no
fpu            : yes
fpu_exception  : yes
cpuid level    : 1
wp             : yes
flags          : fpu vme de pse tsc msr pae mce cx8 apic mtrr pge...
bogomips       : 4803.54

```

Doar pentru a ne amuza, aici este /proc/cpuinfo pe un Sun Sunblade 1000...

```

paul@pasha:~$ cat /proc/cpuinfo
cpu : TI UltraSparc III (Cheetah)
fpu : UltraSparc III integrated FPU
promlib : Version 3 Revision 2
prom : 4.2.2
type : sun4u
ncpus probed : 2
ncpus active : 2
Cpu0Bogo : 498.68
Cpu0ClkTck : 000000002cb41780
Cpu1Bogo : 498.68
Cpu1ClkTck : 000000002cb41780
MMU Type : Cheetah
State:
CPU0: online
CPU1: online

```

Multe dintre fişierele din /proc au drepturi doar de citire, unele cer privilegii root, unele fişiere pot fi scrise, şi multe fişiere din **/proc/sys** pot fi scrise. Să discutăm unele fişiere din /proc.

/proc/interrupts

Pe arhitectura x86, **/proc/interrupts** afişează întreruperile.

```
paul@RHELv4u4:~$ cat /proc/interrupts
CPU0
0:      13876877      IO-APIC-edge timer
1:         15      IO-APIC-edge i8042
8:         1      IO-APIC-edge rtc
9:         0      IO-APIC-level acpi
12:        67      IO-APIC-edge i8042
14:       128      IO-APIC-edge ide0
15:     124320      IO-APIC-edge ide1
169:    111993      IO-APIC-level ioc0
177:     2428      IO-APIC-level eth0
NMI:         0
LOC:    13878037
ERR:         0
MIS:         0
```

Pe o mașină cu două CPU, fișierul arată astfel.

```
paul@laika:~$ cat /proc/interrupts
CPU0      CPU1
0:      860013      0 IO-APIC-edge timer
1:      4533      0 IO-APIC-edge i8042
7:         0      0 IO-APIC-edge parport0
8:     6588227      0 IO-APIC-edge rtc
10:      2314      0 IO-APIC-fasteoi acpi
12:      133      0 IO-APIC-edge i8042
14:         0      0 IO-APIC-edge libata
15:     72269      0 IO-APIC-edge libata
18:         1      0 IO-APIC-fasteoi yenta
19:    115036      0 IO-APIC-fasteoi eth0
20:    126871      0 IO-APIC-fasteoi libata, ohci1394
21:     30204      0 IO-APIC-fasteoi ehci_hcd:usb1, uhci_hcd:usb2
22:      1334      0 IO-APIC-fasteoi saa7133[0], saa7133[0]
24:    234739      0 IO-APIC-fasteoi nvidia
NMI:         72      42
LOC:    860000 859994
ERR:         0
```

/proc/kcore

Memoria fizică este reprezentată în **/proc/kcore**. Nu încercați să concatenați cu cat acest fișier, utilizați în schimb un debugger. Mărimea lui /proc/kcore este la fel cu memoria fizică din sistem, plus patru biți.

```
paul@laika:~$ ls -lh /proc/kcore
-r----- 1 root root 2.0G 2007-01-30 08:57 /proc/kcore
paul@laika:~$
```

/sys Linux 2.6 hotplugging

Directorul **/sys** a fost creat pentru kenel-ul Linux 2.6. De la versiunea 2.6, Linux folosește **sysfs** pentru a susține dispozitive hot plug **usb** și **IEEE 1394 (FireWire)**.

Vedeți paginile de manual a udev (8) (succesorul lui **devfs**) pentru mai multe informații (sau vizitați <http://linux-hotplug.sourceforge.net/>).

În mod concret directorul **/sys** conține informații kernel despre hardware.

9.8. resurse de sistem Unix /usr

Deși **/usr** este pronunțat ca user, țineți minte că el înseamnă **Unix System Resources**. Ierarhia **/usr** ar trebui să conțină date cu **permisiuni doar de citire, partajate**. Unii oameni aleg să monteze **/usr** cu permisiuni doar de citire. Asta poate fi făcut din propria lui partiție sau din partaj NFS cu permisiuni numai de citire.

/usr/bin

Directorul **/usr/bin** conține multe comenzi.

```
paul@deb508:~$ ls /usr/bin | wc -l
1395
```

(Pe Solaris directorul **/bin** are un link simbolic către **/usr/bin**.)

/usr/include

Directorul **/usr/include** conține fișiere de incluziune generale pentru C.

```
paul@ubu1010:~$ ls /usr/include/
aalib.h          expat_config.h  math.h          search.h
af_vfs.h         expat_external.h mcheck.h        semaphore.h
aio.h            expat.h         memory.h        setjmp.h
AL               fcntl.h         menu.h          sgtty.h
aliases.h        features.h      mntent.h        shadow.h
...
```

/usr/lib

Directorul **/usr/lib** conține biblioteci care nu sînt în mod direct executate de utilizatori sau de script-uri.

```
paul@deb508:~$ ls /usr/lib | head -7
4Suite
ao
apt
arj
aspell
avahi
bonobo
```

/usr/local

Directorul **/usr/local** poate fi folosit de un administrator pentru a instala software local.

```
paul@deb508:~$ ls /usr/local/
bin etc games include lib man sbin share src
paul@deb508:~$ du -sh /usr/local/
128K /usr/local/
```

/usr/share

Directorul **/usr/share** conține date independente de arhitectură. Precum puteți vedea, acesta este un director destul de mare.

```
paul@deb508:~$ ls /usr/share/ | wc -l
263
paul@deb508:~$ du -sh /usr/share/
1.3G /usr/share/
```

Acest director conține în mod tipic **/usr/share/man** pentru pagini de manual.

```
[root@debian /]# ls /usr/share/man
ar/ cs/ de/ es/ gl/ id/ ja/ ko/ man0/ man2/ man4/ man6/ man8/ nl/ pt/
ru/ sr/ tr/ uk/ zh_TW/ ca/ da/ el/ fr/ hu/ it/ jp/ lt/ man1/ man3/
man5/ man7/ mann/ pl/ pt_BR/ sl/ sv/ ug/ zh_CN/
[root@debian /]#
```

Mai conține **/usr/share/games** pentru toate datele statice de jocuri (fără scoruri și fișiere-jurnal ale jocurilor).

```
paul@ubu1010:~$ ls /usr/share/games/
openttd wesnoth
```

/usr/src

Directorul **/usr/src** este locația recomandată pentru fișierele sursă kernel.

```
paul@deb508:~$ ls -l /usr/src/
total 12
drwxr-xr-x 4 root root 4096 2011-02-01 14:43 linux-headers-2.6.26-2-686
drwxr-xr-x 18 root root 4096 2011-02-01 14:43 linux-headers-2.6.26-2-common
drwxr-xr-x 3 root root 4096 2009-10-28 16:01 linux-kbuild-2.6.26
```

9.9. date variabile /var

Fișiere care sînt nepredictibile în mărime, ca fișiere-jurnal, cache -tampon- sau fișiere spool, ar trebui să fie localizate în **/var**.

/var/log

Directorul **/var/log** folosește ca punct central pentru a conține **toate fișierele-jurnal**.

```
[root@debian /]# ls /var/log
cups/  mdm/  auth.log.2  crond.log.1  daemon.log.2  everything.log  kernel.log.1
messages.log.1  syslog.log  user.log.1  Xorg.0.log.old
hp/  old/  bttmp  crond.log.2  errors.log  everything.log.1  kernel.log.2
messages.log.2  syslog.log.1  user.log.2  journal/  auth.log  bttmp.1  daemon.log
errors.log.1  everything.log.2  lastlog  pacman.log  syslog.log.2  wtmp  debianiso/
auth.log.1  crond.log  daemon.log.1  errors.log.2  kernel.log  messages.log  pm-
powersave.log  user.log  Xorg.0.log
```

/var/log/messages

Primul fișier tipic pentru depanare pe Red Hat (și derivate ale acestuia) este **/var/log/messages**. Prin default acest fișier va conține informație a tocmai ceea ce s-a petrecut în sistem. Fișierul este numit **/var/log/syslog** pe Debian și Ubuntu.

```
[root@RHEL4b ~]# tail /var/log/messages
Jul 30 05:13:56 anacron: anacron startup succeeded
Jul 30 05:13:56 atd: atd startup succeeded
Jul 30 05:13:57 messagebus: messagebus startup succeeded
Jul 30 05:13:57 cups-config-daemon: cups-config-daemon startup succeeded
Jul 30 05:13:58 haldaemon: haldaemon startup succeeded
Jul 30 05:14:00 fstab-sync[3560]: removed all generated mount points
Jul 30 05:14:01 fstab-sync[3628]: added mount point /media/cdrom for...
Jul 30 05:14:01 fstab-sync[3646]: added mount point /media/floppy for...
Jul 30 05:16:46 sshd(pam_unix)[3662]: session opened for user paul by...
Jul 30 06:06:37 su(pam_unix)[3904]: session opened for user root by paul
```

/var/cache

Directorul **/var/cache** poate conține **date cache** pentru câteva aplicații.

```
paul@ubu1010:~$ ls /var/cache/
apt          dictionaries-common  gdm          man          software-center
binfmts      flashplugin-installer  hald         pm-utils
cups         fontconfig          jockey       pppconfig
debconf      fonts              ldconfig    samba
```

/var/spool

Directorul **/var/spool** conține în mod tipic directoare spool (bobină) pentru **mail** și **cron**, dar de asemeni este folosit ca director părinte pentru alte fișiere spool (de ex. fișiere print spool).

/var/lib

Directorul **/var/lib** conține informații despre statusul aplicației.

Red Hat Enterprise Linux de exemplu păstrează fișiere privitoare la **rpm** în **/var/lib/rpm**.

/var/...

/var de asemeni conține fișiere Process ID în **/var/run** (în curînd va fi înlocuit cu **/run**) și fișiere temporare care supraviețuiesc unui reboot în **/var/tmp** și

informații despre fișiere blocate în **/var/lock**. Vor fi mai multe exemple a utilizării **/var** mai departe în această carte.

9.10. practică: sistemul de fișier arborescent

1. Există **/bin/cat**? Dar **/bin/dd** și **/bin/echo**? Care este tipul acestor fișiere?

2. Care este mărimea fișier(-elor) kernel Linux (**vmlinu***) în **/boot**?

3. Creați un director **~/test**. Apoi scrieți următoarele comenzi:

```
cd ~/test
```

```
dd if=/dev/zero of=zeros.txt count=1 bs=100
```

```
od zeros.txt
```

dd va copia o singură dată (**count=1**) un bloc de dimensiunea de 100 bites (**bs=100**) din fișierul **/dev/zero** în **~/test/zeros.txt**. Puteți descrie funcționalitatea lui **/dev/zero**?

4. Acum scrieți următoarea comandă:

```
dd if=/dev/random of=random.txt count=1 bs=100 ; od random.txt
```

dd va copia o singură dată (**count=1**) un bloc de dimensiunea de 100 biți (**bs=100**) din fișierul **/dev/random** în **~/test/random.txt**. Puteți descrie funcționalitatea lui **/dev/random**?

5. Tastați următoarele două comenzi, și uitați-vă la primul caracter al fiecărei linii de ieșire.

```
ls -l /dev/sd* /dev/hd*
```

```
ls -l /dev/tty* /dev/input/mou*
```

Primul **ls** va arăta primele dispozitive block (b), al doilea **ls** va arăta dispozitivele caracter (c). Puteți face diferența între dispozitivele block și caracter?

6. Utilizați **cat** pentru a afișa **/etc/hosts** și **/etc/resolv.conf**. Ce credeți despre scopul acestor fișiere?

7. Există fișiere în **/etc/skel**? Căutați de asemeni fișierele ascunse.

8. Afișați **/proc/cpuinfo**. Pe care arhitectură se execută sistemul Linux?

9. Afișați **/proc/interrupts**. Care este mărimea acestui fișier? Unde este stocat fișierul?

10. Puteți intra în directorul **/root**? Există fișiere (ascunse) aici?

11. Sînt **ifconfig**, **fdisk**, **parted**, **shutdown** și **grub-install** prezente în **/sbin**? De ce există aceste binare în **/sbin** și nu în **/bin**?

12. Este **/var/log** un fișier sau un director? Ce credeți despre **/var/spool**?

13. Deschideți două prompturi de comandă (Ctrl-Shift-T în gnome-terminal) sau terminale (Ctrl-Alt-F1, Ctrl-Alt-F2, ...) și tastați **who am i** în ambele. Apoi încercați echo dintr-un terminal în altul cu un cuvânt.

14. Citiți pagina de manual a **random** și explicați diferența dintre **/dev/random** și **/dev/urandom**.

9.11. soluție: sistemul de fișier arborescent

1. Există **/bin/cat**? Dar **/bin/dd** și **/bin/echo**? Care este tipul acestor fișiere?

```
ls /bin/cat ; file /bin/cat
ls /bin/dd ; file /bin/dd
ls /bin/echo ; file /bin/echo
```

2. Care este mărimea fișier(-elor) kernel Linux (vmlinu*) în **/boot**?

```
ls -lh /boot/vm*
```

3. Creați un director **~/test**. Apoi scrieți următoarele comenzi:

```
cd ~/test
```

```
dd if=/dev/zero of=zeros.txt count=1 bs=100
```

```
od zeros.txt
```

dd va copia o singură dată (count=1) un bloc de mărimea 100 de biți (bs=100) din fișierul **/dev/zero** către **~/test/zeros.txt**. Puteți descrie funcționalitatea lui **/dev/zero**?

/dev/zero este un dispozitiv special Linux. Poate fi considerat o sursă de zerouri. Nu puteți trimite nimic către **/dev/zero** dar puteți citi zerouri din el.

4. Acum scrieți următoarea comandă:

```
dd if=/dev/random of=random.txt count=1 bs=100 ; od random.txt
```

dd va copia o dată (count=1) un bloc de mărimea de 100 biți (bs=100) din fișierul **/dev/random** în **~/test/random.txt**. Puteți descrie funcționalitatea lui **/dev/random**?

/dev/random funcționează ca un generator de numere aleatoare pe mașina Linux.

5. Tastați următoarele două comenzi, și uitați-vă la primul caracter al fiecărei linii de ieșire.

```
ls -l /dev/sd* /dev/hd*
```

```
ls -l /dev/tty* /dev/input/mou*
```

Primul **ls** va arăta primele dispozitive block (b), al doilea **ls** va arăta dispozitivele caracter (c). Puteți face diferența între dispozitivele block și caracter?

Dispozitivele block sînt întotdeauna scrise (sau citite din) block-uri. Pentru hard-disk-uri, block-uri de 512 biți sînt o realitate comună. Dispozitivele caracter acționează ca un șir de caractere (sau biți). Mausul și tastatura sînt dispozitive tipice caracter.

6. Utilizați **cat** pentru a afișa **/etc/hosts** și **/etc/resolv.conf**. Ce credeți despre scopul acestor fișiere?

/etc/hosts conține nume de mașini cu adresa lor ip

/etc/resolv.conf ar trebui să conțină adresa ip a unui server DNS.

7. Există fișiere în **/etc/skel**? Căutați de asemeni fișierele ascunse.

Tastați comanda `ls -al /etc/skel/`. Da, ar trebui să existe fișiere ascunse acolo.

8. Afișați **/proc/cpuinfo**. Pe care arhitectură se execută sistemul Linux?

Fișierul ar trebui să conțină cel puțin o linie cu Intel sau alt cpu.

9. Afișați **/proc/interrupts**. Care este mărimea acestui fișier? Unde este stocat acest fișier?

Mărimea este zero, totuși fișierul conține date. Nu este stocat nicăieri pentru că /proc este un fișier de sistem virtual care vă permite să comunicați cu kernelul. (Dacă ați răspuns stocat în memoria RAM, asta e de asemeni corect ...).

10. Puteți intra în directorul **/root**? Există fișiere (ascunse) aici?

Încercați `cd /root`. Da, există fișiere (ascunse) aici.

11. Sînt ifconfig, fdisk, parted, shutdown și grub-install prezente în **/sbin**? De ce există aceste binare în **/sbin** și nu în **/bin**?

Pentru că aceste fișiere sînt destinate doar administratorilor de sistem.

12. Este **/var/log** un fișier sau un director? Ce credeți despre **/var/spool**?

Ambele sînt directoare.

13. Deschideți două prompturi de comandă (Ctrl-Shift-T în gnome-terminal) sau terminale (Ctrl-Alt-F1, Ctrl-Alt-F2, ...) și tastați **who am i** în ambele. Apoi încercați `echo` dintr-un terminal în altul cu un cuvînt.

```
tty-terminal: echo Hello > /dev/tty1
```

```
pts-terminal: echo Hello > /dev/pts/1
```

14. Citiți pagina de manual a **random** și explicați diferența dintre **/dev/random** și **/dev/urandom**.

```
man 4 random
```

Partea III. extindere shell

Capitolul 10. comenzi și argumente

Conținut

10.1. echo.	.77
10.2. argumente.	77
10.3. comenzi.	78
10.4. aliasuri.	.79
10.5. afișarea extinderii shell.	80
10.6. practică: comenzi și argumente.	.82
10.7. soluție: comenzi și argumente.	.84

Acest capitol vă introduce în **extinderea shell-ului** privind îndeaproape la **comenzi** și **argumente**. Este important să cunoașteți **extinderea shell-ului** pentru că multe **comenzi** pe sistemul Linux sînt procesate și de cele mai multe ori schimbate de către **shell** înainte de a fi executate.

Interfața liniei de comandă sau **shell-ul** folosit pe cele mai multe sisteme Linux se numește **bash**, care provine de la **Bourne again shell**. Shell-ul **bash** încorporează avantaje din **sh** (shell-ul Bourne original), **cs**h (shell-ul C), și **ksh** (shell-ul Korn).

10.1. echo

Acest capitol folosește frecvent comanda **echo** pentru a demonstra avantajele shell-ului. Comanda **echo** este foarte simplă: ea face ecou informației primite.

```
paul@laika:~$ echo Burtonville
Burtonville
paul@laika:~$ echo Smurfs are blue
Smurfs are blue
```

10.2. argumente

Unul dintre avantajele primare ale shell-ului este că face o **scanare a liniei de comandă**. Când tastați o comandă la prompterul de comandă a shell-ului și apăsați tasta enter, atunci terminalul va începe scanarea acelei linii, împărțind-o în **argumente**. În timp ce scanează linia, shell-ul poate face multe schimbări **argumentelor** pe care le-ați tastat. Acest proces se numește **extindere shell**. Când shell-ul a terminat de scanat și decodificat acea linie, atunci ea va fi executată.

scoaterea spațiilor albe

Părțile care sînt separate de unul sau mai multe **spații albe** (sau tab-uri) consecutive sînt considerate **argumente** separate, și orice spațiu alb este scos. Primul **argument** este comanda care trebuie executată, celelalte **argumente** sînt date comenzi. Terminalul efectiv taie comanda în una sau mai multe argumente.

Asta explică de ce următoarele patru linii de comandă diferite sînt la fel după **extinderea shell-ului**.

```
[paul@RHELv4u3 ~]$ echo Hello World
Hello World
[paul@RHELv4u3 ~]$ echo Hello      World
Hello World
[paul@RHELv4u3 ~]$ echo   Hello   World
Hello World
[paul@RHELv4u3 ~]$ echo          Hello          World
Hello World
[paul@RHELv4u3 ~]$
```

Comanda **echo** va afișa fiecare argument pe care îl primește de la shell. Comanda **echo** de asemeni va pune un nou spațiu alb între argumentele pe care le-a primit.

ghilimele simple

Puteți preveni scoaterea spațiilor albe punînd ghilimele spațiilor. Conținutul șirurilor ghilimele sînt considerate ca un singur argument. În captura de ecran de mai jos **echo** primește doar un singur **argument**.

```
[paul@RHELv4u3 ~]$ echo 'A line with      single      quotes'
A line with      single      quotes
[paul@RHELv4u3 ~]$
```

ghilimele duble

Puteți de asemeni preveni ștergerea spațiilor albe punînd ghilimele duble

spațiilor. La fel ca mai sus, **echo** primește doar un singur **argument**.

```
[paul@RHELv4u3 ~]$ echo "A line with double quotes"
A line with double quotes
[paul@RHELv4u3 ~]$
```

Mai târziu în această carte, când vom discuta **variabile** vom vedea diferențe importante dintre ghilimele simple și ghilimele duble.

echo și ghilimele

Liniile dintre ghilimele pot include caractere speciale evitante recunoscute de comanda **echo** (când utilizăm **echo -e**). Captura de ecran de dedesubt arată cum să utilizăm **\n** pentru o nouă linie și **\t** pentru un tab (de obicei opt spații albe).

```
[paul@RHEL4b ~]$ echo -e "A line with \na newline"
A line with
a newline
[paul@RHEL4b ~]$ echo -e 'A line with \na newline'
A line with
a newline
[paul@RHEL4b ~]$ echo -e "A line with \ta tab"
A line with      a tab
[paul@RHEL4b ~]$ echo -e 'A line with \ta tab'
A line with      a tab
[paul@RHEL4b ~]$
```

Comanda **echo** poate genera mai mult decât spații albe, taburi și linii noi. Căutați în pagina man o listă a opțiunilor.

10.3. comenzi

comenzi externe sau interne?

Nu toate comenzile sînt externe terminalului, unele sînt **interne**. **Comenzile externe** sînt programe care au propriile lor binare și își au locul undeva în sistemul de fișier. Multe comenzi externe se află în **/bin** sau **/sbin**. Comenzile interne fac parte integrală din însuși programul shell-ului.

type

Pentru a afla dacă o anumită comandă dată shell-ului va fi executată ca o **comandă externă** sau ca o **comandă internă**, folosiți comanda **type**.

```
paul@laika:~$ type cd
cd is a shell builtin
paul@laika:~$ type cat
cat is /bin/cat
```

Precum puteți vedea, comanda **cd** este **internă**, și comanda **cat** este **externă**.

Putem de asemeni utiliza această comandă pentru a vă arăta dacă are **alias** sau nu.

```
paul@laika:~$ type ls
ls is aliased to `ls -color=auto'
```

executarea comenzilor externe

Unele comenzi au și versiuni interne și externe. Când una dintre aceste comenzi este executată, versiunile interne au prioritate. Pentru a executa versiuni externe, trebuie să scrieți întreaga cale către comandă.

```
paul@laika:~$ type -a echo
echo is a shell builtin
echo is /bin/echo
paul@laika:~$ /bin/echo Running the external echo command...
Running the external echo command...
```

which

Comanda **which** va căuta binare în variabilele mediu **\$PATH** (variabilele vor fi explicate mai târziu). În captura de ecran de mai jos, se determină că **cd** este comandă internă, și **ls**, **cp**, **rm**, **mv**, **mkdir**, **pwd**, și **which** sînt comenzi externe.

```
[root@RHEL4b ~]# which cp ls cd mkdir pwd
/bin/cp
/bin/ls
/usr/bin/which: no cd in (/usr/kerberos/sbin:/usr/kerberos/bin:...
/bin/mkdir
/bin/pwd
```

10.4. aliasuri

creați un alias

Shell-ul vă permite să creați **aliasuri**. Aliasurile sînt deseori folosite pentru a crea o modalitate mai ușoară de a reaminti un nume pentru o comandă existentă sau pentru a oferi parametri mai ușor.

```
[paul@RHELv4u3 ~]$ cat count.txt
one
two
three
[paul@RHELv4u3 ~]$ alias dog=tac
[paul@RHELv4u3 ~]$ dog count.txt
three
two
one
```

abreviați comenzi

Un **alias** poate fi la fel de folositor să abreviem o comandă existentă.

```
paul@laika:~$ alias ll='ls -lh --color=auto'
paul@laika:~$ alias c='clear'
paul@laika:~$
```

opțiuni default

Aliasurile pot fi folosite pentru a înzestra comenzile cu opțiuni default. Exemplul de mai jos arată cum să setăm opțiunea default **-i** când tastăm **rm**.

```
[paul@RHELv4u3 ~]$ touch winter.txt
[paul@RHELv4u3 ~]$ rm -i winter.txt
rm: remove regular empty file 'winter.txt'? no
[paul@RHELv4u3 ~]$ rm winter.txt
[paul@RHELv4u3 ~]$ ls winter.txt
ls: cannot access winter.txt: No such file or directory
[paul@RHELv4u3 ~]$ touch winter.txt
[paul@RHELv4u3 ~]$ alias rm='rm -i'
[paul@RHELv4u3 ~]$ rm winter.txt
rm: remove regular empty file 'winter.txt'? no
[paul@RHELv4u3 ~]$
```

Unele distribuții autorizează aliasuri default pentru a proteja utilizatorii să nu șteargă accidental fișiere ('rm -i', 'mv -i', 'cp -i').

vizualizarea aliasurilor

Puteți da unul sau mai multe aliasuri ca argumente la comanda **alias** pentru a avea definițiile lor. Nealocînd nici un argument afișează o listă completă a aliasurilor curente.

```
paul@laika:~$ alias c ll
alias c='clear'
alias ll='ls -lh --color=auto'
```

Puteți desface un alias cu comanda **unalias**.

```
[paul@RHEL4b ~]$ which rm
/bin/rm
[paul@RHEL4b ~]$ alias rm='rm -i'
[paul@RHEL4b ~]$ which rm
alias rm='rm -i'
/bin/rm
[paul@RHEL4b ~]$ unalias rm
[paul@RHEL4b ~]$ which rm
/bin/rm
[paul@RHEL4b ~]$
```

10.5 afișarea extinderii shell

Puteți afișa **extinderea shell** cu **set -x**, și să o opriți cu **set +x**. Ați putea dori să utilizați asta pe mai departe în acest curs, sau cînd aveți dubii despre ce anume exact face shell-ul cu comanda.


```
[paul@RHELv4u3 ~]$ set -x
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ echo $USER
+ echo paul
paul
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ echo \$USER
+ echo '$USER'
$USER
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ set +x
+ set +x
[paul@RHELv4u3 ~]$ echo $USER
paul
```

10.6. practică: comenzi și argumente

1. Cîte **argumente** există în această linie (nepunînd la socoteală comanda însăși).

```
touch '/etc/cron/cron.allow' 'file 42.txt' "file 32.txt"
```

2. Este **tac** o comandă internă shell?

3. Există un alias pentru **rm**?

4. Citiți pagina de manual a **rm**, fiind siguri că înțelegeți opțiunea **-i** a **rm**.
Creați și ștergeți un fișier pentru a testa opțiunea **-i**.

5. Executați: **alias rm='rm -i'**. Testați aliasul cu un fișier test. Funcționează cum vă așteptați?

6. Listați toate aliasurile curente.

7a. Creați un alias numit **'city'** cu **echo hometown**.

7b. Folosiți aliasul pentru a testa dacă funcționează.

8. Executați **set -x** pentru a afișa extinderea shell pentru fiecare comandă.

9. Testați funcționalitatea lui **set -x** executînd aliasurile **city** și **rm**.

10. Executați **set +x** pentru a opri afișarea extinderii shell.

11. Ștergeți aliasul **city**.

12. Care este locația comenzilor **cat** și **passwd**?

13. Explicați diferența dintre următoarele comenzi:

```
echo
```

```
/bin/echo
```

14. Explicați diferența dintre următoarele comenzi:

```
echo Hello
```

```
echo -n Hello
```

15. Afișați **A B C** cu două spații între B și C

(opțional) 16. Completați următoarea comandă (nu utilizați spații) pentru a afișa exact următoarea ieșire:

```
4+4      =8  
10+14    =24
```

17. Folosiți **echo** pentru a afișa exact următoarele:

```
??\
```

Găsiți două soluții cu ghilimele simple, două cu ghilimele duble și una fără ghilimele. (și spuneți mulțumesc lui Rene și Darioush de pe Google pentru acest lucru).

18. Folosiți o singură comandă **echo** pentru a afișa trei cuvinte pe trei linii.

10.7. soluție: comenzi și argumente

1. Câte **argumente** există în această linie (nepunând la socoteală comanda însăși).

```
touch '/etc/cron/cron.allow' 'file 42.txt' "file 32.txt"
```

trei

2. Este **tac** o comandă internă shell?

tastați tac

3. Există un alias pentru **rm**?

```
alias rm
```

4. Citiți pagina de manual a **rm**, fiind siguri că înțelegeți opțiunea **-i** a **rm**. Creați și ștergeți un fișier pentru a testa opțiunea **-i**.

```
man rm
```

```
touch testfile
```

```
rm -i testfile
```

5. Executați: **alias rm='rm -i'**. Testați aliasul cu un fișier test. Funcționează cum vă așteptați?

```
touch testfile
```

```
rm testfile (ar trebui să ceară confirmarea)
```

6. Listați toate aliasurile curente.

```
alias
```

7a. Creați un alias numit **'city'** cu **echo hometown**.

```
alias city='echo Antwerp'
```

7b. Folosiți aliasul pentru a testa dacă funcționează.

```
city (ar trebui să afișeze Antwerp)
```

8. Executați **set -x** pentru a afișa extinderea shell pentru fiecare comandă.

```
set -x
```

9. Testați funcționalitatea lui **set -x** executând aliasurile **city** și **rm**.

shell-ul ar trebui să afișeze aliasurile rezolvate și apoi să fie executate comenzile:

```
paul@deb503:~$ set -x
paul@deb503:~$ city
+ echo antwerp
antwerp
```

10. Executați **set +x** pentru a opri afișarea extinderii shell.

```
set +x
```

11. Ștergeți aliasul **city**.

```
unalias city
```

12. Care este locația comenzilor **cat** și **passwd**?

```
which cat (probabil /bin/cat)
```

```
which passwd (probabil /usr/bin/passwd)
```

13. Explicați diferența dintre următoarele comenzi:

```
echo
```

```
/bin/echo
```

Comanda `echo` va fi interpretată de către shell ca o comandă internă `echo`.
Comanda `/bin/echo` va face ca terminalul să execute binarul `echo` localizat în directorul `/bin`.

14. Explicați diferența dintre următoarele comenzi:

```
echo Hello
```

```
echo -n Hello
```

Opțiunea `-n` a comenzii `echo` va preveni `echo` să facă un ecou șir al unei noi linii.
`echo Hello` va face ecou a șase caractere în total, `echo -n hello` face ecou doar cinci caractere.

(Opțiunea `-n` probabil nu va funcționa în shell-ul Korn.)

15. Afișați **A B C** cu două spații între B și C.

```
echo "A B  C"
```

16. Completați următoarea comandă (nu utilizați spații) pentru a afișa exact următoarea ieșire:

```
4+4      =8
10+14    =24
```

Soluția este să utilizăm tab cu `\t`.

```
echo -e "4+4\t=8" ; echo -e "10+14\t=24"
```

17. Folosiți **echo** pentru a afișa exact următoarele:

```
??\
```

```
echo '??\'
echo -e '??\\'
echo "??\\"
echo -e "??\\"
echo ??\
```

Găsiți două soluții cu ghilimele simple, două cu ghilimele duble și una fără ghilimele. (și spuneți mulțumesc lui Rene și Darioush de pe Google pentru acest lucru).

18. Folosiți o singură comandă **echo** pentru a afișa trei cuvinte pe trei linii.

```
echo -e "one \ntwo \nthree"
```

Capitolul 11. operatori de control

Conținut

11.1.	; punct și virgulă.	.88
11.2.	semnul &.	88
11.3.	\$? dolar semnul întrebării.	88
11.4.	semnele &&.	89
11.5.	bară verticală dublă.	.89
11.6.	combinare a semnelor && 	89
11.7.	semnul #.	90
11.8.	\ caractere speciale evitante.	.90
11.9.	practică: operatori de control.	91
11.10.	soluție: operatori de control.	.92

În acest capitol punem mai mult de o singură comandă pe linia de comandă folosind **operatori de control**. Vom discuta de asemeni pe scurt parametri înrudiți (\$?) și caractere similare speciale (&).

11.1. punct și virgulă ;

Puteți pune două sau mai multe comenzi pe aceeași linie separate de punct și virgulă ; .Terminalul va scana linia până ajunge la punct și virgulă. Toate argumentele de dinainte de ; vor fi considerate o comandă separată de toate argumentele de după semnul ; . Ambele serii vor fi executate secvențial de terminal care va aștepta ca fiecare comandă să se termine înainte de a începe următoarea.

```
[paul@RHELv4u3~]$ echo Hello
Hello
[paul@RHELv4u3~]$ echo World
World
[paul@RHELv4u3~]$ echo Hello ; echo World
Hello
World
[paul@RHELv4u3~]$
```

11.2. semnul &

Când o linie se termină cu un semn &, terminalul nu va aștepta să se termine comanda. Veți avea din nou promptul shell, și comanda este executată în background. Veți primi un mesaj când această comandă s-a terminat de executat în background.

```
[paul@RHELv4u3 ~]$ sleep 20 &
[1] 7925
[paul@RHELv4u3 ~]$ sleep 20
...wait 20 seconds...
[paul@RHELv4u3 ~]$
[1]+ Done          sleep 20
```

Explicația tehnică a ceea ce se întâmplă în acest caz este scrisă în capitolul despre **procese**.

11.3. \$? dolar semnul întrebării

Codul de ieșire a comenzii anterioare este stocat într-o variabilă shell **\$?**. În fapt **\$?** este un parametru și nu o variabilă, de vreme ce nu puteți încredința o valoare către **\$?**.

```
paul@debian5:~/test$ touch file1
paul@debian5:~/test$ echo $?
0
paul@debian5:~/test$ rm file1
paul@debian5:~/test$ echo $?
0
paul@debian5:~/test$ rm file1
rm: cannot remove `file1': No such file or directory
paul@debian5:~/test$ echo $?
1
paul@debian5:~/test$
```


11.4 semnele &&

Terminalul va interpreta **&&** ca un **logic ȘI**. Când folosim **&&** a doua comandă este executată doar dacă prima are succes (întoarce un status exit zero).

```
paul@barry:~$ echo first && echo second
first
second
paul@barry:~$ zecho first && echo second
-bash: zecho: command not found
```

Un alt exemplu a aceluiași principiu **logic ȘI**. Acest exemplu începe cu un **cd** care funcționează, urmat de **ls**, apoi un **cd** non-funcțional care **nu** este urmat de **ls**.

```
[paul@RHELv4u3 ~]$ cd gen && ls
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
[paul@RHELv4u3 gen]$ cd gen && ls
-bash: cd: gen: No such file or directory
```

11.5. || bară dublă verticală

Bara dublă verticală **||** reprezintă un **logic SAU**. A doua comandă este executată doar când prima eșuează (întoarce un status exit non-zero).

```
paul@barry:~$ echo first || echo second ; echo third
first
third
paul@barry:~$ zecho first || echo second ; echo third
-bash: zecho: command not found
second
third
paul@barry:~$
```

Un alt exemplu a aceluiași principiu **logic SAU**:

```
[paul@RHELv4u3 ~]$ cd gen || ls
[paul@RHELv4u3 gen]$ cd gen || ls
-bash: cd: gen: No such file or directory
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
```

11.6. combinînd && și ||

Puteți folosi aceste expresii **logic ȘI** și **logic SAU** pentru a scrie o structură **if-then-else** pe linia de comandă. Acest exemplu folosește **echo** pentru a afișa dacă comanda **rm** a fost cu succes.

```
paul@laika:~/test$ rm file1 && echo It worked! || echo It failed!
It worked!
paul@laika:~/test$ rm file1 && echo It worked! || echo It failed!
rm: cannot remove `file1': No such file or directory
It failed!
paul@laika:~/test$
```

11.7. semnul

Tot ceea ce este scris după **semnul #** este **ignorat** de către shell. Asta este folositor pentru a scrie un **comentariu shell**, dar nu are nici o influență asupra execuției comenzii sau extinderii shell-ului.

```
paul@debian4:~$ mkdir test      # creăm un director
paul@debian4:~$ cd test        ##### intrăm în director
paul@debian4:~$ ls              # e directorul gol?
paul@debian4:~$
```

11.8. \ caractere speciale evitante

Caracterul backslash \ care permite utilizarea caracterelor control, dar fără ca terminalul să-l interpreteze, este numit **caracter evitant**.

```
[paul@RHELv4u3 ~]$ echo hello \; world
hello ; world
[paul@RHELv4u3 ~]$ echo hello\ \ \ world
hello  world
[paul@RHELv4u3 ~]$ echo escaping \\ \# \& \" \'
escaping \ # & " '
[paul@RHELv4u3 ~]$ echo escaping \\?\*\\"\'
escaping \ ? * "'
```

sfârșit de linie backslash

Liniile care se termină cu un backslash sînt continuate pe următoarea linie. Shell-ul nu va interpreta caracterul linie nouă și va aștepta pe extinderea shell și execuția liniei de comandă pînă ce o nouă linie fără backslash este întîlnită.

```
[paul@RHEL4b ~]$ echo This command line \
> is split in three \
> parts
This command line is split in three parts
[paul@RHEL4b ~]$
```

11.9. practică: operatori de control

0. Fiecare întrebare poate fi răspunsă printr-o singură linie de comandă!
1. Când tastați **passwd**, care fișier e executat?
2. Ce tip de fișier e acesta?
3. Executați comanda **pwd** de două ori (luați în vedere afirmația nr.0 de mai sus).
4. Executați **ls** după **cd /etc**, dar doar dacă **cd /etc** nu a dat eroare.
5. Executați **cd /etc** după **cd etc**, dar dacă **cd etc** eșuează.
6. Tastați **echo it worked** când **touch test42** funcționează, și **echo it failed** când **touch** a eșuat. Toate pe o singură linie de comandă ca utilizator normal (non root). Testați această linie în directorul home și în **/bin/**.
7. Executați **sleep 6**, ce face această comandă?
8. Executați **sleep 200** în background (nu așteptați să se termine comanda).
9. Scrieți o linie de comandă care execută **rm file55**. Linia de comandă ar trebui să afișeze 'success' dacă file55 e ștersă, și să afișeze 'failed' dacă a existat o problemă.
10. (opțional) Folosiți **echo** pentru a afișa "Hello World with strange' characters \
* [} ~ \." (incluzînd toate ghilimelele).

11.10. soluție: operatori de control

0. Fiecare întrebare poate fi răspunsă printr-o singură linie de comandă!

1. Când tastați **passwd**, care fișier e executat?

```
which passwd
```

2. Ce tip de fișier e acesta?

```
file /usr/bin/passwd
```

3. Executați comanda **pwd** de două ori (luați în vedere afirmația nr.0 de mai sus).

```
pwd ; pwd
```

4. Executați **ls** după **cd /etc**, dar doar dacă **cd /etc** nu a dat eroare.

```
cd /etc && ls
```

5. Executați **cd /etc** după **cd etc**, dar dacă **cd etc** eșuează.

```
cd etc || cd /etc
```

6. Tastați **echo it worked** când **touch test42** funcționează, și **echo it failed** când **touch** a eșuat. Toate pe o singură linie de comandă ca utilizator normal (non root). Testați această linie în directorul home și în **/bin/**.

```
paul@deb503:~$ cd ; touch test42 && echo it worked || echo it failed
it worked
paul@deb503:~$ cd /bin; touch test42 && echo it worked || echo it failed
touch: cannot touch `test42': Permission denied
it failed
```

7. Executați **sleep 6**, ce face această comandă?

Face pauză 6 secunde în terminal.

8. Executați **sleep 200** în background (nu așteptați să se termine comanda).

```
sleep 200 &
```

9. Scrieți o linie de comandă care execută **rm file55**. Linia de comandă ar trebui să afișeze 'success' dacă file55 e ștersă, și să afișeze 'failed' dacă a existat o problemă.

```
rm file55 && echo success || echo failed
```

10. (opțional) Folosiți **echo** pentru a afișa "Hello World with strange" characters \ * [] ~ \. (incluzând toate ghilimelele).

```
echo \"Hello World with strange\" characters \\ \\* \\[ \\} \\~ \\\\ \\ . \\\"
```

sau

```
echo \"\"Hello World with strange' characters \ * [ ] ~ \\ . \"\"
```

Capitolul 12. variabile

Conținut

12.1.	despre variabile.	95
12.2.	ghilimele.	97
12.3.	set.	97
12.4.	unset.	97
12.5.	env.	97
12.6.	export.	98
12.7.	variabile deliniate.	98
12.8.	variabile unbound.	99
12.9.	opțiuni shell.	99
12.10.	înglobare shell.	100
12.11.	practică: variabile shell.	102
12.12.	soluție: variabile shell.	103

În acest capitol vom învăța să gestionăm **variabile** de mediu în shell. Aceste **variabile** sînt deseori citite de către aplicații.

Vom arunca de asemeni o scurtă privire la **shell-uri child**, **terminale înglobate** și **opțiuni shell**.

12.1. despre variabile

semnul dolar \$

Un alt caracter important interpretat de shell este semnul dolar \$. Shell-ul va căuta o **variabilă mediu** numită ca șirul care urmează **semnului dolar** și-l va înlocui cu valoarea variabilei (sau cu nimic dacă variabila nu există).

Acestea sînt unele exemple folosind \$HOSTNAME, \$USER, \$UID, \$SHELL și \$HOME.

```
[paul@RHELv4u3 ~]$ echo This is the $SHELL shell
This is the /bin/bash shell
[paul@RHELv4u3 ~]$ echo This is $SHELL on computer $HOSTNAME
This is /bin/bash on computer RHELv4u3.localdomain
[paul@RHELv4u3 ~]$ echo The userid of $USER is $UID
The userid of paul is 500
[paul@RHELv4u3 ~]$ echo My homedir is $HOME
My homedir is /home/paul
```

taste senzitive

Acest exemplu arată că variabilele shell au taste senzitive!

```
[paul@RHELv4u3 ~]$ echo Hello $USER
Hello paul
[paul@RHELv4u3 ~]$ echo Hello $user
Hello
```

\$PS1

Variabila **\$PS1** determină promptul shell-ului. Puteți folosi caractere evitante backslash speciale ca \u pentru username sau \w pentru directorul curent. Manualul **bash** are o referință completă.

În acest exemplu schimbăm valoarea lui **\$PS1** de mai multe ori.

```
paul@deb503:~$ PS1=prompt
prompt
promptPS1='prompt '
prompt
prompt PS1='> '
>
> PS1='\u@\h$ '
paul@deb503$
paul@deb503$ PS1='\u@\h:\W$'
paul@deb503:~$
```

Pentru a evita problemele nerecuperabile, puteți seta prompterul userilor normali în verde și promptul root în roșu. Adăugați următoarele în fișierul **.bashrc** pentru un prompt verde pentru utilizator:

```
# color prompt by paul
RED='\[\033[01;31m\]'
WHITE='\[\033[01;00m\]'
GREEN='\[\033[01;32m\]'
BLUE='\[\033[01;34m\]'
export PS1="${debian_chroot:+($debian_chroot)}$GREEN\u$WHITE@$BLUE\h$WHITE\w\$ "
```

\$PATH

Variabila **\$PATH** determină locul unde shell-ul va căuta comenzi pentru a le executa (doar dacă comanda este internă sau cu alias). Variabila conține o listă de directoare, separate de coloane.

```
[[paul@RHEL4b ~]$ echo $PATH
/usr/kerberos/bin:/usr/local/bin:/bin:/usr/bin:
```

Shell-ul nu va căuta în directorul curent comenzi pe care să le execute! (Căutînd executabile în directorul curent dădea posibilitatea ușoară de a sparge computere PC-DOS). Dacă vreți ca shell-ul să caute în directorul curent, atunci adăugați un **.** la sfîrșitul \$PATH.

```
[paul@RHEL4b ~]$ PATH=$PATH:.
[paul@RHEL4b ~]$ echo $PATH
/usr/kerberos/bin:/usr/local/bin:/bin:/usr/bin:
[paul@RHEL4b ~]$
```

Calea poate fi diferită cînd utilizați su în loc de **su** – pentru că ultima va lua calea utilizatorului țintă. Utilizatorul root în mod tipic are directoarele **/sbin** adăugate la variabila \$PATH.

```
[paul@RHEL3 ~]$ su
Password:
[root@RHEL3 paul]# echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin
[root@RHEL3 paul]# exit
[paul@RHEL3 ~]$ su -
Password:
[root@RHEL3 ~]# echo $PATH
/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin:
[root@RHEL3 ~]#
```

creînd variabilie

Acest exemplu crează variabila **\$MyVar** și îi setează valoarea. Apoi folosește **echo** pentru a verifica valoarea.

```
[paul@RHELv4u3 gen]$ MyVar=555
[paul@RHELv4u3 gen]$ echo $MyVar
555
[paul@RHELv4u3 gen]$
```


12.2 ghilimele

Observați că ghilimelele duble încă permit analiza variabilelor, în timp ce ghilimele simple previn asta.

```
[paul@RHELv4u3 ~]$ MyVar=555
[paul@RHELv4u3 ~]$ echo $MyVar
555
[paul@RHELv4u3 ~]$ echo "$MyVar"
555
[paul@RHELv4u3 ~]$ echo '$MyVar'
$MyVar
```

Shell-ul bash va înlocui variabilele cu valorile lor în linii cu ghilimele duble, dar nu cu linii între ghilimele simple.

```
paul@laika:~$ city=Burtonville
paul@laika:~$ echo "We are in $city today."
We are in Burtonville today.
paul@laika:~$ echo 'We are in $city today.'
We are in $city today.
```

12.3. set

Puteți folosi comanda **set** pentru a afișa o listă de variabile mediu. Pe sistemele Ubuntu și Debian, comanda **set** va lista de asemeni funcții shell după variabilele shell. Folosiți **set | more** pentru a vedea variabilele.

12.4. unset

Folosiți comanda **unset** pentru a șterge o variabilă din mediul shell.

```
[paul@RHEL4b ~]$ MyVar=8472
[paul@RHEL4b ~]$ echo $MyVar
8472
[paul@RHEL4b ~]$ unset MyVar
[paul@RHEL4b ~]$ echo $MyVar

[paul@RHEL4b ~]$
```

12.5. env

Comanda **env** fără opțiuni va afișa o listă a **variabilelor exportate**. Diferența cu **set** cu opțiuni este că **set** listează toate variabilele, incluzând cele neexportate la shell-uri child.

Dar **env** poate fi de asemeni utilizat pentru a începe un shell liber (un shell fără nici un mediu moștenit). Comanda **env -i** curăță mediul pentru subshell.

Observați în această captură de ecran că **bash** va seta variabila **\$SHELL** la startup.

```
[paul@RHEL4b ~]$ bash -c 'echo $SHELL $HOME $USER'
/bin/bash /home/paul paul
[paul@RHEL4b ~]$ env -i bash -c 'echo $SHELL $HOME $USER'
/bin/bash
[paul@RHEL4b ~]$
```

Puteți folosi comanda **env** pentru a seta **\$LANG**, sau oricare altă variabilă, pentru o singură instanță a **bash-ului** cu o singură comandă. Exemplul de mai jos folosește asta pentru a arăta influența variabilei **\$LANG** asupra fișierelor globulare (vedeți capitolul despre fișiere globulare).

```
[paul@RHEL4b test]$ env LANG=C bash -c 'ls File[a-z]'
Filea Fileb
[paul@RHEL4b test]$ env LANG=en_US.UTF-8 bash -c 'ls File[a-z]'
Filea FileA Fileb FileB
[paul@RHEL4b test]$
```

12.6. export

Puteți exporta variabile shell către alte shell-uri cu comanda **export**. Aceasta va exporta variabila la shell-uri child.

```
[paul@RHEL4b ~]$ var3=three
[paul@RHEL4b ~]$ var4=four
[paul@RHEL4b ~]$ echo $var3 $var4
three four
[paul@RHEL4b ~]$ bash
[paul@RHEL4b ~]$ echo $var3 $var4
four
```

Dar comanda nu va exporta spre shell-ul părinte (urmează captura de ecran continuată).

```
[paul@RHEL4b ~]$ export var5=five
[paul@RHEL4b ~]$ echo $var3 $var4 $var5
four five
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ echo $var3 $var4 $var5
three four
[paul@RHEL4b ~]$
```

12.7. variabile delimitate

Pînă acum, am văzut că bash interpretează o variabilă începînd de la semnul dolar, continuînd pînă la primul eveniment a unui caracter nonalfanumeric care nu este un underscore. În anumite situații, asta poate fi o problemă. Această problemă poate fi rezolvată cu paranteze rotunde ca în acest exemplu.

```
[paul@RHEL4b ~]$ prefix=Super
[paul@RHEL4b ~]$ echo Hello $prefixman and $prefixgirl
Hello and
[paul@RHEL4b ~]$ echo Hello ${prefix}man and ${prefix}girl
Hello Superman and Supergirl
[paul@RHEL4b ~]$
```

12.8 variabile unbound

Exemplul de mai jos încearcă să afișeze valoarea variabilei **\$MyVar**, dar eșuează pentru că variabila nu există. Prin default shell-ul nu va afișa nimic când o variabilă este unbound (dezlegată) (nu există).

```
[paul@RHELv4u3 gen]$ echo $MyVar
```

```
[paul@RHELv4u3 gen]$
```

Există, însă, opțiunea shell **nounset** pe care o puteți folosi pentru a genera o eroare când o variabilă nu există.

```
paul@laika:~$ set -u
paul@laika:~$ echo $Myvar
bash: Myvar: unbound variable
paul@laika:~$ set +u
paul@laika:~$ echo $Myvar
paul@laika:~$
```

În shell-ul bash **set -u** este identic cu **set -o nounset** și la fel **set +u** este identic cu **set +o nounset**

12.9. opțiuni shell

Ambele comenzi **set** și **unset** sînt comenzi interne bash. Ele pot fi folosite pentru a seta înseși opțiunile shell-ului bash. Următorul exemplu va clarifica asta. Prin default, shell-ul va trata variabilele unset ca neavînd nici o valoare. Setînd opțiunea -u, shell-ul va trata orice referință la variabilele unset ca o eroare. Vedeți pagina de manual al bash-ului pentru mai multe informații.

```
[paul@RHEL4b ~]$ echo $var123
```

```
[paul@RHEL4b ~]$ set -u
[paul@RHEL4b ~]$ echo $var123
-bash: var123: unbound variable
[paul@RHEL4b ~]$ set +u
[paul@RHEL4b ~]$ echo $var123
```

```
[paul@RHEL4b ~]$
```

Pentru a lista toate opțiunile shell-ului, folosiți **echo \$-**. Opțiunea **noclobber** (sau **-C**) va fi explicată mai tîrziu în această carte (în capitolul redirectare I/O).

```
[paul@RHEL4b ~]$ echo $-
himBH
[paul@RHEL4b ~]$ set -C ; set -u
[paul@RHEL4b ~]$ echo $-
himuBCH
[paul@RHEL4b ~]$ set +C ; set +u
[paul@RHEL4b ~]$ echo $-
himBH
[paul@RHEL4b ~]$
```

Cînd tastați **set** fără opțiuni, primiți o listă a tuturor variabilelor fără funcție cînd shell-ul este pe mod **posix**. Puteți seta bash-ul în mod posix tastînd **set -o posix**.

12.10. înglobare shell

Terminalele pot fi înglobate în linia de comandă, sau cu alte cuvinte, scanarea liniei de comandă poate produce noi procese conținînd o ramificație a shell-ului curent. Puteți folosi variabile pentru a demonstra că noi shell-uri sînt create. În captura de ecran de mai jos, variabila \$var1 există doar în (temporarul) sub-shell.

```
[paul@RHELv4u3 gen]$ echo $var1

[paul@RHELv4u3 gen]$ echo $(var1=5;echo $var1)
5
[paul@RHELv4u3 gen]$ echo $var1

[paul@RHELv4u3 gen]$
```

Puteți îngloba un shell într-un **shell înglobat**, asta se numește **înglobarea îmbinată** a shell-urilor.

Această captură de ecran arată un shell înglobat înăuntrul unui shell înglobat.

```
paul@deb503:~$ A=shell
paul@deb503:~$ echo $C$B$A $(B=sub;echo $C$B$A; echo $(C=sub;echo $C$B$A))
shell subshell subsubshell
```

apostrofuri

Înglobarea singulară poate fi folositoare pentru a evita schimbarea directorului curent. Captura de ecran de mai jos folosește **apostrofuri** în loc de dolar-paranteză pentru a îngloba.

```
[paul@RHELv4u3 ~]$ echo `cd /etc; ls -d * | grep pass`
passwd passwd- passwd.OLD
[paul@RHELv4u3 ~]$
```

Puteți folosi doar notarea **\$()** pentru a înzestra shell-uri înglobate, **apostrofurile** nu pot face asta.

apostrofuri sau glilimele simple

Pentru a plasa înglobarea între **apostrofuri** se folosește cu un caracter mai puțin față de combinația dolar și paranteză. Fiți atenți totuși, apostrofurile sînt deseori confundate cu glilimelele simple. Diferența tehnică dintre ' și ` este semnificativă!

```
[paul@RHELv4u3 gen]$ echo `var1=5;echo $var1`  
5  
[paul@RHELv4u3 gen]$ echo 'var1=5;echo $var1'  
var1=5;echo $var1  
[paul@RHELv4u3 gen]$
```

12.11. practică: variabile shell

1. Folosiți `echo` pentru a afișa Hello urmat de numele de utilizator (folosiți o variabilă `bash`!).
2. Creați o variabilă `answer` cu o valoare de `42`.
3. Copiați valoarea lui `$LANG` în `$MyLANG`.
4. Listați toate variabilele shell curente.
5. Listați toate variabilele shell exportate.
- 6a. Comenzile `env` și `set` au afișat variabilele?
- 6b. Distrugeți variabila `answer`.
7. Căutați lista cu opțiunile shell în pagina de manual `bash`. Care este diferența dintre `set -u` și `set -o nounset`?
8. Creați două variabile, și **exportați** una dintre ele.
9. Afișați variabila exportată într-un shell child interactiv.
10. Creați o variabilă, înzestrați-o cu valoarea 'Dumb', creați o altă variabilă cu valoarea 'do'. Folosiți `echo` și cele două variabile pentru echo Dumbledore.
11. Activați `nounset` în shell. Testați dacă afișează un mesaj eroare când utilizați variabile non-existente.
12. Deactivați `nounset`.
13. Găsiți lista caracterelor evitante backslash în manualul `bash`. Adăugați timp la promptul `PS1`.
14. Executați `cd /var` și `ls` într-un shell înglobat.
15. Creați variabila `embvar` într-un shell înglobat și înzestrați-l cu `echo`. Variabila există în shell-ul curent acum?
16. Explicați ce anume face `set -x`. Comanda poate fi folositoare?
- (opțional) 17. Dat fiind următoarea captură de ecran, adăugați exact patru caractere acelei linii de comandă astfel încât ieșirea totală să fie `FirstMiddleLast`.

```
[paul@RHEL4b ~]$ echo First; echo Middle; echo Last
```
18. Afișați o **listă lungă** (`ls -l`) a comenzii `passwd` folosind comanda `which` în interiorul apostrofurilor.

12.12. soluție: variabile shell

1. Folosiți `echo` pentru a afișa Hello urmat de numele de utilizator (folosiți o variabilă `bash`!).

```
echo Hello $USER
```

2. Creați o variabilă `answer` cu o valoare de `42`.

```
answer=42
```

3. Copiați valoarea lui `$LANG` în `$MyLANG`.

```
MyLANG=$LANG
```

4. Listați toate variabilele shell curente.

```
set
```

```
set | more pe Ubuntu/Debian
```

5. Listați toate variabilele shell exportate.

```
env
```

6a. Comenzile `env` și `set` au afișat variabilele?

```
env | more
```

```
set | more
```

6b. Distrugeți variabila `answer`.

```
unset answer
```

7. Căutați lista cu opțiunile shell în pagina de manual `bash`. Care este diferența dintre `set -u` și `set -o nounset`?

Citiți manualul `bash` (`man bash`), căutați `nouset` – ambele comenzi înseamnă același lucru.

8. Creați două variabile, și **exportați** una dintre ele.

```
var1=1; export var2=2
```

9. Afișați variabila exportată într-un shell child interactiv.

```
bash
echo $var2
```

10. Creați o variabilă, înzestrați-o cu valoarea `'Dumb'`, creați o altă variabilă cu valoarea `'do'`. Folosiți `echo` și cele două variabile pentru `echo Dumbledore`.

```
varx=Dumb; vary=do
```

```
echo ${varx}le${vary}re
```

soluție de la Yves de la Dexia: `echo $varx'le'$vary're'`

soluție de la Erwin din Telenet: `echo "$varx"le"$vary"re`

11. Activați **nounset** în shell. Testați dacă afișează o eroare când utilizați variabile non-existente.

```
set -u
sau
set -o nounset
```

Ambele linii au același efect.

12. Deactivați **nounset**.

```
set +u
sau
set +o nounset
```

13. Găsiți lista caracterelor evitante backslash în manualul bash. Adăugați timp la promptul **PS1**.

```
PS1='\t \u@\h \W$ '
```

14. Executați **cd /var** și **ls** într-un shell înglobat.

```
echo $(cd /var ; ls)
```

Comanda `echo` aici este folosită doar să afișeze rezultatul comenzii `ls`. A o omite va rezulta ca shell-ul să încerce să execute primul fișier ca o comandă.

15. Creați variabila `embvar` într-un shell înglobat și înzestrați-l cu `echo`. Variabila există în shell-ul curent acum?

```
$(embvar=emb;echo $embvar) ; echo $embvar (ultimul echo eșuează).
```

`$embvar` nu există în shell-ul curent.

16. Explicați ce anume face `set -x`. Comanda poate fi folosită?

Afișează extinderea shell pentru a depana comanda.

(opțional) 17. Dat fiind următoarea captură de ecran, adăugați exact patru caractere acelei linii de comandă astfel încât ieșirea totală să fie `FirstMiddleLast`.

```
[paul@RHEL4b ~]$ echo First; echo Middle; echo Last
```

```
echo -n First; echo -n Middle; echo Last
```


17. Afişaţi o **listă lungă** (ls -l) a comenzii **passwd** folosind comanda **which** în interiorul apostrofurilor.

```
ls -l `which passwd`
```

Capitolul 13. istorie shell

Conținut

13.1.	repetarea ultimei comenzi.	107
13.2.	repetarea altor comenzi.	107
13.3.	history.	107
13.4.	!n.	107
13.5.	Ctrl-r.	107
13.6.	\$HISTSIZE.	108
13.7.	\$HISTFILE.	108
13.8.	\$HISTFILESIZE.	108
13.9.	(opțional) expresii regulate.	108
13.10	(opțional) repetarea comenzilor în ksh.	108
13.11.	practică: istorie shell.	110
13.12.	soluție: istorie shell.	111

Shell-ul ne face să repetăm comenzile ușor, acest capitol explică cum anume.

13.1. repetarea ultimei comenzi

Pentru a repeta ultima comandă în bash, tastați **!!**. Asta se pronunță **bang bang**.

```
paul@debian5:~/test42$ echo this will be repeated > file42.txt
paul@debian5:~/test42$ !!
echo this will be repeated > file42.txt
paul@debian5:~/test42$
```

13.2. repetarea altor comenzi

Puteți repeta alte comenzi folosind un singur **bang** urmat de unul sau mai multe caractere. Shell-ul va repeta ultima comandă care a început cu acele caractere.

```
paul@debian5:~/test42$ touch file42
paul@debian5:~/test42$ cat file42
paul@debian5:~/test42$ !to
touch file42
paul@debian5:~/test42$
```

13.3. history

Pentru a vedea comenzi mai vechi, folosiți **history** pentru a afișa istoria comenzilor din shell (sau folosiți **history n** pentru a vedea ultimele comenzi **n**).

```
paul@debian5:~/test$ history 10
38 mkdir test
39 cd test
40 touch file1
41 echo hello > file2
42 echo It is very cold today > winter.txt
43 ls
44 ls -l
45 cp winter.txt summer.txt
46 ls -l
47 history 10
```

13.4. !n

Când tastați **!** urmat de un număr precedat de comanda pe care o vreți să o repetați, atunci shell-ul va face ecou comenzii și o va executa.

```
paul@debian5:~/test$ !43
ls
file1 file2 summer.txt winter.txt
```

13.5. ctrl-r

O altă opțiune este de a utiliza **ctrl-r** pentru a căuta în comanda history. În captura de ecran de mai jos am tastat doar **ctrl-r** urmat de patru caractere **apti** și ea a găsit ultima comandă conținând aceste patru caractere consecutive.

```
paul@debian5:~$
(reverse-i-search)`apti': sudo aptitude install screen
```

13.6. \$HISTSIZE

Variabila \$HISTSIZE determină numărul de comenzi care vor fi reamintite în mediul de lucru curent. Cele mai multe distribuții fac default această variabilă între 500 sau 1000.

```
paul@debian5:~$ echo $HISTSIZE
500
```

Puteți schimba valoarea cu orice valoare doriți.

```
paul@debian5:~$ HISTSIZE=15000
paul@debian5:~$ echo $HISTSIZE
15000
```

13.7. \$HISTFILE

Variabila \$HISTFILE direcționează spre fișierul care conține history. Shell-ul **bash** face default această valoare către **~/.bash_history**.

```
paul@debian5:~$ echo $HISTFILE
/home/paul/.bash_history
```

O istorie a sesiunii este salvată în acest fișier când **ieșiți** din sesiune!

*Închizînd un terminal-gnome cu mausul, sau tastînd **reboot** ca root NU va salva istoria terminalului.*

13.8. \$HISTFILESIZE

Numărul comenzilor păstrate în fișierul history poate fi setat folosind \$HISTFILESIZE.

```
paul@debian5:~$ echo $HISTFILESIZE
15000
```

13.9. (opțional) expresii regulate

Este posibil să utilizăm **expresii regulate** când folosim **bang** pentru a repeta comenzi. Captura de ecran de mai jos schimbă 1 în 2.

```
paul@debian5:~/test$ cat file1
paul@debian5:~/test$ !c:s/1/2
cat file2
hello
paul@debian5:~/test$
```

13.10. (opțional) repetarea comenzilor în ksh

A repeta o comandă în **shell-ul Korn** este foarte asemănătoare. Shell-ul Korn are de asemeni comanda **history**, dar folosește litera **r** pentru a repeta linii din history.

Această captură de ecran arată comanda history. Observați înțelesul diferit al parametrului.

```
$ history 17
17 clear
18 echo hoi
19 history 12
20 echo world
21 history 17
```

Repetarea cu **r** poate fi combinată cu șirul de numere dat de comanda `history`, sau cu primele cîteva litere a comenzii.

```
$ r e
echo world
world
$ cd /etc
$ r
cd /etc
$
```

13.11. practică: istorie shell

1. Tastați comanda **echo The answer to the meaning of life, the universe and everything is 42**.
2. Repetați comanda anterioară folosind doar două caractere (există două soluții!)
3. Afișați ultimele 5 comenzi pe care le-ați tastat.
4. Tastați **echo** din întrebarea 1 din nou, folosind liniile de număr pe care le-ați primit din comanda din întrebarea 3.
5. Câte comenzi pot fi păstrate în memorie din sesiunea curentă shell?
6. Unde sînt stocate aceste comenzi cînd ieșiți din shell?
7. Câte comenzi pot fi scrise în **fișierul history** după ce ieșiți din sesiunea shell curentă?
8. Asigurați-vă că shell-ul bash curent își va aminti de următoarele 5000 de comenzi pe care le tastați.
9. Deschideți mai multe console (apăsați Ctrl-shift-t în gnome-terminal) cu același cont de utilizator. Cînd este scrisă comanda history în fișierul history?

13.12. soluție: istorie shell

1. Tastați comanda **echo The answer to the meaning of life, the universe and everything is 42**.

```
echo The answer to the meaning of life, the universe and everything is 42
```

2. Repetați comanda anterioară folosind doar două caractere (există două soluții!)

```
!!  
sau  
!e
```

3. Afișați ultimele 5 comenzi pe care le-ați tastat.

```
paul@ubu1010:~$ history 5  
52 ls -l  
53 ls  
54 df -h | grep sda  
55 echo The answer to the meaning of life, the universe and everything is 42  
56 history 5
```

Veți primi linii de număr diferite.

4. Tastați **echo** din întrebarea 1 din nou, folosind liniile de număr pe care le-ați primit din comanda din întrebarea 3.

```
paul@ubu1010:~$ !56  
echo The answer to the meaning of life, the universe and everything is 42  
The answer to the meaning of life, the universe and everything is 42
```

5. Câte comenzi pot fi păstrate în memorie din sesiunea curentă shell?

```
echo $HISTSIZE
```

6. Unde sînt stocate aceste comenzi cînd ieșiți din shell?

```
echo $HISTFILE
```

7. Câte comenzi pot fi scrise în **fișierul history** după ce ieșiți din sesiunea shell curentă?

```
echo $HISTFILESIZE
```

8. Asigurați-vă că shell-ul bash își va aminti de următoarele 5000 de comenzi pe care le tastați.

```
HISTSIZE=5000
```

9. Deschideți mai multe console (apăsați Ctrl-shift-t în gnome-terminal) cu același cont de utilizator. Cînd este scrisă comanda history în fișierul history?

Cînd tastați **exit**.

Capitolul 14. fișiere globulare

Conținut

14.1. * asterisc.	.113
14.2. ? semnul întrebării.	.113
14.3. [] paranteze pătrate.	.113
14.4. cîmpuri a-z și 0-9.	.114
14.5. \$LANG și paranteze pătrate.	.114
14.6. prevenirea fișierelor globulare.	.115
14.7. practică: shell-uri globulare.	.116
14.8. soluție: shell-uri globulare.	.117

Shell-ul este de asemeni responsabil pentru **fișiere globulare** (sau generare dinamică a numelor fișierelor). Acest capitol va explica **fișierele globulare**.

14.1. * asterisc

asteriscul * este interpretat de către shell ca un semn pentru a genera nume de fișiere, potrivit asteriscul cu orice combinație de caractere (chiar și cu nici un caracter). Când nu i se atribuie nici o cale, shell-ul va folosi nume de fișiere în directorul curent. Vedeți pagina de manual **glob(7)** pentru mai multe informații. (Aceasta face parte din subiectul LPI 1.103.3.)

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File*
File4 File55 FileA Fileab FileAB
[paul@RHELv4u3 gen]$ ls file*
file1 file2 file3 fileab fileabc
[paul@RHELv4u3 gen]$ ls *ile55
File55
[paul@RHELv4u3 gen]$ ls F*ile55
File55
[paul@RHELv4u3 gen]$ ls F*55
File55
[paul@RHELv4u3 gen]$
```

14.2. ? semnul întrebării

Similar cu asteriscul, **semnul întrebării ?** este interpretat de către shell ca un semn pentru a genera nume de fișiere, potrivit semnul întrebării cu exact un caracter.

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File?
File4 FileA
[paul@RHELv4u3 gen]$ ls Fil?4
File4
[paul@RHELv4u3 gen]$ ls Fil??
File4 FileA
[paul@RHELv4u3 gen]$ ls File??
File55 Fileab FileAB
[paul@RHELv4u3 gen]$
```

14.3 [] paranteze pătrate

Paranteza pătrată **[** este interpretată de shell ca un semn de a genera nume de fișiere, potrivit fiecare caracter dintre **[** și primul subsecvent, **]**. Ordinea în această listă dintre paranteze nu este importantă. Fiecare pereche de paranteze este înlocuită cu exact un singur caracter.

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File[5A]
FileA
[paul@RHELv4u3 gen]$ ls File[A5]
FileA
[paul@RHELv4u3 gen]$ ls File [A5] [5b]
File55
[paul@RHELv4u3 gen]$ ls File [a5] [5b]
File55 Fileab
[paul@RHELv4u3 gen]$ ls File[a5][5b][abcdefghijklm]
ls: File[a5][5b][abcdefghijklm]: No such file or directory
[paul@RHELv4u3 gen]$ ls file[a5][5b][abcdefghijklm]
fileabc
[paul@RHELv4u3 gen]$
```

Puteți de asemenea exclude caractere dintr-o listă dintre paranteze pătrate cu semnul exclamării !. Și vă este permis să faceți combinații între aceste **wild cards**.

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls file[a5][!Z]
fileab
[paul@RHELv4u3 gen]$ ls file[!5]*
file1 file2 file3 fileab fileabc
[paul@RHELv4u3 gen]$ ls file[!5]?
fileab
[paul@RHELv4u3 gen]$
```

14.4. cîmpuri a-z și 0-9

Shell-ul bash va înțelege de asemenea cîmpuri de caractere dintre paranteze.

```
[paul@RHELv4u3 gen]$ ls
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
[paul@RHELv4u3 gen]$ ls file[a-z]*
fileab fileab2 fileabc
[paul@RHELv4u3 gen]$ ls file[0-9]
file1 file2 file3
[paul@RHELv4u3 gen]$ ls file[a-z][a-z][0-9]*
fileab2
[paul@RHELv4u3 gen]$
```

14.5. \$LANG și paranteze pătrate

Dar, nu uitați influența variabilei **LANG**. Unele limbi includ caractere cu litere mici într-un cîmp de caractere mari (și invers).

```

paul@RHELv4u4:~/test$ [A-Z]ile?
file1 file2 file3 File4
paul@RHELv4u4:~/test$ ls [a-z]ile?
file1 file2 file3 File4
paul@RHELv4u4:~/test$ echo $LANG
en_US.UTF-8
paul@RHELv4u4:~/test$ LANG=C
paul@RHELv4u4:~/test$ echo $LANG
C
paul@RHELv4u4:~/test$ ls [a-z]ile?
file1 file2 file3
paul@RHELv4u4:~/test$ ls [A-Z]ile?
File4
paul@RHELv4u4:~/test$

```

14.6. prevenirea fișierelor globulare

Captura de ecran de mai jos nu ar trebui să fie o surpriză. **echo *** va produce un ecou * când este într-un director gol. Și va produce echo numelor tuturor fișierelor când directorul nu este gol.

```

paul@ubu1010:~$ mkdir test42
paul@ubu1010:~$ cd test42
paul@ubu1010:~/test42$ echo *
*
paul@ubu1010:~/test42$ touch file42 file33
paul@ubu1010:~/test42$ echo *
file33 file42

```

Globularea poate fi prevenită folosind ghilimele sau prin folosirea caracterelor evitante speciale, așa cum e prezentat în această captură de ecran.

```

paul@ubu1010:~/test42$ echo *
file33 file42
paul@ubu1010:~/test42$ echo \*
*
paul@ubu1010:~/test42$ echo '*'
*
paul@ubu1010:~/test42$ echo "*"
*

```

14.7. practică: shell-uri globulare

1. Creați un director test și intrați în el.
2. Creați fișierele file1 file10 file11 file2 File2 File3 file33 fileAB filea fileA fileAAA file(file2 (ultima are 6 caractere incluzînd un spațiu).
3. Listați (cu ls) toate fișierele care încep cu file.
4. Listați (cu ls) toate fișierele care încep cu File.
5. Listați (cu ls) toate fișierele care încep cu file și se termină cu un număr.
6. Listați (cu ls) toate fișierele care încep cu file și se termină cu o literă.
7. Listați (cu ls) toate fișierele care încep cu File și au ca număr un al cincilea caracter.
8. Listați (cu ls) toate fișierele care încep cu File și au ca număr un al cincilea caracter și nimic altceva.
9. Listați (cu ls) toate fișierele care încep cu o literă și se termină cu un număr.
10. Listați (cu ls) toate fișierele care au exact cinci caractere.
11. Listați (cu ls) toate fișierele care încep cu f sau F și se termină cu 3 sau A.
12. Listați (cu ls) toate fișierele care încep cu f și au i sau R caracter secund și se termină cu un număr.
13. Listați toate fișierele care nu încep cu litera F.
14. Copiați valoarea lui \$LANG în \$MyLANG.
15. Arătați influența lui \$LANG în listarea cîmpurilor A-Z sau a-z.
16. Ați primit informația că unul dintre servere a fost spart, atacatorul probabil a înlocuit comanda **ls**. Știți că comanda **echo** este sigur de utilizat. Poate **echo** să înlocuiască **ls**? Cum puteți lista fișierele în directorul curent cu **echo**?
17. Există o altă comandă în afară de cd pentru a schimba directoarele?

14.8. soluție: shell-uri globulare

1. Creați un director test și intrați în el.

```
mkdir testdir; cd testdir
```

2. Creați fișierele file1 file10 file11 file2 File2 File3 file33 fileAB filea fileA fileAAA file(file2 (ultima are 6 caractere incluzînd un spațiu).

```
touch file1 file10 file11 file2 File2 File3  
touch file33 fileAB filea fileA fileAAA  
touch "file("  
touch "file 2"
```

3. Listați (cu ls) toate fișierele care încep cu file.

```
ls file*
```

4. Listați (cu ls) toate fișierele care încep cu File.

```
ls File*
```

5. Listați (cu ls) toate fișierele care încep cu file și se termină cu un număr.

```
ls file*[0-9]
```

6. Listați (cu ls) toate fișierele care încep cu file și se termină cu o literă.

```
ls file*[a-z]
```

7. Listați (cu ls) toate fișierele care încep cu File și au ca număr un al cincilea caracter.

```
ls File[0-9]*
```

8. Listați (cu ls) toate fișierele care încep cu File și au ca număr un al cincilea caracter și nimic altceva.

```
ls File[0-9]
```

9. Listați (cu ls) toate fișierele care încep cu o literă și se termină cu un număr.

```
ls [a-z]*[0-9]
```

10. Listați (cu ls) toate fișierele care au exact 5 caractere.

```
ls ?????
```

11. Listați (cu ls) toate fișierele care încep cu f sau F și se termină cu 3 sau A.

```
ls [fF]*[3A]
```

12. Listați (cu ls) toate fișierele care încep cu f și au i sau R caracter secund și se termină cu un număr.

```
ls f[!R]*[0-9]
```

13. Listați toate fișierele care nu încep cu litera F.

```
ls [!F]*
```

14. Copiați valoarea lui \$LANG în \$MyLANG.

```
MyLANG=$LANG
```

15. Arătați influența lui \$LANG în listarea câmpurilor A-Z sau a-z.

Vedeți exemplul din carte.

16. Ați primit informația că unul dintre servere a fost spart, atacatorul probabil a înlocuit comanda **ls**. Știți că comanda **echo** este sigur de utilizat. Poate **echo** să înlocuiască **ls**? Cum puteți lista fișierele în directorul curent cu **echo**?

```
echo *
```

17. Există o altă comandă în afară de cd pentru a schimba directoarele?

```
pushd popd
```

Partea IV. conducte și comenzi

Capitolul 15. redirectare și conducte

Conținut

15.1.	stdin, stdout, și stderr.	.121
15.2.	redirectare ieșire.	.121
15.3.	redirectare eroare.	.122
15.4.	redirectare intrare.	.123
15.5.	redirectare greșită.	.123
15.6.	curățare rapidă a fișierului.	.124
15.7.	swap stdout și stderr.	.124
15.8.	conduce.	.124
15.9.	practică: redirectare și conducte.	.126
15.10.	soluție: redirectare și conducte.	.127

Una dintre puterile liniei de comandă Unix este folosirea **redirectării** și a **conductelor**.

Acest capitol explică mai întâi **redirectarea** intrării, ieșire și eroare. Apoi vă introduce în **conduce** care constau din câteva **comenzi**.

15.1. stdin, stdout, și stderr

Shell-ul (și aproape orice altă comandă Linux) ia intrare din **stdin** (curs 0) și trimite ieșire către **stdout** (curs 1) și mesaje de eroare către **stderr** (curs 2).

Tastatura deseori este folosită ca **stdin**, **stdout** și **stderr** ambele mergînd spre afișare. Shell-ul permite întotdeauna să redirectați aceste cursuri.

15.2. redirectarea ieșirii

> stdout

stdout poate fi redirectat cu semnul **mai mare** >. În timp ce scanează linia, shell-ul va vedea semnul > și va curăța fișierul.

```
[paul@RHELv4u3 ~]$ echo It is cold today!
It is cold today!
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$
```

Observați că notarea > este de fapt abrevierea lui **1>** (**stdout** făcînd referire la el precum curs 1).

ieșirea fișierului este ștearsă

Repetăm: în timp ce scanează linia, shell-ul va vedea semnul > și **va curăța fișierul**! Asta înseamnă că chiar dacă comanda eșuează, fișierul va fi curățat!

```
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ zcho It is cold today! > winter.txt
bash: zcho: command not found
[paul@RHELv4u3 ~]$ cat winter.txt
[paul@RHELv4u3 ~]$
```

noclobber

Ștergerea unui fișier în timp ce folosim > poate fi prevenită setînd opțiunea **noclobber**.

```
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ set -o noclobber
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
-bash: winter.txt: cannot overwrite existing file
[paul@RHELv4u3 ~]$ set +o noclobber
[paul@RHELv4u3 ~]$
```

suprascrisiere noclobber

noclobber poate fi suprascris cu >|

```
[paul@RHELv4u3 ~]$ set -o noclobber
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
-bash: winter.txt: cannot overwrite existing file
[paul@RHELv4u3 ~]$ echo It is very cold today! >| winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is very cold today!
[paul@RHELv4u3 ~]$
```

>> aplicare

Folosiți >> pentru a **aplica** ieșirea într-un fișier.

```
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ echo Where is the summer ? >> winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
Where is the summer ?
[paul@RHELv4u3 ~]$
```

15.3. redirectarea erorilor

2> stderr

Redirectarea **stderr** se face cu **2>**. Aceasta poate fi foarte folositor pentru a preveni mesajele de eroare să umple ecranul. Captura de ecran de mai jos arată redirectarea lui **stdout** într-un fișier, și **stderr** în **/dev/null**. A scrie **1>** e la fel ca **>**.

```
[paul@RHELv4u3 ~]$ find / > allfiles.txt 2> /dev/null
[paul@RHELv4u3 ~]$
```

2>&1

Pentru a redirecta și **stdout** și **stderr** în același fișier, folosiți **2>&1**.

```
[paul@RHELv4u3 ~]$ find / > allfiles_and_errors.txt 2>&1
[paul@RHELv4u3 ~]$
```

Notați că ordinea redirectărilor este importantă. De exemplu, comanda

```
ls > dirlist 2>&1
```

îndreaptă și ieșirea standard (descriptor de fișier 1) și eroarea standard (descriptor de fișier 2) către fișierul `dirlist`, în timp ce comanda

```
ls 2>&1 > dirlist
```

îndreaptă doar ieșirea standard către fișierul `dirlist`, pentru că eroarea standard a făcut o copie a ieșirii standard înainte ca ieșirea standard să fie redirectată în `dirlist`.

15.4. redirectarea intrării

< stdin

Redirectarea **stdin** se face cu < (prescurtare de la 0<).

```
[paul@RHEL4b ~]$ cat < text.txt
one
two
[paul@RHEL4b ~]$ tr 'onetw' 'ONEZZ' < text.txt
ONE
ZZO
[paul@RHEL4b ~]$
```

<< here document

here document (uneori numit here-is-document) este o modalitate de a adăuga intrarea pînă ce o anumită secvență (de obicei EOF) este întîlnită. Markerul **EOF** poate fi tastat literal sau poate fi invocat cu Ctrl-D.

```
[paul@RHEL4b ~]$ cat <<EOF > text.txt
> one
> two
> EOF
[paul@RHEL4b ~]$ cat text.txt
one
two
[paul@RHEL4b ~]$ cat <<bro1 > text.txt
> bre1
> bro1
[paul@RHEL4b ~]$ cat text.txt
bre1
[paul@RHEL4b ~]$
```

<<< șirul here

șirul here poate fi folosit pentru a trece șiruri în mod direct către o comandă. Rezultatul e același ca folosind **echo șir | comandă** (dar aveți un proces mai puțin care se execută).

```
paul@ubu1110~$ base64 <<< linux-training.be
bGludXgt dHJhaW5pbmcuYmUK
paul@ubu1110~$ base64 -d <<< bGludXgt dHJhaW5pbmcuYmUK
linux-training.be
```

Vedeți rfc 3548 pentru mai multe informații despre **base64**.

15.5. redirectare greșită

Shell-ul va scana întreaga linie înainte de a aplica redirectarea. Următoarea linie de comandă este foarte lizibilă și este corectă.

```
cat winter.txt > snow.txt 2> errors.txt
```

Dar aceasta de mai jos este de asemeni corectă, dar mai puțin lizibilă.

```
2> errors.txt cat winter.txt > snow.txt
```

Chiar și asta va fi înțeleasă în mod perfect de către shell.

```
< winter.txt > snow.txt 2> errors.txt cat
```

15.6. curățarea rapidă a fișierului

Deci care este cea mai rapidă cale de a curăța un fișier?

```
>foo
```

Și care este cea mai rapidă modalitate de a curăța un fișier când opțiunea **noclobber** este setată?

```
>|bar
```

15.7. swap stdout și stderr

Cînd filtrăm un curs ieșire, de ex. prin o conductă regulată (|) se poate filtra numai **stdout**. Să spunem că vreți să filtrați o eroare neimportantă, în afara cursului **stderr**. Asta nu poate fi făcut în mod direct, și trebuie să schimbați **stdout** și **stderr**. Asta poate fi făcut prin folosirea unui al 4-lea curs la care se face referință cu numărul 3:

```
3>&1 1>&2 2>&3
```

Această construcție tip Turn Hanoi folosește un curs temporar 3, pentru a fi capabil să facă swap **stdout** (1) și **stderr** (2). Următoarele sînt un exemplu a cum să filtrăm toate liniile în cursul **stderr**, conținînd \$error.

```
$command 3>&1 1>&2 2>&3 | grep -v $error 3>&1 1>&2 2>&3
```

Dar în acest exemplu, asta poate fi făcut pe o cale mult mai scurtă, folosind o conductă pe STDERR:

```
/usr/bin/$somecommand |& grep -v $error
```

15.8. conducte

Unul dintre cele mai puternice avantaje a **Linux-ului** este folosirea **conductelor**.

O conductă ia **stdout** din comanda precedentă și o trimite ca **stdin** următoarei comenzi. Toate comenzile într-o **conductă** rulează simultan.

| bară verticală

Luați în considerare următorul exemplu.

```
paul@debian5:~/test$ ls /etc > etcfiles.txt
paul@debian5:~/test$ tail -4 etcfiles.txt
X11
xdg
xml
xpdf
paul@debian5:~/test$
```

Asta poate fi scris cu o singură linie de comandă folosind o **conductă**.

```
paul@debian5:~/test$ ls /etc | tail -4
X11
xdg
xml
xpdf
paul@debian5:~/test$
```

Conducta este reprezentată de o bară verticală | între cele două comenzi.

conducte multiple

O linie de comandă poate utiliza **conducte** multiple. Toate comenzile din **conductă** pot să se execute în același timp.

```
paul@deb503:~/test$ ls /etc | tail -4 | tac
xpdf
xml
xdg
X11
```

15.9. practică: redirectare și conducte

1. Folosiți **ls** pentru a obține ieșirea conținutului directorului **/etc/** într-un fișier numit **etc.txt**.
2. Activați opțiunea shell **noclobber**.
3. Verificați dacă **noclobber** este activ repetând comanda **ls** pe **/etc/**.
4. Când listăm toate opțiunile shell, care caracter reprezintă opțiunea **noclobber**?
5. Dezactivați opțiunea **noclobber**.
6. Asigurați-vă că aveți două shell-uri deschise pe același computer. Creați un fișier gol **tailing.txt**. Apoi tastați **tail -f tailing.txt**. Folosiți al doilea shell pentru a **adăuga** o linie de text către acel fișier. Verificați dacă primul terminal afișează această linie.
7. Creați un fișier care conține numele a cinci oameni. Folosiți **cat** și redirectarea ieșirii pentru a crea fișierul și folosiți un **here document** pentru a termina intrarea.

15.10. soluție: redirectare și conducte

1. Folosiți **ls** pentru a obține ieșirea conținutului directorului **/etc/** într-un fișier numit **etc.txt**.

```
ls /etc > etc.txt
```

2. Activați opțiunea shell **noclobber**.

```
set -o noclobber
```

3. Verificați dacă **noclobber** este activ repetând comanda **ls** pe **/etc/**.

```
ls /etc > etc.txt (nu ar trebui să meargă)
```

4. Când listăm toate opțiunile shell, care caracter reprezintă opțiunea **noclobber**?

```
echo $- (noclobber este vizibil ca litera C)
```

5. Deactivați opțiunea **noclobber**.

```
set +o noclobber
```

6. Asigurați-vă că aveți două shell-uri deschise pe același computer. Creați un fișier gol **tailing.txt**. Apoi tastați **tail -f tailing.txt**. Folosiți al doilea shell pentru a **adăuga** o linie de text către acel fișier. Verificați dacă primul terminal afișează această linie.

```
paul@deb503:~$ > tailing.txt
paul@deb503:~$ tail -f tailing.txt
hello
world
```

într-un alt shell:

```
paul@deb503:~$ echo hello >> tailing.txt
paul@deb503:~$ echo world >> tailing.txt
```

7. Creați un fișier care conține numele a cinci oameni. Folosiți **cat** și redirectarea ieșirii pentru a crea fișierul și folosiți un **here document** pentru a termina intrarea.

```
paul@deb503:~$ cat > tennis.txt << ace
> Justine Henin
> Venus Williams
> Serena Williams
> Martina Hingis
> Kim Clijsters
> ace
paul@deb503:~$ cat tennis.txt
Justine Henin
Venus Williams
Serena Williams
Martina Hingis
Kim Clijsters
paul@deb503:~$
```


Capitolul 16. filtre

Conținut

16.1.	cat.	130
16.2.	tee.	130
16.3.	grep.	130
16.4.	cut.	132
16.5.	tr.	132
16.6.	wc.	134
16.7.	sort.	134
16.8.	uniq.	135
16.9.	comm.	135
16.10.	od.	136
16.11.	sed.	137
16.12.	exemple de conducte.	137
16.13.	practică: filtre.	139
16.14.	soluție: filtre.	140

Comenzile care sînt create pentru a fi utilizate cu o **conductă** sînt deseori numite **filtre**. Aceste **filtre** sînt programe foarte mici care fac un singur lucru specific foarte eficient. Ele pot fi folosite ca **blocuri de construcție**.

Acest capitol vă va introduce în cele mai comune **filtre**. Combinarea comenzilor simple și a filtrelor într-o **conductă** lungă vă permite să creați soluții elegante.

16.1. cat

Cînd este între două **conducte**, comanda **cat** nu face nimic (cu excepția că pune **stdin** în **stdout**).

```
[paul@RHEL4b pipes]$ tac count.txt | cat | cat | cat | cat | cat
five
four
three
two
one
[paul@RHEL4b pipes]$
```

16.2. tee

A scrie **conducte** lungi în Unix este amuzant, dar uneori ați dori rezultate ajutătoare. Aici **tee** vine în ajutor. Filtrul **tee** pune **stdin** în **stdout** și de asemenea într-un fișier. Astfel **tee** este aproape la fel ca și comanda **cat**, cu excepția că are două ieșiri identice.

```
[paul@RHEL4b pipes]$ tac count.txt | tee temp.txt | tac
one
two
three
four
five
[paul@RHEL4b pipes]$ cat temp.txt
five
four
three
two
one
[paul@RHEL4b pipes]$
```

16.3. grep

Filtrul **grep** este faimos printre utilizatorii de Unix. Cea mai comună utilizare a **grep** este să filtreze linii de text conținând (sau neconținând) un anumit șir.

```
[paul@RHEL4b pipes]$ cat tennis.txt
Amelie Mauresmo, Fra
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
Venus Williams, USA
[paul@RHEL4b pipes]$ cat tennis.txt | grep Williams
Serena Williams, usa
Venus Williams, USA
```

Puteți scrie asta fără comanda **cat**.

```
[paul@RHEL4b pipes]$ grep Williams tennis.txt
Serena Williams, usa
Venus Williams, USA
```

Una dintre cele mai folositoare opțiuni a comenzii **grep** este **grep -i** care filtrează într-o modalitate insensitivă.

```
[paul@RHEL4b pipes]$ grep Bel tennis.txt
Justine Henin, Bel
[paul@RHEL4b pipes]$ grep -i Bel tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
[paul@RHEL4b pipes]$
```

O altă opțiune foarte folositoare este **grep -v** care face ieșire liniilor care nu se potrivesc șirului.

```
[paul@RHEL4b pipes]$ grep -v Fra tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
Venus Williams, USA
[paul@RHEL4b pipes]$
```

Și desigur, ambele opțiuni pot fi combinate pentru a filtra toate liniile care nu conțin un șir insensitiv.

```
[paul@RHEL4b pipes]$ grep -vi usa tennis.txt
Amelie Mauresmo, Fra
Kim Clijsters, BEL
Justine Henin, Bel
[paul@RHEL4b pipes]$
```

Cu **grep -A1** o linie **după** rezultat este de asemeni afișată.

```
paul@debian5:~/pipes$ grep -A1 Henin tennis.txt
Justine Henin, Bel
Serena Williams, usa
```

Cu **grep -B1** o linie **înainte** de rezultat este afișată de asemeni.

```
paul@debian5:~/pipes$ grep -B1 Henin tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
```

Cu **grep -C1** (context) sînt afișate de asemeni o linie **înainte** și una **după**. Toate cele trei opțiuni (A, B, și C) pot afișa orice număr de linii (folosind de ex. A2, B4 sau C20).

```
paul@debian5:~/pipes$ grep -C1 Henin tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
```

16.4. cut

Filtrul **cut** poate selecta coloane din fișiere, depinzînd de un delimitator sau un număr de biți. Captura de ecran de mai jos folosește **cut** pentru a filtra usernameul și userid în fișierul **/etc/passwd**. Folosește două puncte ca un delimitator, și selectează cîmpurile 1 și 3.

```
[paul@RHEL4b pipes]$ cut -d: -f1,3 /etc/passwd | tail -4
Figo:510
Pfaff:511
Harry:516
Hermione:517
[paul@RHEL4b pipes]$
```

Cînd folosim un spațiu ca delimitator pentru **cut**, trebuie să puneți în ghilimele spațiul.

```
[paul@RHEL4b pipes]$ cut -d" " -f1 tennis.txt
Amelie
Kim
Justine
Serena
Venus
[paul@RHEL4b pipes]$
```

Acest exemplu folosește **cut** pentru a afișa al doilea pînă la al șaptelea caracter din **/etc/passwd**.

```
[paul@RHEL4b pipes]$ cut -c2-7 /etc/passwd | tail -4
igo:x:
faff:x
arry:x
ermion
[paul@RHEL4b pipes]$
```

16.5. tr

Puteți translata caractere cu **tr**. Captura de ecran arată translarea tuturor incidentelor lui e la E.

```
[paul@RHEL4b pipes]$ cat tennis.txt | tr 'e' 'E'
AmELiE MaurEsmo, Fra
Kim CLijstErs, BEL
JustinE HEnin, BEL
SErEna Williams, usa
VEnus Williams, USA
```

Aici setăm toate literele la literă mare definind două cîmpuri.

```
[paul@RHEL4b pipes]$ cat tennis.txt | tr 'a-z' 'A-Z'
AMELIE MAURESMO, FRA
KIM CLIJSTERS, BEL
JUSTINE HENIN, BEL
SERENA WILLIAMS, USA
VENUS WILLIAMS, USA
[paul@RHEL4b pipes]$
```

Aici transformăm toate liniile noi la spații.

```
[paul@RHEL4b pipes]$ cat count.txt
one
two
three
four
five
[paul@RHEL4b pipes]$ cat count.txt | tr '\n' ' '
one two three four five [paul@RHEL4b pipes]$
```

Filtrul **tr -s** poate fi de asemenea folosit pentru a stoarce evenimente multiple unui caracter într-unul.

```
[paul@RHEL4b pipes]$ cat spaces.txt
one  two  three
    four  five  six
[paul@RHEL4b pipes]$ cat spaces.txt | tr -s ' '
one two three
    four five six
[paul@RHEL4b pipes]$
```

Puteți de asemenea folosi **tr** pentru a „cripta” texte cu **rot13**.

```
[paul@RHEL4b pipes]$ cat count.txt | tr 'a-z' 'nopqrstuvwxyzabcdefghijklm'
bar
gjb
guerr
sbhe
svir
[paul@RHEL4b pipes]$ cat count.txt | tr 'a-z' 'n-za-m'
bar
gjb
guerr
sbhe
svir
[paul@RHEL4b pipes]$
```

Acest ultim exemplu folosește **tr -d** pentru a șterge caractere.

```
paul@debian5:~/pipes$ cat tennis.txt | tr -d e
Amli Maursmo, Fra
Kim Clijstrs, BEL
Justin Hnin, Bl
Srna Williams, usa
Vnus Williams, USA
```

16.6. wc

Numărarea cuvintelor, liniilor și caracterelor este ușoară cu **wc**.

```
[paul@RHEL4b pipes]$ wc tennis.txt
5 15 100 tennis.txt
[paul@RHEL4b pipes]$ wc -l tennis.txt
5 tennis.txt
[paul@RHEL4b pipes]$ wc -w tennis.txt
15 tennis.txt
[paul@RHEL4b pipes]$ wc -c tennis.txt
100 tennis.txt
[paul@RHEL4b pipes]$
```

16.7. sort

Filtrul **sort** va face prin default o sortare alfabetică.

```
paul@debian5:~/pipes$ cat music.txt
Queen
Brel
Led Zeppelin
Abba
paul@debian5:~/pipes$ sort music.txt
Abba
Brel
Led Zeppelin
Queen
```

Dar filtrul **sort** are multe opțiuni pentru a schimba folosirea lui. Acest exemplu arată sortarea coloanelor diferite (coloana 1 sau coloana 2).

```
[paul@RHEL4b pipes]$ sort -k1 country.txt
Belgium, Brussels, 10
France, Paris, 60
Germany, Berlin, 100
Iran, Teheran, 70
Italy, Rome, 50
[paul@RHEL4b pipes]$ sort -k2 country.txt
Germany, Berlin, 100
Belgium, Brussels, 10
France, Paris, 60
Italy, Rome, 50
Iran, Teheran, 70
```

Captura de ecran de mai jos arată diferența dintre o sortare alfabetică și o sortare numerică (ambele pe a treia coloană).

```
[paul@RHEL4b pipes]$ sort -k3 country.txt
Belgium, Brussels, 10
Germany, Berlin, 100
Italy, Rome, 50
France, Paris, 60
Iran, Teheran, 70
[paul@RHEL4b pipes]$ sort -n -k3 country.txt
Belgium, Brussels, 10
Italy, Rome, 50
France, Paris, 60
Iran, Teheran, 70
Germany, Berlin, 100
```

16.8. **uniq**

Cu **uniq** puteți șterge duplicatele dintr-o listă sortată.

```
paul@debian5:~/pipes$ cat music.txt
Queen
Brel
Queen
Abba
paul@debian5:~/pipes$ sort music.txt
Abba
Brel
Queen
Queen
paul@debian5:~/pipes$ sort music.txt |uniq
Abba
Brel
Queen
```

uniq poate de asemenea să numere referințele cu opțiunea **-c**.

```
paul@debian5:~/pipes$ sort music.txt |uniq -c
1 Abba
1 Brel
2 Queen
```

16.9 **comm**

Compararea șirurilor (sau fișierelor) poate fi făcută cu comanda **comm**. Prin default **comm** va afișa ieșiri pe trei coloane. În acest exemplu, Abba, Cure și Queen sînt în ambele liste, Bowie și Sweet sînt doar în primul fișier, Turner este doar în cel de-al doilea.

```

paul@debian5:~/pipes$ cat > list1.txt
Abba
Bowie
Cure
Queen
Sweet
paul@debian5:~/pipes$ cat > list2.txt
Abba
Cure
Queen
Turner
paul@debian5:~/pipes$ comm list1.txt list2.txt
      Abba

Bowie

      Cure
      Queen

Sweet
      Turner

```

Ieșirea comenzii **comm** poate fi mai ușor de citit când facem să iasă o singură coloană. Tastele numerice arată care coloane de ieșire nu ar trebui afișate.

```

paul@debian5:~/pipes$ comm -12 list1.txt list2.txt
Abba
Cure
Queen
paul@debian5:~/pipes$ comm -13 list1.txt list2.txt
Turner
paul@debian5:~/pipes$ comm -23 list1.txt list2.txt
Bowie
Sweet

```

16.10. od

Europenilor le place să lucreze cu caractere ascii, dar computerele stochează fișiere în biți. Exemplul de mai jos crează un fișier simplu, apoi folosește **od** pentru a arăta conținutul fișierului în biți hexadecimali.

```

paul@laika:~/test$ cat > text.txt
abcdefg
1234567
paul@laika:~/test$ od -t x1 text.txt
00000000 61 62 63 64 65 66 67 0a 31 32 33 34 35 36 37 0a
00000020

```

Același fișier poate fi de asemeni afișat în biți octali.

```

paul@laika:~/test$ od -b text.txt
00000000 141 142 143 144 145 146 147 012 061 062 063 064 065 066 067 012
00000020

```

Și iată aici fișierul în ascii (sau caractere backslash).


```
paul@laika:~/test$ od -c text.txt
00000000  a  b  c  d  e  f  g  \n  1  2  3  4  5  6  7  \n
0000020
```

16.11. sed

Editorul curs **sed** poate face funcții de editare în curs, folosind **expresii regulate**.

```
paul@debian5:~/pipes$ echo level5 | sed 's/5/42/'
level42
paul@debian5:~/pipes$ echo level5 | sed 's/level/jump/'
jump5
```

Adăugați **g** pentru înlocuiri globale (toate evenimentele șirului per linie).

```
paul@debian5:~/pipes$ echo level5 level7 | sed 's/level/jump/'
jump5 level7
paul@debian5:~/pipes$ echo level5 level7 | sed 's/level/jump/g'
jump5 jump7
```

Cu **d** puteți șterge linii dintr-un curs conținând un caracter.

```
paul@debian5:~/test42$ cat tennis.txt
Venus Williams, USA
Martina Hingis, SUI
Justine Henin, BE
Serena williams, USA
Kim Clijsters, BE
Yanina Wickmayer, BE
paul@debian5:~/test42$ cat tennis.txt | sed '/BE/d'
Venus Williams, USA
Martina Hingis, SUI
Serena williams, USA
```

16.12. exemple de conduite

who | wc

Câți utilizatori sînt intrați în acest sistem?

```
[paul@RHEL4b pipes]$ who
root      tty1          Jul 25 10:50
paul      pts/0          Jul 25 09:29 (laika)
Harry     pts/1          Jul 25 12:26 (barry)
paul      pts/2          Jul 25 12:26 (pasha)
[paul@RHEL4b pipes]$ who | wc -l
4
```

who | cut | sort

Afișează o listă sortată a utilizatorilor intrați în sistem.

```
[paul@RHEL4b pipes]$ who | cut -d' ' -f1 | sort
Harry
paul
paul
root
```

Afișează o listă sortată a utilizatorilor intrați în sistem, dar doar fiecare utilizator numai o singură dată.

```
[paul@RHEL4b pipes]$ who | cut -d' ' -f1 | sort | uniq
Harry
paul
root
```

grep | cut

Afișează o listă a tuturor **conturilor utilizatorilor** bash pe acest computer. Conturile utilizatorilor vor fi explicate în detaliu mai târziu.

```
paul@debian5:~$ grep bash /etc/passwd
root:x:0:0:root:/root:/bin/bash
paul:x:1000:1000:paul,,,:/home/paul:/bin/bash
serena:x:1001:1001:./home/serena:/bin/bash
paul@debian5:~$ grep bash /etc/passwd | cut -d: -f1
root
paul
serena
```

16.13. practică: filtre

1. Puneți o listă sortată a tuturor utilizatorilor bash în `bashusers.txt`.
2. Puneți o listă sortată a tuturor utilizatorilor intrați în sistem în `onlineusers.txt`.
3. Faceți o listă a tuturor numelor de fișiere în `/etc` care conține șirul `samba`.
4. Faceți o listă sortată a tuturor fișierelor din `/etc` care conține șirul de curs insenzitiv `samba`.
5. Observați ieșirea lui `/sbin/ifconfig`. Scrieți o linie care afișează doar adresa ip și subnet mask.
6. Scrieți o linie care șterge toate caracterele non-literele dintr-un curs.
7. Scrieți o linie care primește un fișier text, și are ca ieșire toate cuvintele pe o linie separată.
8. Scrieți un corector de cuvinte pe linia de comandă. (Ar putea exista un dicționar în `/usr/share/dict/`).

16.14. soluție: filtre

1. Puneți o listă sortată a tuturor utilizatorilor bash în bashusers.txt.

```
grep bash /etc/passwd | cut -d: -f1 | sort > bashusers.txt
```

2. Puneți o listă sortată a tuturor utilizatorilor intrați în sistem în onlineusers.txt.

```
who | cut -d' ' -f1 | sort > onlineusers.txt
```

3. Faceți o listă a tuturor numelor de fișiere în **/etc** care conține șirul samba.

```
ls /etc | grep samba
```

4. Faceți o listă sortată a tuturor fișierelor din **/etc** care conține șirul de curs insenzitiv samba.

```
ls /etc | grep -i samba | sort
```

5. Observați ieșirea lui **/sbin/ifconfig**. Scrieți o linie care afișează doar adresa ip și subnet mask.

```
/sbin/ifconfig | head -2 | grep 'inet ' | tr -s ' ' | cut -d' ' -f3,5
```

6. Scrieți o linie care șterge toate caracterele non-literale dintr-un curs.

```
paul@deb503:~$ cat text
```

```
This is, yes really! , a text with ?&* too many str$ange# characters ;-)
```

```
paul@deb503:~$ cat text | tr -d ',!$?.*&^%#@;()-'
```

```
This is yes really a text with too many strange characters
```

7. Scrieți o linie care primește un fișier text, și are ca ieșire toate cuvintele pe o linie separată.

```
paul@deb503:~$ cat > text2
```

```
it is very cold today without the sun
```

```
paul@deb503:~$ cat text2 | tr ' ' '\n'
```

```
it
```

```
is
```

```
very
```

```
cold
```

```
today
```

```
without
```

```
the
```

```
sun
```

8. Scrieți un corector de cuvinte pe linia de comandă. (Ar putea exista un dicționar în **/usr/share/dict/**).

```
paul@rhel ~$ echo "The zun is shining today" > text
```

```
paul@rhel ~$ cat > DICT
```

```
is
```

```
shining
```

```
sun
```

```
the
```

```
today
```

```
paul@rhel ~$ cat text | tr 'A-Z ' 'a-z\n' | sort | uniq | comm -23 - DICT zun
```

Puteți de asemeni să adăugați soluția de la întrebarea numărul 6 pentru a șterge caracterele non-litrală, și **tr -s ' '** pentru a șterge spațiile redundante.

Capitolul 17. utilitare de bază Unix

Conținut

17.1.	find.	.143
17.2.	locate.	.143
17.3.	date.	.144
17.4.	cal.	.144
17.5.	sleep.	.145
17.6.	time.	.145
17.7.	gzip - gunzip.	.145
17.8.	zcat - zmore.	.146
17.9.	bzip2 - bunzip2.	.146
17.10.	bzcat - bzmores.	.146
17.11.	practică: utilitare de bază Unix.	.147
17.12.	soluție: utilitare de bază Unix.	.148

Acest capitol introduce comenzi care să **găsească** (find) sau să **localizeze** (locate) fișiere și să **comprezeze** fișiere, împreună cu alte utilitare comune care nu au fost discutate înainte. În timp ce utilitarele discutate aici sînt tehnice și nu sînt considerate **filtre**, ele pot fi folosite în **conducente**.

17.1. find

Comanda **find** poate fi foarte folositoare la începutul unei conduite pentru a căuta fișiere. Aici sînt cîteva exemple. Ați putea dori să adăugați **2>/dev/null** liniilor de comandă pentru a evita încetșarea ecranului cu mesaje de eroare.

Găsește toate fișierele din **/etc** și pune lista în **etcfiles.txt**

```
find /etc > etcfiles.txt
```

Găsește toate fișierele întregului sistem și pune lista în **allfiles.txt**

```
find / > allfiles.txt
```

Găsește fișierele care se termină în **.conf** în directorul curent (și în toate subdirectoarele).

```
find . -name "*.conf"
```

Găsește fișiere de tip fișier (nu directoare, conduite, etc.) care se termină în **.conf**

```
find . -type f -name "*.conf"
```

Găsește fișiere de tip director care se termină în **.bak**.

```
find /data -type d -name "*.bak"
```

Găsește fișiere care sînt mai noi decît **file42.txt**

```
find . -newer file42.txt
```

find poate de asemenea să execute o altă comandă la fiecare fișier găsit. Acest exemplu va căuta fișiere ***.odf** și le va copia în **/backup/**.

```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

find poate de asemenea să execute, după confirmare, o altă comandă în orice fișier găsit. Acest exemplu va șterge toate fișierele ***.odf** dacă aprobați asta pentru fiecare fișier găsit.

```
find /data -name "*.odf" -ok rm {} \;
```

17.2. locate

Utilitarul **locate** este foarte diferit de **find** prin faptul că utilizează un index pentru a localiza fișiere. Asta este mult mai rapid decît traversarea tuturor directoarelor, dar de asemenea înseamnă că acesta este întotdeauna neactualizat. Dacă indexul nu există încă, atunci va trebui să-l creați (ca root pe Red Hat Enterprise Linux) cu comanda **updatedb**.

```
[paul@RHEL4b ~]$ locate Samba
warning: locate: could not open database: /var/lib/slocate/slocate.db:...
warning: You need to run the 'updatedb' command (as root) to create th...
Please have a look at /etc/updatedb.conf to enable the daily cron job.
[paul@RHEL4b ~]$ updatedb
fatal error: updatedb: You are not authorized to create a default sloc...
[paul@RHEL4b ~]$ su -
Password:
[root@RHEL4b ~]# updatedb
[root@RHEL4b ~]#
```

Cele mai multe distribuții Linux vor programa **updatedb** să se execute o dată în fiecare zi.

17.3. date

Comanda **date** poate afișa data, timpul, zona temporală și mai mult.

```
paul@rhel55 ~$ date
Sat Apr 17 12:44:30 CEST 2010
```

Un șir al comenzii **date** poate fi aranjat pentru a afișa formatul ales. Vedeți pagina **man** pentru mai multe opțiuni.

```
paul@rhel55 ~$ date +%A %d-%m-%Y'
Saturday 17-04-2010
```

Timpul pe orice Unix este calculat în numere de secunde din 1969 (prima secundă fiind prima secundă a datei de 1 ianuarie 1970). Folosiți **date +%s** pentru a afișa timpul Unix în secunde.

```
paul@rhel55 ~$ date +%s
1271501080
```

Cînd acest cronometru de secunde va ajunge la două sute de milioane?

```
paul@rhel55 ~$ date -d '1970-01-01 + 2000000000 seconds'
Wed May 18 04:33:20 CEST 2033
```

17.4. cal

Comanda **cal** afișează luna curentă, cu ziua curentă selectată.

```
paul@rhel55 ~$ cal
    April 2010
Su Mo Tu We Th Fr Sa
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30
```

Puteți selecta orice lună în trecut sau în viitor.


```
paul@rhel55 ~$ cal 2 1970
    February 1970
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
```

17.5. sleep

Comanda **sleep** este uneori folosită în script-uri pentru a aștepta un număr de secunde. Acest exemplu arată un **sleep** pentru 5 secunde.

```
paul@rhel55 ~$ sleep 5
paul@rhel55 ~$
```

17.6. time

Comanda **time** poate afișa cât timp este necesar ca o comandă să se execute. Comanda **date** îi trebuie doar puțin timp.

```
paul@rhel55 ~$ time date
Sat Apr 17 13:08:27 CEST 2010
```

```
real    0m0.014s
user    0m0.008s
sys     0m0.006s
```

Comanda **sleep 5** cere 5 secunde **reale** pentru a se executa, dar consumă mai puțin **timp cpu**.

```
paul@rhel55 ~$ time sleep 5
```

```
real    0m5.018s
user    0m0.005s
sys     0m0.011s
```

Această comandă **bzip2** compresează un fișier și utilizează mult **timp cpu**.

```
paul@rhel55 ~$ time bzip2 text.txt
```

```
real    0m2.368s
user    0m0.847s
sys     0m0.539s
```

17.7. gzip - gunzip

Utilizatorii nu au niciodată destul spațiu pe disk, astfel compresia este la îndemână. Comanda **gzip** poate face ca fișierele să ocupe mai puțin spațiu.

```
paul@rhel55 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
paul@rhel55 ~$ gzip text.txt
paul@rhel55 ~$ ls -lh text.txt.gz
-rw-rw-r-- 1 paul paul 760K Apr 17 13:11 text.txt.gz
```

Puteți avea fișierul original cu **gunzip**.

```
paul@rhel55 ~$ gunzip text.txt.gz
paul@rhel55 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
```

17.8. zcat - zmore

Fișierele care sînt compresate cu **gzip** pot fi văzute cu **zcat** și **zmore**.

```
paul@rhel55 ~$ head -4 text.txt
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
paul@rhel55 ~$ gzip text.txt
paul@rhel55 ~$ zcat text.txt.gz | head -4
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
```

17.9. bzip2 - bunzip2

Fișierele pot de asemenea să fie compresate cu **bzip2** ceea ce ia un pic mai mult decît **gzip**, dar compresează mai bine.

```
paul@rhel55 ~$ bzip2 text.txt
paul@rhel55 ~$ ls -lh text.txt.bz2
-rw-rw-r-- 1 paul paul 569K Apr 17 13:11 text.txt.bz2
```

Fișierele pot fi decompresate din nou cu **bunzip2**.

```
paul@rhel55 ~$ bunzip2 text.txt.bz2
paul@rhel55 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
```

17.10. bzip2 - bzip2

Și în aceeași modalitate **bzip2** și **bzip2** pot afișa fișiere compresate cu **bzip2**.

```
paul@rhel55 ~$ bzip2 text.txt
paul@rhel55 ~$ bzip2 text.txt.bz2 | head -4
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
```

17.11. practică: utilitare de bază Unix

1. Explicați diferența dintre aceste două comenzi. Întrebarea este foarte importantă. Dacă nu știți răspunsul, atunci uitați-vă din nou la capitolul **shell**.

```
find /data -name "*.txt"
```

```
find /data -name *.txt
```

2. Explicați diferența dintre aceste două afirmații. Ambele comenzi vor funcționa când există 200 de fișiere **.odf** în **/data**? Dar dacă sînt 2 milioane de fișiere **.odf**?

```
find /data -name "*.odf" > data_odf.txt
```

```
find /data/*.odf > data_odf.txt
```

3. Scrieți o comandă **find** care găsește toate fișierele create după 30 ianuarie 2010.

4. Scrieți o comandă **find** care găsește toate fișierele ***.odf** create în septembrie 2009.

5. Numărați numărul fișierelor ***.conf** din **/etc** și toate subdirectoarele lui.

6. Două comenzi care fac același lucru: copiază fișierele ***.odf** în **/backup/**. Care ar fi motivul de a înlocui prima comandă cu a doua? Din nou, aceasta este o întrebare importantă.

```
cp -r /data/*.odf /backup/
```

```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

7. Creați un fișier numit **loctest.txt**. Puteți găsi acest fișier cu **locate**? De ce nu? Cum faceți ca **locate** să găsească acest fișier?

8. Folosiți **find** și **-exec** pentru a redenumi toate fișierele **.htm** în **.html**.

9. Tastați comanda **date**. Acum afișați data în format **YYYY/MM/DD**.

10. Tastați comanda **cal**. Afișați un calendar al anului 1582 și 1752. Ați observat ceva special?

17.12. soluție: utilitare de bază Unix

1. Explicați diferența dintre aceste două comenzi. Întrebarea este foarte importantă. Dacă nu știți răspunsul, atunci uitați-vă din nou la capitolul shell.

```
find /data -name "*.txt"
```

```
find /data -name *.txt
```

Cînd caracterele *.txt sînt puse între ghilimele terminalul nu se va atinge de ele. Utilitarul find va căuta în /data toate fișierele care se termină în .txt.

Cînd caracterele *.txt nu sînt puse între ghilimele atunci shell-ul ar putea extinde asta (cînd una sau mai multe fișiere care se termină în .txt există în directorul curent). Comanda find ar putea arăta un rezultat diferit, sau poate da ca rezultat o eroare de sintaxă.

2. Explicați diferența dintre aceste două afirmații. Ambele comenzi vor funcționa cînd există 200 de fișiere .odf în /data? Dar dacă sînt 2 milioane de fișiere .odf?

```
find /data -name "*.odf" > data_odf.txt
```

```
find /data/*.odf > data_odf.txt
```

Prima comandă find va face ieșire tuturor fișierelor .odf în /data și tuturor subdirectoarelor. Terminalul va redirecta asta într-un fișier.

A doua comandă find va face ieșire tuturor fișierelor numite .odf în /data și va face de asemeni ieșire tuturor fișierelor care există în directoare numite *.odf (în /data).

Cu două milioane de fișiere linia de comandă va fi extinsă dincolo de maximum pe care shell-ul poate să-l accepte. Ultima parte a comenzii ar fi pierdută.

3. Scrieți o comandă find care găsește toate fișierele create după 30 ianuarie 2010.

```
touch -t 201001302359 marker_date  
find . -type f -newer marker_date
```

Există o altă soluție:

```
find . -type f -newerat "20100130 23:59:59"
```

4. Scrieți o comandă find care găsește toate fișierele *.odf create în septembrie 2009.

```
touch -t 200908312359 marker_start  
touch -t 200910010000 marker_end  
find . -type f -name "*.odf" -newer marker_start ! -newer marker_end
```

Semnul exclamării ! -newer poate fi citit ca **not newer**.

5. Numărați numărul fișierelor *.conf din /etc și din toate subdirectoarele lui.

```
find /etc -type f -name '*.conf' | wc -l
```

6. Două comenzi care fac același lucru: copiază fișierele *.odf în /backup/. Care ar fi motivul de a înlocui prima comandă cu a doua? Din nou, aceasta este o întrebare importantă.

```
cp -r /data/*.odf /backup/
```

```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

Prima ar putea eșua când există prea multe fișiere care să încapă într-o singură linie de comandă.

7. Creați un fișier numit **loctest.txt**. Puteți găsi acest fișier cu **locate**? De ce nu? Cum faceți ca locate să găsească acest fișier?

Nu puteți localiza asta cu locate pentru că nu este încă în index.

```
updatedb
```

8. Folosiți find și -exec pentru a redenumi toate fișierele .htm în .html.

```
paul@rhel55 ~$ find . -name '*.htm'
./one.htm
./two.htm
paul@rhel55 ~$ find . -name '*.htm' -exec mv {} {}l \;
paul@rhel55 ~$ find . -name '*.html'
./one.html
./two.html
```

9. Tastați comanda **date**. Acum afișați data în format YYYY/MM/DD.

```
date +%Y/%m/%d
```

10. Tastați comanda **cal**. Afișați un calendar al anului 1582 și 1752. Ați observat ceva special?

```
cal 1582
```

```
cal 1752
```

Calendarele sînt diferite depinzînd de țară. Vedeți <http://linux-training.be/files/studentfiles/dates.txt>.

Partea V. vi

Capitolul 18. introducere în vi

Conținut

18.1.	modul comandă și modul insert.	152
18.2.	începeți să tastați (a A i I o O).	152
18.3.	înlocuiește și șterge un caracter (r x X).	152
18.4.	undo și repetă (u .).	152
18.5.	taie, copie și lipește o linie (dd yy p P).	153
18.6.	taie, copie și lipește linii (3dd 2yy).	153
18.7.	începutul și sfârșitul unei linii (0 sau ^ și \$).	153
18.8.	îmbinare două linii (J) și mai mult.	153
18.9.	cuvinte (w b).	154
18.10.	salvare (sau nu) și ieșire (:w :q :q!).	154
18.11.	căutare (/ ?).	154
18.12.	înlocuiește tot (:1,\$ s/foo/bar/g).	155
18.13.	citire fișiere (:r :r !cmd).	155
18.14.	memorii intermediare text.	155
18.15.	fișiere multiple.	155
18.16.	abrevieri.	155
18.17.	setări taste.	156
18.18.	opțiuni de setare.	156
18.19.	practică: vi(m).	157
18.20.	soluție: vi(m).	158

Editorul **vi** este instalat pe aproape orice Unix. Linux va avea deseori instalat **vim** (**vi improved**) care este similar. Fiecare administrator de sistem ar trebui să cunoască **vi(m)**, pentru că este un utilitar ușor pentru a rezolva probleme.

Editorul **vi** nu este intuitiv, dar odată ce vă obișnuiți cu el, **vi** devine o aplicație foarte puternică. Cele mai multe distribuții Linux vor include **vimtutor** care este o lecție de 45 de minute în **vi(m)**.

18.1. modul comandă și modul insert

Editorul vi începe în **modul comandă**. În modul comandă, puteți tasta comenzi. Unele comenzi vă vor duce în **modul insert**. În modul insert, puteți tasta text. **Tasta escape** vă va retrimite în modul comandă.

Tabelul 18.1. intrînd în modul comandă

tastă	acțiune
-------	---------

Esc	setează vi(m) în modul comandă
-----	--------------------------------

18.2. începeți să tastați (a A i I o O)

Diferența dintre a A i I o și O este locația unde puteți începe să tastați. a va adăuga după caracterul curent și A va adăuga la sfîrșitul liniei. i va insera după caracterul curent și I va insera la începutul liniei. o vă va pune într-o nouă linie după linia curentă și O vă va pune într-o nouă linie înainte de linia curentă.

Tabelul 18.2. schimbare în modul insert

comandă	acțiune
---------	---------

a	începeți să tastați după caracterul curent
A	începeți să tastați la sfîrșitul liniei curente
i	începeți să tastați înainte de caracterul curent
I	începeți să tastați la începutul liniei curente
o	începeți să tastați pe o nouă linie după linia curentă
O	începeți să tastați pe o nouă linie înainte de linia curentă

18.3. înlocuiește și șterge un caracter (r x X)

Cînd sînteți în modul comandă (nu e deloc rău să apăsați tasta escape de mai multe ori), puteți folosi tasta x pentru a șterge caracterul curent. Tasta X cu literă mare (sau shift x) va șterge caracterul rămas pe cursor. La fel cînd sînteți în modul comandă, puteți folosi tasta r pentru a înlocui un singur caracter. Tasta r vă va duce în modul insert după o singură apăsare de tastă, și vă va readuce imediat în modul comandă.

Tabelul 18.3. înlocuiește și șterge

comandă	acțiune
---------	---------

x	șterge un caracter sub cursor
X	șterge un caracter înainte de cursor
r	înlocuiește un caracter sub cursor
p	lipește după cursor (aici ultima literă ștearsă)
xp	schimbă două caractere

18.4. undo și repetă (u .)

Cînd sînteți în modul comandă, puteți face undo greșelilor cu u. Puteți face greșeli de două ori cu . (cu alte cuvinte, . va repeta ultima comandă).

Tabelul 18.4 undo și repetă (u.)

comandă	acțiune
---------	---------

u	undo ultima acțiune
.	repetă ultima acțiune

18.5. taie, copie și lipește o linie (dd yy p P)

Cînd sînteți în modul comandă, dd va tăia ultima linie. yy va copia linia curentă. Puteți lipi ultima linie copiată sau tăiată după (p) sau înainte (P) de linia curentă.

Tabelul 18.5. taie, copie și lipește o linie

comandă	acțiune
dd	taie linia curentă
yy	(yank yank) copie linia curentă
p	lipește după linia curentă
P	lipește înainte de linia curentă

18.6. taie, copie și lipește linii (3dd 2yy)

Cînd sînteți în linia de comandă, înainte de a tasta dd sau yy, puteți tasta un număr pentru a repeta comanda un număr de ori. Astfel, 5dd va tăia 5 linii și 4yy va copia (yank) 4 linii. Acea ultimă comandă va fi notată de vi în colțul de sfîrșit jos ca „4 line yanked”.

Tabelul 18.6. taie, copie și lipește linii

comandă	acțiune
3dd	taie trei linii
4yy	copie patru linii

18.7. început și sfîrșit linie (0 sau ^ și \$)

Cînd sînteți în linia de comandă, 0 și caret ^ vă aduce la începutul liniei curente, în timp ce \$ va pune cursorul la sfîrșitul liniei curente. Puteți adăuga 0 și \$ la comanda d, d0 va șterge fiecare literă dintre caracterul curent și începutul liniei. Tot astfel d\$ va șterge orice de la caracterul curent pînă la sfîrșitul liniei. La fel y0 și y\$ vor copia de la început la sfîrșitul liniei curente.

Tabelul 18.7. început și sfîrșit linie

comandă	acțiune
0	sare la începutul liniei curente
^	sare la începutul liniei curente
\$	sare la sfîrșitul liniei curente
d0	șterge pînă la începutul liniei
d\$	șterge pînă la sfîrșitul liniei

18.8. îmbinare două linii (J) și mai mult

Cînd sînteți în linia de comandă, apăsînd J va adăuga următoarea linie liniei curente. Cu yyp duplicați o linie și cu ddp schimbați două linii.

Tabelul 18.8. îmbinare două linii

comandă	acțiune
J	îmbinare două linii
yyp	duplicare a unei linii
ddp	schimbare a două linii

18.9. cuvinte (w b)

Cînd sînteți în linia de comandă, **w** va sări la cuvîntul următor și **b** se va muta la cuvîntul anterior. **w** și **b** pot fi combinate de asemeni cu **d** și **y** pentru a copia și tăia cuvinte (**dw db yw yb**).

Tabelul 18.9. cuvinte

comandă	acțiune
w	înainte un cuvînt
b	înapoi un cuvînt
3w	înainte trei cuvinte
dw	șterge un cuvînt
yw	yank (copiază) un cuvînt
5yb	yank 5 cuvinte înapoi
7dw	șterge 7 cuvinte

18.10 salvează (sau nu) și iese (:w :q :q!)

Apăsînd tasta două puncte **:** ne permite să dăm instrucțiuni lui **vi** (din punct de vedere tehnic vorbind, tastînd **:** va deschide editorul **ex**). **:w** va scrie (salva) fișierul, **:q** va închide un fișier neschimbat fără să-l salveze, și **:q!** va părăsi **vi** excluzînd orice schimbări. **:wq** va salva și părăsi fișierul și este la fel ca tastînd **ZZ** în modul comandă.

Tabelul 18.10 salvare și ieșire vi

comandă	acțiune
:w	salvează (scrie)
:w fname	salvează ca fname
:q	renunță
:wq	salvează și renunță
ZZ	salvează și renunță
:q!	renunță (excluzînd schimbările)
:w!	salvează (și scrie într-un fișier fără permisiuni de scriere!)

Ultima comandă este un pic mai specială. Cu **:w!** **vi** va încerca să facă **chmod** fișierului pentru a obține permisiuni de scriere (asta funcționează dacă sînteți proprietarul fișierului) și va face **chmod** înapoi cînd permisiunea de scriere are loc. Asta ar trebui să funcționeze mereu cînd sînteți root (și sistemul de fișiere are permisiune de scriere).

18.11. căutare (/ ?)

Cînd sînteți în modul comandă tastînd **/** vă va permite să căutați în **vi** șiruri (pot fi expresii regulate). Tastînd **/foo** va face o căutare înainte pentru șirul **foo** și tastînd **?bar** va face o căutare inversă pentru **bar**.

Tabelul 18.11. căutare

comandă	acțiune
/șir	caută înainte șirul
?șir	caută înapoi șirul
n	se duce la noul eveniment de căutare a șirului
/^string	caută înainte șirul la începutul liniei
/br[aeio]l	caută bral brel bril și brol
/\<he\>	caută cuvîntul he (și nu here sau the)

18.12. înlocuiește tot (:1,\$ s/foo/bar/g)

Pentru a înlocui toate evenimentele șirului foo cu bar, mai întâi schimbăm în modul ex cu : . Apoi îi spunem lui vi care linii să utilizeze, de exemplu 1,\$ va înlocui totul de la prima la ultima linie. Putem scrie 1,5 pentru a procesa doar primele cinci linii. s/foo/bar/g va înlocui toate evenimentele foo cu bar.

Tabelul 18.12. înlocuiri

comandă	acțiune
:4,8 s/foo/bar/g	înlocuiește foo cu bar pe liniile 4 pînă la 8
:1,\$ s/foo/bar/g	înlocuiește foo cu bar pe toate liniile

18.13. citire fișiere (:r :r !cmd)

Cînd sînteți în modul comandă, :r foo va citi fișierul numit foo, :r !foo va executa comanda foo. Rezultatul va fi pus în locația curentă. Astfel :r !ls va pune o listare a directorului curent în fișierul text.

Tabelul 18.13. citire fișiere și introducere

comandă	acțiune
:r fname	(citire) fișier fname și lipește conținut
:r !cmd	execută cmd și lipește ieșirile

18.14. memorii intermediare text

Există 36 de memorii intermediare în vi pentru a stoca text. Puteți să le folosiți cu caracterul ".

Tabelul 18.14. memorii intermediare text

comandă	acțiune
"add	șterge linia curentă și pune text în memoria intermediară a
"g7yy	copie șapte linii în memoria intermediară g
"ap	lipește din memoria intermediară a

18.15 fișiere multiple

Puteți edita fișiere multiple cu vi. Mai jos sînt unele sfaturi.

Tabelul 18.15. fișiere multiple

comandă	acțiune
vi file1 file2 file3	începe editarea a trei fișiere
:args	listează fișiere și marchează fișier activ
:n	începe editarea următorului fișier
:e	schimbă cu ultimul fișier editat
:rew	învîrte indicatorul de adresă fișier la primul fișier

18.16. prescurtări

Cu :ab puteți pune prescurtări în vi. Folosiți :una pentru a anula prescurtările.

Tabelul 18.16. prescurtări

comandă	acțiune
:ab str long string	prescurtare str pentru a fi 'long string'
:una str	deabreviere str

18.17. mapări taste

În mod similar cu abrevierile lor, puteți folosi mapări cu **:map** în modul comandă și **:map!** în modul inserare.

Acest exemplu arată cum să setăm funcția tastei F6 pentru a schimba între **set number** și **set nonumber**. <bar> separă cele două comenzi, **set number!** schimbă starea și **set number?** raportează statusul curent.

```
:map <F6> :set number!<bar>set number?<CR>
```

18.18. opțiuni setare

Anumite opțiuni pe care le puteți seta în vim.

```
:set number (încercați de asemeni :se nu)
:set nonumber
:syntax on
:syntax off
:set all (listează toate opțiunile)
:set tabstop=8
:set tx (CR/LF stiluri de sfârșit)
:set notx
```

Putem seta aceste opțiuni (și mai mult) în ~/.vimrc pentru vim sau în ~/.exrc pentru vi standard.

```
paul@barry:~$ cat ~/.vimrc
set number
set tabstop=8
set textwidth=78
map <F6> :set number!<bar>set number?<CR>
paul@barry:~$
```

18.19. practică: vi(m)

1. Începeți vimtutor și faceți unele sau toate exercițiile. S-ar putea să aveți nevoie să executați **aptitude install vim** pe Xubuntu.
2. Care combinații de 3 taste în modul comandă va duplica linia curentă.
3. Care combinații de 3 taste în modul comandă va schimba locul a două linii (linia cinci devine linia șase și linia șase devine linia cinci).
4. Care combinații de două taste în modul comandă va schimba locul unui caracter cu următorul.
5. vi poate înțelege macro-uri. Un macro poate fi înregistrat cu q urmat de numele macro-ului. Astfel qa va înregistra macro-ul numit a. Apăsând q din nou va termina înregistrarea. Puteți rechema macro-ul cu @ urmat de numele macro-ului. Încercați acest exemplu: i 1 'Tasta Escape' qa yyp 'Ctrl a' q 5@a (Ctrl a va mări numărul cu unu).
6. Copiați /etc/passwd în ~/passwd. Deschideți ultimul fișier în vi și apăsați Ctrl v. Folosiți tastele săgeți pentru a selecta un Bloc Vizual, puteți copia asta cu y sau să-l ștergeți cu d. Încercați să-l lipiți.
7. Ce face comanda dwwP când sînteți la începutul unui cuvînt într-o propoziție?

18.20. soluție: vi(m)

1. Începeți vimtutor și faceți unele sau toate exercițiile. S-ar putea să aveți nevoie să executați **aptitude install vim** pe Xubuntu.

vimtutor

2. Care combinații de 3 taste în modul comandă va duplica linia curentă.

yy

3. Care combinații de 3 taste în modul comandă va schimba locul a două linii (linia cinci devine linia șase și linia șase devine linia cinci).

dd

4. Care combinații de două taste în modul comandă va schimba locul unui caracter cu următorul.

x

5. vi poate înțelege macro-uri. Un macro poate fi înregistrat cu q urmat de numele macro-ului. Astfel qa va înregistra macro-ul numit a. Apăsând q din nou va termina înregistrarea. Puteți rechema macro-ul cu @ urmat de numele macro-ului. Încercați acest exemplu: i 1 'Tasta Escape' qa yyp 'Ctrl a' q 5@a (Ctrl a va mări numărul cu unu).

6. Copiați /etc/passwd în ~/passwd. Deschideți ultimul fișier în vi și apăsați Ctrl V. Folosiți tastele săgeți pentru a selecta un Bloc Vizual, puteți copia asta cu y sau să-l ștergeți cu d. Încercați să-l lipiți.

cp /etc/passwd ~

vi passwd

(apăsați Ctrl-V)

7. Ce face comanda **dwwP** când sînteți la începutul unui cuvînt într-o propoziție?

dwwP poate schimba cuvîntul curent cu următorul cuvînt.

Partea VI. scripting

Capitolul 19. introducere în scripting

Conținut

19.1.	cerințe.	161
19.2.	hello world.	161
19.3.	she-bang.	161
19.4.	comentariu.	161
19.5.	variabile.	162
19.6.	sursa unui script.	162
19.7.	depanarea unui script.	162
19.8.	prevenirea falsificării setuid root.	163
19.9.	practică: introducere în scripting.	164
19.10.	soluție: introducere în scripting.	165

Shell-uri ca **bash** și **korn** au suport pentru construcții de programare care pot fi salvate ca **script-uri**. Aceste **script-uri** ca rezultat devin apoi mai multe comenzi **shell**. Multe comenzi Linux sînt **script-uri**. **Script-uri de profil utilizator** sînt executate cînd un utilizator intră în sistem și **script-uri init** sînt executate cînd un **daemon** este închis sau deschis.

Asta înseamnă că administratorii de sistem de asemeni au nevoie de cunoștințe de bază despre **scripting** pentru a înțelege cum serverele și aplicațiile lor sînt deschise, aduse la zi, reînprospătate, cîrpite, menținute, configurate și șterse, și de asemeni să înțeleagă cum este construit un mediu utilizator.

Scopul acestui capitol este de a vă da destulă informație pentru a fi capabili să citiți și să înțelegeți script-uri. Nu de a deveni un scriitor de script-uri complexe.

19.1. cerințe

Ar trebui să citiți și să înțelegeți **partea III extindere shell** și **partea IV conduite și comenzi** înainte de a începe acest capitol.

19.2. hello world

Ca în orice curs de programare, începem cu un script simplu **hello_world**. Următorul script va afișa ieșirea **Hello World**.

```
echo Hello World
```

După ce creăm acest script simplu cu **vi** sau cu **echo**, trebuie să facem executabil **hello_world** cu **chmod+x**. Și numai dacă adăugați directorul script-urilor la cale, trebuie să tastați calea către script pentru ca shell-ul să fie capabil să-l găsească.

```
[paul@RHEL4a~]$ echo echo Hello World > hello_world
[paul@RHEL4a~]$ chmod +x hello_world
[paul@RHEL4a~]$ ./hello_world
Hello World
[paul@RHEL4a~]$
```

19.3. she-bang

Haideți să mărim exemplul nostru puțin mai departe punînd un **#!/bin/bash** primei linii a scriptului. **#!** este numit un **she-bang** (uneori numit **sha-bang**), unde **she-bang** sînt primele două caractere ale script-ului.

```
#!/bin/bash
echo Hello World
```

Nu puteți fi niciodată sigur ce fel de shell rulează un utilizator. Un script care merge fără greșală în **bash** s-ar putea să nu meargă în **ksh**, **csh**, sau **dash**. Pentru a da instrucțiuni shell-ului să execute script-ul într-un anumit shell, puteți începe script-ul cu un **she-bang** urmat de shell-ul în care ar trebui să se execute. Acest script va rula într-un shell bash.

```
#!/bin/bash
echo -n hello
echo A bash subshell `echo -n hello`
```

Acest script se va executa într-un shell Korn (doar dacă **/bin/ksh** este un hard link către **/bin/bash**). Fișierul **/etc/shells** conține o listă a shell-urilor de pe sistem.

```
#!/bin/ksh
echo -n hello
echo a Korn subshell `echo -n hello`
```

19.4. comentariu

Să mărim exemplul nostru puțin mai departe adăugînd linii de comentariu.

```
#!/bin/bash
#
# Hello World Script
#
echo Hello World
```

19.5. variabile

Aici este un exemplu simplu a unei variabile înăuntrul unui script.

```
#!/bin/bash
#
# variabilă simplă în script
#
var1=4
echo var1 = $var1
```

Scripturile pot conține variabile, dar pentru că script-urile sînt executate în propriul lor shell, variabilele nu supraviețuiesc la sfîrșitul scriptului.

```
[paul@RHEL4a ~]$ echo $var1
```

```
[paul@RHEL4a ~]$ ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1
```

```
[paul@RHEL4a ~]$
```

19.6. sursa unui script

Din fericire, puteți forța un script să se ruleze în același shell; asta se numește **sursa** unui script.

```
[paul@RHEL4a ~]$ source ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1
4
[paul@RHEL4a ~]$
```

Ce e scris deasupra este identic cu ce e scris mai jos.

```
[paul@RHEL4a ~]$ . ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1
4
[paul@RHEL4a ~]$
```

19.7. depanarea unui script

O altă modalitate de a executa un script într-un shell separat este de a tasta **bash** cu numele scriptului ca un parametru.

```
paul@debian6~/test$ bash runme
42
```

Mărind asta la **bash -x** permite să vedeți comenzile pe care shell-ul le execută (după extinderea shell-ului).

```
paul@debian6~/test$ bash -x runme
+ var4=42
+ echo 42
42
paul@debian6~/test$ cat runme
# the runme script
var4=42
echo $var4
paul@debian6~/test$
```

Observați absența liniei comentate (#), și înlocuirea variabilei înaintea execuției lui **echo**.

19.8. prevenirea falsificării **setuid root**

Unii utilizatori ar putea încerca să facă un script **setuid** bazat pe **falsificare root**. Acest lucru este rar dar un posibil atac. Pentru a îmbunătăți securitatea scriptului și pentru a evita înșelătoria interpretării, trebuie să adăugați **--** după **#!/bin/bash**, care dizabilitează procesarea opțiunilor ulterioare astfel că shell-ul nu va accepta nici o opțiune.

```
#!/bin/bash -
sau
#!/bin/bash --
```

Orice argumente după **--** sînt tratate ca nume de fișier și argumente. Un argument **al -** este echivalent cu **--**.

19.9. practică: introducere în scripting

0. Dați fiecărui script un nume diferit, păstrați-le pentru mai târziu!
1. Scrieți un script care face ieșire numelui unui oraș.
2. Asigurați-vă că script-ul se execută într-un shell bash.
3. Asigurați-vă că script-ul se execută în shell-ul Korn.
4. Creați un script care definește două variabile, și face ieșirea valorii lor.
5. Script-ul de mai înainte nu influențează shell-ul curent (variabilele nu există în afara script-ului). Acum executați script-ul astfel ca să influențeze shell-ul curent.
6. Există o cale mai scurtă pentru **a da sursă** script-ului?
7. Comentați script-urile astfel încât să știți ce fac ele.

19.10. soluție: introducere în scripting

0. Dați fiecărui script un nume diferit, păstrați-le pentru mai târziu!

1. Scrieți un script care face ieșire numelui unui oraș.

```
echo 'echo Antwerp' > first.bash
chmod +x first.bash
./first.bash
Antwerp
```

2. Asigurați-vă că script-ul se execută într-un shell bash.

```
$ cat first.bash
#!/bin/bash
echo Antwerp
```

3. Asigurați-vă că script-ul se execută în shell-ul Korn.

```
$ cat first.bash
#!/bin/ksh
echo Antwerp
```

Observați că în timp ce first.bash va funcționa din punct de vedere tehnic ca un script shell Korn, numele care se termină în .bash este confuz.

4. Creați un script care definește două variabile, și face ieșirea valorii lor.

```
$ cat second.bash
#!/bin/bash

var33=300
var42=400
```

```
echo $var33 $var42
```

5. Script-ul de mai înainte nu influențează shell-ul curent (variabilele nu există în afara script-ului). Acum executați scriptul astfel ca să influențeze shell-ul curent.

```
source second.bash
```

6. Există o cale mai scurtă pentru a da sursă scriptului?

```
. ./second.bash
```

7. Comentați script-urile astfel încât să știți ce fac ele.

```
$ cat second.bash
#!/bin/bash
# script pentru a testa variabile și surse
# definire a două variabile
var33=300
var42=400

# ieșirea valorii acestor variabile
echo $var33 $var42
```

Capitolul 20. loop-uri script

Conținut

20.1. test [].	.168
20.2. if then else.	169
20.3. if then elif.	.169
20.4. for loop.	.169
20.5. while loop.	.170
20.6. until loop.	.171
20.7. practică: test scripting și loop-uri.	.172
20.8. soluție: test scripting și loop-uri.	173

20.1. test []

Comanda **test** poate testa dacă ceva este adevărat sau fals. Să începem prin a testa dacă 10 este mai mare decât 55.

```
[paul@RHEL4b ~]$ test 10 -gt 55 ; echo $?  
1  
[paul@RHEL4b ~]$
```

Comanda **test** dă 1 dacă testul eșuează. Și cum vedeți în următoarea captură de ecran, **test** dă 0 când un text este corect.

```
[paul@RHEL4b ~]$ test 56 -gt 55 ; echo $?  
0  
[paul@RHEL4b ~]$
```

Dacă preferați ca rezultatul să afișeze adevărat sau fals, atunci scrieți comanda **test** ca aceasta.

```
[paul@RHEL4b ~]$ test 56 -gt 55 && echo true || echo false  
true  
[paul@RHEL4b ~]$ test 6 -gt 55 && echo true || echo false  
false
```

Comanda **test** poate să fie scrisă de asemeni ca paranteze pătrate, captura de ecran de mai jos este identică cu cea de mai sus.

```
[paul@RHEL4b ~]$ [ 56 -gt 55 ] && echo true || echo false  
true  
[paul@RHEL4b ~]$ [ 6 -gt 55 ] && echo true || echo false  
false
```

Mai jos sînt cîteva exemple **test**. Priviți la **man test** pentru a vedea mai multe opțiuni pentru teste.

```
[ -d foo ] Directorul foo există?  
[ -e bar ] Fișierul bar există?  
[ '/etc' = $PWD ] Este șirul /etc egal cu variabila $PWD?  
[ $1 != 'secret' ] Este primul parametru diferit de secret?  
[ 55 -lt $bar ] Este 55 mai mic de valoarea lui $bar?  
[ $foo -ge 1000 ] Este valoarea $foo mai mare sau egală cu 1000?  
[ "abc" < $bar ] Este abc sortată înainte de valoarea lui $bar?  
[ -f foo ] Este foo un fișier regular?  
[ -r bar ] Este bar un fișier care poate fi citit?  
[ foo -nt bar ] Este fișierul foo mai nou decât fișierul bar?  
[ -o nounset ] Este opțiunea shell nounset setată?
```

test poate fi combinat cu **ȘI** sau **ORI** logic.


```
paul@RHEL4b:~$ [ 66 -gt 55 -a 66 -lt 500 ] && echo true || echo false
true
paul@RHEL4b:~$ [ 66 -gt 55 -a 660 -lt 500 ] && echo true || echo false
false
paul@RHEL4b:~$ [ 66 -gt 55 -o 660 -lt 500 ] && echo true || echo false
true
```

20.2. if then else

În construcția **if then else** este vorba despre alegere. Dacă o anumită condiție este întâlnită, *if* execută ceva, *else* execută altceva. Exemplul de mai jos testează dacă un fișier există, și dacă fișierul există atunci un mesaj bine definit are ecou.

```
#!/bin/bash

if [ -f isit.txt ]
then echo isit.txt exists!
else echo isit.txt not found!
fi
```

Dacă numim scriptul de mai sus 'choice', atunci el se execută astfel.

```
[paul@RHEL4a scripts]$ ./choice
isit.txt not found!
[paul@RHEL4a scripts]$ touch isit.txt
[paul@RHEL4a scripts]$ ./choice
isit.txt exists!
[paul@RHEL4a scripts]$
```

20.3. if then elif

Puteți face serie unui nou **if** înăuntrul unui **else** cu **elif**. Acesta este un exemplu simplu.

```
#!/bin/bash
count=42
if [ $count -eq 42 ]
then
    echo "42 is correct."
elif [ $count -gt 42 ]
then
    echo "Too much."
else
    echo "Not enough."
fi
```

20.4. for loop

Exemplul de mai jos arată sintaxa unui **for loop** clasic în bash.

```
for i in 1 2 4
do
    echo $i
done
```

Un exemplu de **for loop** combinat cu un shell înglobat.

```
#!/bin/ksh
for counter in `seq 1 20`
do
    echo counting from 1 to 20, now at $counter
    sleep 1
done
```

Același exemplu de mai sus poate fi scris fără shell-ul înglobat folosind stenografia bash **{from..to}**

```
#!/bin/bash
for counter in {1..20}
do
    echo counting from 1 to 20, now at $counter
    sleep 1
done
```

Acest **for loop** folosește globulizare fișier (din expansiunea shell-ului). A pune instrucțiunea pe linia de comandă are funcționalitate identică ca rezultat.

```
kahlan@solexp11$ ls
count.ksh go.ksh
kahlan@solexp11$ for file in *.ksh ; do cp $file $file.backup ; done
kahlan@solexp11$ ls
count.ksh count.ksh.backup go.ksh go.ksh.backup
```

20.5. while loop

Mai jos este un exemplu simplu a unui **while loop**.

```
i=100;
while [ $i -ge 0 ] ;
do
    echo Counting down, from 100 to 0, now at $i;
    let i--;
done
```

Loop-uri fără sfârșit pot fi făcute cu **while true** sau **while :**, unde **două puncte** este echivalentul al **no operation** în shell-urile **korn** și **bash**.

```
#!/bin/ksh
# endless loop
while :
do
    echo hello
    sleep 1
done
```

20.6. until loop

Mai jos este un exemplu simplu al unui **until loop**.

```
let i=100;
until [ $i -le 0 ] ;
do
    echo Counting down, from 100 to 1, now at $i;
    let i--;
done
```

20.7. practică: test scripting și loop-uri

1. Scrieți un script care folosește un loop **for** pentru a număra de la 3 la 7.
2. Scrieți un script care folosește un loop **for** pentru a număra de la 1 la 17000.
3. Scrieți un script care folosește un loop **while** pentru a număra de la 3 la 7.
4. Scrieți un script care folosește un loop **until** pentru a număra de la 8 la 4.
5. Scrieți un script care numără numărul fișierelor care se termină în **.txt** în directorul curent.
6. Ascundeți o declarație **if** împrejurul unui script astfel încât să fie de asemeni corect când există zero fișiere care se termină în **.txt**.

20.8. soluție: test scripting și loop-uri

1. Scrieți un script care folosește un loop **for** pentru a număra de la 3 la 7.

```
#!/bin/bash

for i in 3 4 5 6 7
do
    echo Counting from 3 to 7, now at $i
done
```

2. Scrieți un script care folosește un loop **for** pentru a număra de la 1 la 17000.

```
#!/bin/bash

for i in `seq 1 17000`
do
    echo Counting from 1 to 17000, now at $i
done
```

3. Scrieți un script care folosește un loop **while** pentru a număra de la 3 la 7.

```
#!/bin/bash

i=3
while [ $i -le 7 ]
do
    echo Counting from 3 to 7, now at $i
    let i=i+1
done
```

4. Scrieți un script care folosește un loop **until** pentru a număra de la 8 la 4.

```
#!/bin/bash

i=8
until [ $i -lt 4 ]
do
    echo Counting down from 8 to 4, now at $i
    let i=i-1
done
```

5. Scrieți un script care numără numărul fișierelor care se termină în **.txt** în directorul curent.

```
#!/bin/bash

let i=0
for file in *.txt
do
    let i++
done
echo "There are $i files ending in .txt"
```

6. Ascundeți o declarație **if** împrejurul unui script astfel încât să fie de asemeni corect când există zero fișiere care se termină în **.txt**.

```
#!/bin/bash

ls *.txt > /dev/null 2>&1
if [ $? -ne 0 ]
then echo "There are 0 files ending in .txt"
else
    let i=0
    for file in *.txt
do
    let i++
done
echo "There are $i files ending in .txt"
fi
```

Capitolul 21. parametri scripting

Conținut

21.1. parametri script.	176
21.2. schimbați între parametri.	177
21.3. intrare runtime.	177
21.4. faceți sursa unui fișier de configurare.	177
21.5. luați opțiuni script cu getopt.	178
21.6. luați opțiuni shell cu shopt.	180
21.7. practică: parametri și opțiuni.	181
21.8. soluție: parametri și opțiuni.	182

21.1. parametri script

Un script shell **bash** poate avea parametri. Numerele pe care le vedeți în scriptul de mai jos continuă dacă aveți mai mulți parametri. Există parametri speciali conținând numărul parametrilor, un șir al tuturor parametrilor, și de asemenea process id, și codul last return. Pagina de manual **bash** are o listă deplină.

```
#!/bin/bash
echo The first argument is $1
echo The second argument is $2
echo The third argument is $3

echo \$ $$ PID of the script
echo \# $# count arguments
echo \? $? last return code
echo \* $* all the arguments
```

Mai jos este ieșirea script-ului de deasupra în acțiune.

```
[paul@RHEL4a scripts]$ ./pars one two three
The first argument is one
The second argument is two
The third argument is three
$ 5610 PID of the script
# 3 count arguments
? 0 last return code
* one two three all the arguments
```

Încă odată același script, dar doar cu doi parametri.

```
[paul@RHEL4a scripts]$ ./pars 1 2
The first argument is 1
The second argument is 2
The third argument is
$ 5612 PID of the script
# 2 count arguments
? 0 last return code
* 1 2 all the arguments
[paul@RHEL4a scripts]$
```

Aici este un alt exemplu, unde folosim **\$0**. Parametrul **\$0** conține numele scriptului.

```
paul@debian6~$ cat myname
echo this script is called $0
paul@debian6~$ ./myname
this script is called ./myname
paul@debian6~$ mv myname test42
paul@debian6~$ ./test42
this script is called ./test42
```


21.2. schimbați între parametri

Declarația **shift** poate analiza toți **parametrii** unul câte unul. Acesta este un exemplu script.

```
kahlan@solexp11$ cat shift.ksh
#!/bin/ksh
if [ "$#" == "0" ]
then
    echo You have to give at least one parameter.
    exit 1
fi

while (( $# ))
do
    echo You gave me $1
    shift
done
```

Mai jos sînt unele exemple de ieșire din scriptul de de-asupra.

```
kahlan@solexp11$ ./shift.ksh one
You gave me one
kahlan@solexp11$ ./shift.ksh one two three 1201 "33 42"
You gave me one
You gave me two
You gave me three
You gave me 1201
You gave me 33 42
kahlan@solexp11$ ./shift.ksh
You have to give at least one parameter.
```

21.3. intrare runtime

Puteți solicita utilizatorului informație cu comanda **read** într-un script.

```
#!/bin/bash
echo -n Enter a number:
read number
```

21.4. faceți sursa unui fișier de configurare

Sursa (după câte am văzut în capitolele shell) poate fi folosită pentru a da sursă unui fișier de configurare.

Mai jos este un exemplu de fișier de configurare pentru o aplicație.

```
[paul@RHEL4a scripts]$ cat myApp.conf
# The config file of myApp
```

```
# Enter the path here
myAppPath=/var/myApp
```

```
# Enter the number of quines here
quines=5
```

Și aici este o aplicație care utilizează acest fișier.

```
[paul@RHEL4a scripts]$ cat myApp.bash
#!/bin/bash
#
# Welcome to the myApp application
#
```

```
. ./myApp.conf
```

```
echo There are $quines quines
```

Aplicația care rulează poate folosi valorile dinăuntrul fișierului de configurare sursă.

```
[paul@RHEL4a scripts]$ ./myApp.bash
There are 5 quines
[paul@RHEL4a scripts]$
```

21.5. luați opțiuni script cu getopt

Funcția **getopts** permite să analizați opțiunile date unei comenzi. Următorul script permite orice combinație a opțiunilor a, f și z.

```
kahlan@solexp11$ cat options.ksh
#!/bin/ksh
while getopts ":afz" option;
do
case $option in
  a)
    echo received -a
    ;;
  f)
    echo received -f
    ;;
  z)
    echo received -z
    ;;
  *)
    echo "invalid option -$OPTARG"
    ;;
esac
done
```

Acesta este un exemplu de ieșire din scriptul de de-asupra. Mai întâi folosim opțiunile corecte, apoi introducem de două ori o opțiune invalidă.

```
kahlan@solexp11$ ./options.ksh
kahlan@solexp11$ ./options.ksh -af
received -a
received -f
kahlan@solexp11$ ./options.ksh -zfg
received -z
received -f
invalid option -g
kahlan@solexp11$ ./options.ksh -a -b -z
received -a
invalid option -b
received -z
```

Puteți de asemeni să căutați opțiuni care au nevoie de un argument, precum arată acest exemplu.

```
kahlan@solexp11$ cat argoptions.ksh
#!/bin/ksh

while getopts ":af:z" option;
do
case $option in
  a)
    echo received -a
    ;;
  f)
    echo received -f with $OPTARG
    ;;
  z)
    echo received -z
    ;;
  :)
    echo "option -$OPTARG needs an argument"
    ;;
  *)
    echo "invalid option -$OPTARG"
    ;;
esac
done
```

Acesta este un exemplu de ieșire din script-ul de de-asupra.

```

kahlan@solexp11$ ./argoptions.ksh -a -f hello -z
received -a
received -f with hello
received -z
kahlan@solexp11$ ./argoptions.ksh -zaf 42
received -z
received -a
received -f with 42
kahlan@solexp11$ ./argoptions.ksh -zf
received -z
option -f needs an argument

```

21.6. luați opțiuni shell cu shopt

Puteți schimba valorile variabilelor controlînd comportamentul shell-ului opțional cu comanda internă shell **shopt**. Exemplul de mai jos mai întîi verifică dacă opțiunea `cdspell` este setată; nu este setată. Următoarea comandă `shopt` setează valoarea, și a treia comandă `shopt` verifică dacă opțiunea este într-adevăr setată. Puteți de-acum înainte să folosiți greșeli minore de scriere în comanda `cd`. Pagina de manual `bash` are o listă completă a opțiunilor.

```

paul@laika:~$ shopt -q cdspell ; echo $?
1
paul@laika:~$ shopt -s cdspell
paul@laika:~$ shopt -q cdspell ; echo $?
0
paul@laika:~$ cd /Etc
/etc

```

21.7. practică: parametri și opțiuni

1. Scrieți un script care primește patru parametri, și le face ieșirea în ordine inversă.
2. Scrieți un script care primește doi parametri (două nume de fișiere) și arată ieșirile, dacă acele fișiere există.
3. Scrieți un script care solicită un nume de fișier. Verificați existența fișierului, apoi verificați dacă dețineți fișierul, și dacă are permisiuni de scriere. Dacă nu are, atunci adăugați permisiunea de a fi scris.
4. Faceți un fișier de configurare pentru scriptul anterior. Puneți un întrerupător de logare în fișierul de configurare, de logare înseamnă să scrie ieșiri detaliate a tot ceea ce face script-ul într-un fișier de jurnalizare în /tmp.

21.8. soluție: parametri și opțiuni

1. Scrieți un script care primește patru parametri, și le face ieșirea în ordine inversă.

```
echo $4 $3 $2 $1
```

2. Scrieți un script care primește doi parametri (două nume de fișiere) și arată dacă acele fișiere există.

```
#!/bin/bash
```

```
if [ -f $1 ]
then echo $1 exists!
else echo $1 not found!
fi
```

```
if [ -f $2 ]
then echo $2 exists!
else echo $2 not found!
fi
```

3. Scrieți un script care solicită un nume de fișier. Verificați existența fișierului, apoi verificați dacă dețineți fișierul, și dacă are permisiuni de scriere. Dacă nu are, atunci adăugați permisiunea de a fi scris.

4. Faceți un fișier de configurare pentru scriptul anterior. Puneți un întrerupător de logare în fișierul de configurare, de logare înseamnă să scrie ieșiri detaliate a tot ceea ce face script-ul într-un fișier de jurnalizare în /tmp.

Capitolul 22. mai mult scripting

Conținut

22.1. eval.	.184
22.2. (()).	184
22.3. let.	184
22.4. case.	.185
22.5. funcții shell.	186
22.6. practică: mai mult scripting.	.188
22.7. soluție: mai mult scripting.	189

22.1 eval

eval citește argumente ca informație către shell (comenzile care rezultă sînt executate). Asta permite folosirea valorii unei variabile ca o variabilă.

```
paul@deb503:~/test42$ answer=42
paul@deb503:~/test42$ word=answer
paul@deb503:~/test42$ eval x=\$$word ; echo $x
42
```

Și în **bash** și în **Korn** argumentele pot fi puse între ghilimele.

```
kahlan@solexp11$ answer=42
kahlan@solexp11$ word=answer
kahlan@solexp11$ eval "y=\$$word" ; echo $y
42
```

Uneori este nevoie de **eval** pentru a avea o analiză corectă a argumentelor. Luați în considerare acest exemplu unde comanda **date** primește un parametru **1 week ago**.

```
paul@debian6~$ date --date="1 week ago"
Thu Mar 8 21:36:25 CET 2012
```

Cînd setăm această comandă într-o variabilă, atunci executînd acea variabilă eșuează dacă nu folosim **eval**.

```
paul@debian6~$ lastweek='date --date="1 week ago"'
paul@debian6~$ $lastweek
date: extra operand `ago'
Try `date --help' for more information.
paul@debian6~$ eval $lastweek
Thu Mar 8 21:36:39 CET 2012
```

22.2. (())

Parantezele **(())** permit evaluarea expresiilor numerice.

```
paul@deb503:~/test42$ (( 42 > 33 )) && echo true || echo false
true
paul@deb503:~/test42$ (( 42 > 1201 )) && echo true || echo false
false
paul@deb503:~/test42$ var42=42
paul@deb503:~/test42$ (( 42 == var42 )) && echo true || echo false
true
paul@deb503:~/test42$ (( 42 == $var42 )) && echo true || echo false
true
paul@deb503:~/test42$ var42=33
paul@deb503:~/test42$ (( 42 == var42 )) && echo true || echo false
false
```

22.3 let

Funcția internă shell **let** dă instrucțiuni shell-ului să facă o evaluare a expresiilor aritmetice. Va da 0 doar dacă ultima expresie aritmetică o evaluează la

0.

```
[paul@RHEL4b ~]$ let x="3 + 4" ; echo $x
7
[paul@RHEL4b ~]$ let x="10 + 100/10" ; echo $x
20
[paul@RHEL4b ~]$ let x="10-2+100/10" ; echo $x
18
[paul@RHEL4b ~]$ let x="10*2+100/10" ; echo $x
30
```

Shell-ul poate de asemeni să schimbe între baze diferite.

```
[paul@RHEL4b ~]$ let "x=0xFF" ; echo $x
255
[paul@RHEL4b ~]$ let x="0xC0" ; echo $x
192
[paul@RHEL4b ~]$ let x="0xA8" ; echo $x
168
[paul@RHEL4b ~]$ let x="8#70" ; echo $x
56
[paul@RHEL4b ~]$ let x="8#77" ; echo $x
63
[paul@RHEL4b ~]$ let x="16#c0" ; echo $x
192
```

Există o diferență între a desemna o variabilă direct, sau utilizînd **let** pentru a evalua expresiile aritmetice (chiar dacă este doar pentru a desemna o valoare).

```
kahlan@solexp11$ dec=15 ; oct=017 ; hex=0x0f
kahlan@solexp11$ echo $dec $oct $hex
15 017 0x0f
kahlan@solexp11$ let dec=15 ; let oct=017 ; let hex=0x0f
kahlan@solexp11$ echo $dec $oct $hex
15 15 15
```

22.4. case

Uneori puteți simplifica afirmațiile **nested if** cu construcția **case**.

```

[paul@RHEL4b ~]$ ./help
What animal did you see ? lion
You better start running fast!
[paul@RHEL4b ~]$ ./help
What animal did you see ? dog
Don't worry, give it a cookie.
[paul@RHEL4b ~]$ cat help
#!/bin/bash
#
# Wild Animals Helpdesk Advice
#
echo -n "What animal did you see ? "
read animal
case $animal in
    "lion" | "tiger")
        echo "You better start running fast!"
        ;;
    "cat")
        echo "Let that mouse go..."
        ;;
    "dog")
        echo "Don't worry, give it a cookie."
        ;;
    "chicken" | "goose" | "duck" )
        echo "Eggs for breakfast!"
        ;;
    "liger")
        echo "Approach and say 'Ah you big fluffy kitty...'"
        ;;
    "babelfish")
        echo "Did it fall out your ear ?"
        ;;
    *)
        echo "You discovered an unknown animal, name it!"
        ;;
esac
[paul@RHEL4b ~]$

```

22.5. funcții shell

Funcțiile shell pot fi utilizate pentru a grupa comenzile într-o modalitate logică.

```
kahlan@solexp11$ cat funcs.ksh
#!/bin/ksh
```

```
function greetings {
echo Hello World!
echo and hello to $USER to!
}
```

```
echo We will now call a function
greetings
echo The end
```

Aici este o ieșire exemplu din scriptul anterior cu o **funcție**.

```
kahlan@solexp11$ ./funcs.ksh
We will now call a function
Hello World!
and hello to kahlan to!
The end
```

O funcție shell poate de asemenea să primească parametri.

```
kahlan@solexp11$ cat addfunc.ksh
#!/bin/ksh
```

```
function plus {
let result="$1 + $2"
echo $1 + $2 = $result
}
```

```
plus 3 10
plus 20 13
plus 20 22
```

Acest script produce următoarea ieșire.

```
kahlan@solexp11$ ./addfunc.ksh
3 + 10 = 13
20 + 13 = 33
20 + 22 = 42
```

22.6. practică: mai mult scripting

1. Scrieți un script care solicită două numere, și face ieșirea sumei și a produsului (cum este arătat aici).

```
Enter a number: 5
```

```
Enter another number: 2
```

```
Sum: 5 + 2 = 7
```

```
Product: 5 x 2 = 10
```

2. Îmbunătățiți script-ul de mai sus pentru a testa dacă numerele sînt între 1 și 100, ieșire cu o eroare dacă este necesar.

3. Îmbunătățiți script-ul de mai sus pentru a felicita utilizatorul dacă suma este egală cu produsul.

4. Scrieți un script cu declarație caractere insenzitive, folosind opțiunea shopt nocasematch. Opțiunea nocasematch este resetată la valoarea pe care o avea înainte ca script-ul să pornească.

5. Dacă timpul permite (sau dacă așteptați ca ceilalți studenți să termine această practică), uitați-vă la script-urile de sistem linux în /etc/init.d și /etc/rc.d și încercați să le înțelegeți. Unde începe execuția unui script în /etc/init.d/samba? Există de asemeni cîteva script-uri ascunse în ~, le vom discuta mai tîrziu.

22.7. soluție: mai mult scripting

1. Scrieți un script care solicită două numere, și face ieșirea sumei și a produsului (cum este arătat aici).

```
Enter a number: 5
```

```
Enter another number: 2
```

```
Sum: 5 + 2 = 7
```

```
Product: 5 x 2 = 10
```

```
#!/bin/bash
```

```
echo -n "Enter a number : "
```

```
read n1
```

```
echo -n "Enter another number : "
```

```
read n2
```

```
let sum="$n1+$n2"
```

```
let pro="$n1*$n2"
```

```
echo -e "Sum\t: $n1 + $n2 = $sum"
```

```
echo -e "Product\t: $n1 * $n2 = $pro"
```

2. Îmbunătățiți script-ul de mai sus pentru a testa dacă numerele sînt între 1 și 100, ieșire cu o eroare dacă e necesar.

```
echo -n "Enter a number between 1 and 100 : "
```

```
read n1
```

```
if [ $n1 -lt 1 -o $n1 -gt 100 ]
```

```
then
```

```
echo Wrong number...
```

```
exit 1
```

```
fi
```

3. Îmbunătățiți script-ul de mai sus pentru a felicita utilizatorul dacă suma este egală cu produsul.

```
if [ $sum -eq $pro ]
```

```
then echo Congratulations $sum == $pro
```

```
fi
```

4. Scrieți un script cu declarație caractere insenzitive, folosind opțiunea shopt nocasematch. Opțiunea nocasematch este resetată la valoarea pe care o avea înainte ca script-ul să pornească.

```
#!/bin/bash
#
# Wild Animals Case Insensitive Helpdesk Advice
#

if shopt -q nocasematch; then
    nocase=yes;
else
    nocase=no;
    shopt -s nocasematch;
fi

echo -n "What animal did you see ? "
read animal
case $animal in
    "lion" | "tiger")
        echo "You better start running fast!"
        ;;
    "cat")
        echo "Let that mouse go..."
        ;;
    "dog")
        echo "Don't worry, give it a cookie."
        ;;
    "chicken" | "goose" | "duck" )
        echo "Eggs for breakfast!"
        ;;
    "liger")
        echo "Approach and say 'Ah you big fluffy kitty.'"
        ;;
    "babelfish")
        echo "Did it fall out your ear ?"
        ;;
    *)
        echo "You discovered an unknown animal, name it!"
        ;;
esac

if [ nocase = yes ] ; then
    shopt -s nocasematch;
else
    shopt -u nocasematch;
fi
```

5. Dacă timpul permite (sau dacă așteptați ca ceilalți studenți să termine această practică), uitați-vă la script-urile de sistem linux în /etc/init.d și /etc/rc.d și încercați să le înțelegeți. Unde începe execuția unui script în /etc/init.d/samba? Există de asemenea câteva script-uri ascunse în ~, le vom discuta mai târziu.

Partea VII. management utilizator local

Capitolul 23. utilizatori

Conținut

23.1.	identificare.	.193
23.2.	utilizatori.	194
23.3.	parole.	.195
23.4.	directoare home.	200
23.5.	shell utilizator.	.200
23.6.	schimbați utilizatori cu su.	201
23.7.	executați un program ca un alt utilizator.	202
23.8.	practică: utilizatori.	204
23.9.	soluție: utilizatori.	.205
23.10.	mediu shell.	207

23.1. identificare

whoami

Comanda **whoami** vă spune username-ul pe care îl aveți.

```
[root@RHEL5 ~]# whoami
root
[root@RHEL5 ~]# su - paul
[paul@RHEL5 ~]$ whoami
paul
```

who

Comanda **who** vă furnizează informații despre cine este intrat în sistem.

```
[paul@RHEL5 ~]$ who
root      tty1      2008-06-24 13:24
sandra    pts/0      2008-06-24 14:05 (192.168.1.34)
paul      pts/1      2008-06-24 16:23 (192.168.1.37)
```

who am i

Cu **who am i** comanda who va afișa doar linia îndreptată către sesiunea curentă.

```
[paul@RHEL5 ~]$ who am i
paul      pts/1      2008-06-24 16:23 (192.168.1.34)
```

w

Comanda **w** vă arată cine este intrat în sistem și ce fac utilizatorii.

```
$ w
05:13:36 up 3 min, 4 users, load average: 0.48, 0.72, 0.33
USER  TTY      FROM          LOGIN@      IDLE   JCPU   PCPU   WHAT
root  tty1      -              05:11       2.00s  0.32s  0.27s  find / -name shad
inge  pts/0    192.168.1.33  05:12       0.00s  0.02s  0.02s  -ksh
paul  pts/2    192.168.1.34  05:13      25.00s  0.07s  0.04s  top
```

id

Comanda **id** afișează user-ul id, grupul primar id, și o listă a grupurilor de care aparțineți.

```
root@laika:~# id
uid=0(root) gid=0(root) groups=0(root)
root@laika:~# su - brel
brel@laika:~$ id
uid=1001(brel) gid=1001(brel) groups=1001(brel),1008(chanson),11578(wolf)
```

23.2. utilizatori

managementul utilizatorilor

Managementul utilizatorilor pe orice Unix poate fi făcut în trei modalități complementare. Puteți folosi utilitare **grafice** date de distribuție. Aceste utilitare au un aspect care depinde de distribuție. Dacă sînteți un utilizator de Linux novice pe computerul de acasă, atunci folosiți utilitarul grafic care este dat de către distribuție. Asta vă dă siguranța că nu veți întâmpina probleme.

O altă opțiune este să folosim **utilitare de linie de comandă** ca useradd, usermod, gpasswd, passwd și altele. Administratorii de servere sînt predispuși să utilizeze aceste utilitare, deoarece ele sînt familiare și foarte similare în multe distribuții diferite. Acest capitol se va focaliza pe aceste utilitare de linie de comandă.

O a treia și mai curînd cale extremă este de a **edita fișierele de configurare locale** direct folosind vi (sau vipw/vigr). Nu încercați asta dacă sînteți novice pe sisteme de producție!

/etc/passwd

Baza de date locală a utilizatorilor pe Linux (și pe cele mai multe sisteme Unix) este **/etc/passwd**.

```
[root@RHEL5 ~]# tail /etc/passwd
inge:x:518:524:art dealer:/home/inge:/bin/ksh
ann:x:519:525:flute player:/home/ann:/bin/bash
frederik:x:520:526:rubius poet:/home/frederik:/bin/bash
steven:x:521:527:roman emperor:/home/steven:/bin/bash
pascale:x:522:528:artist:/home/pascale:/bin/ksh
geert:x:524:530:kernel developer:/home/geert:/bin/bash
wim:x:525:531:master damuti:/home/wim:/bin/bash
sandra:x:526:532:radish stresser:/home/sandra:/bin/bash
annelies:x:527:533:sword fighter:/home/annelies:/bin/bash
laura:x:528:534:art dealer:/home/laura:/bin/ksh
```

După cum puteți vedea, acest fișier conține șapte coloane separate de două puncte. Coloanele conțin numele de utilizator, un x, userul id, grupul primar id, o descriere, numele directorului home, și shell-ul de login.

root

Utilizatorul **root** de asemeni numit **superutilizator** este cel mai puternic cont pe sistemul Linux. Acest utilizator poate face aproape tot, incluzînd crearea de alți utilizatori. Utilizatorul root întotdeauna are userid 0 (fără a lua în seamă numele contului).

```
[root@RHEL5 ~]# head -1 /etc/passwd
root:x:0:0:root:/root:/bin/bash
```

useradd

Puteți adăuga utilizatori cu comanda **useradd**. Exemplul de mai jos arată cum să adăugăm un utilizator numit yanina (ultimul parametru) și în același timp forțarea

creării directorului home (-m), setînd numele directorului home (-d), și setînd o descriere (-c).

```
[root@RHEL5 ~]# useradd -m -d /home/yanina -c "yanina wickmayer" yanina
[root@RHEL5 ~]# tail -1 /etc/passwd
yanina:x:529:529:yanina wickmayer:/home/yanina:/bin/bash
```

Utilizatorul numit yanina a primit userid 529 și **grupul primar** id 529.

/etc/default/useradd

Și Red Hat Enterprise Linux și Debian/Ubuntu au un fișier numit **/etc/default/useradd** care conține cîteva opțiuni user default. Pe lîngă faptul că puteți folosi cat pentru a afișa acest fișier, puteți de asemeni să utilizați **useradd -D**.

```
[root@RHEL4 ~]# useradd -D
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
```

userdel

Puteți șterge utilizatorul yanina cu **userdel**. Opțiunea -r a userdel va șterge și directorul home.

```
[root@RHEL5 ~]# userdel -r yanina
```

usermod

Puteți modifica proprietățile unui utilizator cu comanda **usermod**. Acest exemplu folosește **usermod** pentru a schimba descrierea utilizatorului harry.

```
[root@RHEL4 ~]# tail -1 /etc/passwd
harry:x:516:520:harry potter:/home/harry:/bin/bash
[root@RHEL4 ~]# usermod -c 'wizard' harry
[root@RHEL4 ~]# tail -1 /etc/passwd
harry:x:516:520:wizard:/home/harry:/bin/bash
```

23.3. parole

passwd

Parolele utilizatorilor pot fi setate cu comanda **passwd**. Utilizatorii vor trebui să scrie vechile lor parole înainte de a scrie de două ori parola nouă.

```
[harry@RHEL4 ~]$ passwd
Changing password for user harry.
Changing password for harry
(current) UNIX password:
New UNIX password:
BAD PASSWORD: it's WAY too short
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
[harry@RHEL4 ~]$
```

Precum puteți vedea, utilitarul `passwd` va face unele verificări de bază pentru a preveni utilizatorii să utilizeze parole prea simple. Utilizatorul `root` nu trebuie să urmeze aceste reguli (va exista un avertisment totuși). De asemeni utilizatorul `root` nu trebuie să scrie vechea parolă înainte de a introduce parola nouă de două ori.

/etc/shadow

Parolele utilizatorilor sînt criptate și ținute în **/etc/shadow**. Fișierul `/etc/shadow` are numai drepturi de citire și poate fi citit doar de către `root`. Vom vedea în secțiunea permisiuni fișiere cum este posibil pentru utilizatori să-și schimbe parola. Deocamdată, trebuie să știți că utilizatorii pot să-și schimbe parola cu comanda **/usr/bin/passwd**.

```
[root@RHEL5 ~]# tail /etc/shadow
inge:$1$yWMSimOV$YsYvcVKqByFVYlKnU3ncd0:14054:0:99999:7:::
ann:!!:14054:0:99999:7:::
frederik:!!:14054:0:99999:7:::
steven:!!:14054:0:99999:7:::
pascale:!!:14054:0:99999:7:::
geert:!!:14054:0:99999:7:::
wim:!!:14054:0:99999:7:::
sandra:!!:14054:0:99999:7:::
annelies:!!:14054:0:99999:7:::
laura:$1$Tvby1Kpa$LL.WzgobujUS3LC1IRmdv1:14054:0:99999:7:::
```

Fișierul **/etc/shadow** conține nouă semne două puncte coloane separate. Al nouălea cîmp conține (de la stînga la dreapta) numele utilizatorului, parola criptată (observați că doar `inge` și `laura` au o parolă criptată), ziua în care parola a fost schimbată ultima dată (ziua 1 este 1 ianuarie, 1970), numărul de zile pînă cînd parola trebuie lăsată neschimbată, ziua expirării parolei, atenționare asupra numărului de zile înainte de expirarea parolei, numărul de zile de expirare înainte de dezautorizarea contului și ziua în care contul a fost șters (din nou, din 1970). Nu explicăm încă ultimul cîmp.

cripatea parolei

criptare cu passwd

Parolele sînt stocate într-un format criptat. Această criptare este făcută de funcția **crypt**. Cea mai ușoară (și recomandată) modalitate de adăugare a unui utilizator cu o parolă la sistem este de a-l adăuga cu comanda **useradd -m user**, și apoi setînd parola utilizatorului cu **passwd**.

```
[root@RHEL4 ~]# useradd -m xavier
[root@RHEL4 ~]# passwd xavier
Changing password for user xavier.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
[root@RHEL4 ~]#
```

criptare cu openssl

O altă modalitate de a crea utilizatori cu parolă este de a folosi opțiunea `-p` a `useradd`, dar această opțiune cere o parolă criptată. Puteți genera această parolă criptată cu comanda **openssl passwd**.

```
[root@RHEL4 ~]# openssl passwd stargate
ZZNX16QZVgUQg
[root@RHEL4 ~]# useradd -m -p ZZNX16QZVgUQg mohamed
```

criptare cu crypt

O a treia opțiune este să creați propriul program C folosind funcția `crypt`, și să compilați asta într-o comandă.

```
[paul@laika ~]$ cat MyCrypt.c
#include <stdio.h>
#define __USE_XOPEN
#include <unistd.h>

int main(int argc, char** argv)
{
    if(argc==3)
    {
        printf("%s\n", crypt(argv[1],argv[2]));
    }
    else
    {
        printf("Usage: MyCrypt $password $salt\n" );
    }
    return 0;
}
```

Acest mic program poate fi compilat cu **gcc** astfel.

```
[paul@laika ~]$ gcc MyCrypt.c -o MyCrypt -lcrypt
```

Pentru a-l utiliza, trebuie să dăm doi parametri lui `MyCrypt`. Primul este parola necriptată, a doua este `salt`-ul. `Salt`-ul este folosit pentru a perturba algoritmul criptării în unul din 4096 de căi diferite. Această variație previne doi utilizatori cu aceeași parolă să aibă aceeași intrare în **/etc/shadow**.

```
paul@laika:~$ ./MyCrypt stargate 12
12L4FoTS3/k9U
paul@laika:~$ ./MyCrypt stargate 01
01Y.yPnlQ6R.Y
paul@laika:~$ ./MyCrypt stargate 33
330asFUbzgVeg
paul@laika:~$ ./MyCrypt stargate 42
42XFxoT4R75gk
```

Ați observat că primele două caractere ale parolei sînt **salt-ul**?

Ieșirea standard a funcției crypt folosește algoritmul DES care este vechi și poate fi spart în câteva minute. O metodă mai bună este a utiliza parole **md5** care pot fi recunoscute de un salt începînd cu \$1\$.

```
paul@laika:~$ ./MyCrypt stargate '$1$12'
$1$12$xUIQ4116Us.Q50sc2Khbm1
paul@laika:~$ ./MyCrypt stargate '$1$01'
$1$01$yNs8brjp4b4TEw.v9/ILJ/
paul@laika:~$ ./MyCrypt stargate '$1$33'
$1$33$tLh/Ldy2wskdKAJR.Ph4M0
paul@laika:~$ ./MyCrypt stargate '$1$42'
$1$42$Hb3nvP0KwHSQ7fQmILY7R.
```

md5 salt poate avea o lungime de pînă la opt caractere. Salt este afișat în **/etc/shadow** între al doilea și al treilea \$, așa că nu utilizați niciodată parola ca salt!

```
paul@laika:~$ ./MyCrypt stargate '$1$stargate'
$1$stargate$qqxoLqiSVNvGr5ybMxEVM1
```

parole default

/etc/login.defs

Fișierul **/etc/login.defs** conține unele setări default pentru parole utilizator ca învechirea parolei și setările lungimii. (Veți găsi de asemeni limite numerice a user id și grup id și dacă un director home ar trebui sau nu să fie creat prin default.)

```
[root@RHEL4 ~]# grep -i pass /etc/login.defs
# Password aging controls:
# PASS_MAX_DAYS Maximum number of days a password may be used.
# PASS_MIN_DAYS Minimum number of days allowed between password changes.
# PASS_MIN_LEN Minimum acceptable password length.
# PASS_WARN_AGE Number of days warning given before a password expires.
PASS_MAX_DAYS 99999
PASS_MIN_DAYS 0
PASS_MIN_LEN 5
PASS_WARN_AGE 7
```

chage

Comanda **chage** poate fi folosită pentru a seta o dată de expirare pentru un cont de utilizator (-E), o învechire a parolei situată la minimum (-m) și maximum (-M), o dată de expirare a parolei, și setarea numărului zilelor de avertizare înainte de expirarea parolei. Mult din această funcționalitate este de asemeni disponibilă din comanda **passwd**. Opțiunea -l a chage va afișa aceste setări pentru un utilizator.

```
[root@RHEL4 ~]# chage -l harry
Minimum:          0
Maximum:          99999
Warning:          7
Inactive:         -1
Last Change:      Jul 23, 2007
Password Expires: Never
Password Inactive: Never
Account Expires:  Never
[root@RHEL4 ~]#
```

dezactivarea unei parole

Parolele în **/etc/shadow** nu pot începe cu un semn al exclamării. Când al doilea câmp în **/etc/shadow** începe cu un semn al exclamării, atunci parola nu poate fi folosită.

Folosirea acestei posibilități este deseori numită **blocarea**, **dezautorizarea**, sau **suspendarea** a unui cont de utilizator. În afară de vi (sau vipw) puteți face asta cu **usermod**.

Prima linie în următoarea captură de ecran va dezautoriza parola pentru userul harry, făcând imposibil pentru harry de a se autentifica folosind această parolă.

```
[root@RHEL4 ~]# usermod -L harry
[root@RHEL4 ~]# tail -1 /etc/shadow
harry:!!$1$143T09IZ$RLm/FpQkpDrV4/Tkhku5e1:13717:0:99999:7:::
```

Utilizatorul root (și utilizatorii cu drepturi **sudo** asupra **su**) vor fi capabili în continuare să scrie **su** utilizatorului harry (pentru că parola nu este necesară aici). De asemeni observați că harry va fi în continuare capabil să intre în sistem dacă a setat o parolă fără ssh!

```
[root@RHEL4 ~]# su - harry
[harry@RHEL4 ~]$
```

Puteți debloca contul din nou folosind **usermod -U**.

Atenție la micile diferențe în opțiunile liniei de comandă a **passwd**, **usermod**, și **useradd** pe distribuții diferite! Verificați fișierele locale când utilizați capabilități ca **dezautorizarea**, **suspendarea**, sau **blocarea** utilizatorilor și a parolelor!

editarea fișierelor locale

Dacă totuși vreți să editați manual **/etc/passwd** sau **/etc/shadow**, după ce știți aceste comenzi pentru managementul parolei, atunci folosiți direct **vipw** în loc de vi(m). Utilitarul **vipw** va face sigură blocarea fișierului.

```
[root@RHEL5 ~]# vipw /etc/passwd
vipw: the password file is busy (/etc/ptmp present)
```

23.4. directoare home

crearea directoarelor home

Cea mai ușoară modalitate de a crea un director home este de a furniza opțiunea **-m** cu **useradd** (este deseori setat ca opțiune default pe Linux).

O cale mai puțin ușoară este de a crea un director home manual cu **mkdir** care de asemeni cere setarea proprietarului și permisiunilor pe director cu **chmod** și **chown** (ambele comenzi sînt discutate în detaliu într-un alt capitol).

```
[root@RHEL5 ~]# mkdir /home/laura
[root@RHEL5 ~]# chown laura:laura /home/laura
[root@RHEL5 ~]# chmod 700 /home/laura
[root@RHEL5 ~]# ls -ld /home/laura/
drwx----- 2 laura laura 4096 Jun 24 15:17 /home/laura/
```

/etc/skel

Cînd utilizăm **useradd** cu opțiunea **-m**, directorul **/etc/skel** este copiat noului director home creat. Directorul **/etc/skel** conține unele fișiere (de obicei ascunse) care conțin setările de profil și valorile default pentru aplicații. În această modalitate **/etc/skel** servește ca director home default și ca profil de utilizator default.

```
[root@RHEL5 ~]# ls -la /etc/skel/
total 48
drwxr-xr-x  2 root root  4096 Apr  1 00:11 .
drwxr-xr-x 97 root root 12288 Jun 24 15:36 ..
-rw-r--r--  1 root root   24 Jul 12 2006 .bash_logout
-rw-r--r--  1 root root  176 Jul 12 2006 .bash_profile
-rw-r--r--  1 root root  124 Jul 12 2006 .bashrc
```

ștergerea directoarelor home

Opțiunea **-r** a **userdel** ne va asigura că directorul home este șters împreună cu contul de utilizator.

```
[root@RHEL5 ~]# ls -ld /home/wim/
drwx----- 2 wim wim 4096 Jun 24 15:19 /home/wim/
[root@RHEL5 ~]# userdel -r wim
[root@RHEL5 ~]# ls -ld /home/wim/
ls: /home/wim/: No such file or directory
```

23.5 shell utilizator

shell login

Fișierul **/etc/passwd** specifică **shell-ul** **login** pentru utilizator. În captura de ecran de mai jos puteți vedea că utilizatorul annelies va intra în sistem în shell-ul **/bin/bash**, și utilizatorul laura cu shell-ul **/bin/ksh**.


```
[root@RHEL5 ~]# tail -2 /etc/passwd
annelies:x:527:533:sword fighter:/home/annelies:/bin/bash
laura:x:528:534:art dealer:/home/laura:/bin/ksh
```

Puteți folosi comanda **usermod** pentru a schimba shell-ul pentru un utilizator.

```
[root@RHEL5 ~]# usermod -s /bin/bash laura
[root@RHEL5 ~]# tail -1 /etc/passwd
laura:x:528:534:art dealer:/home/laura:/bin/bash
```

chsh

Utilizatorii pot să-și schimbe shell-ul de login cu comanda **chsh**. Mai întâi, utilizatorul **harry** obține o listă de shell-uri disponibile (ar putea să invoce comanda **cat /etc/shells**) și apoi să schimbe shell-ul de login în **shell-ul Korn** (/bin/ksh). La următoarea intrare în sistem, harry va intra în ksh în loc de bash în mod default.

```
[harry@RHEL4 ~]$ chsh -l
/bin/sh
/bin/bash
/sbin/nologin
/bin/ash
/bin/bsh
/bin/ksh
/usr/bin/ksh
/usr/bin/pdksh
/bin/tcsh
/bin/csh
/bin/zsh
[harry@RHEL4 ~]$ chsh -s /bin/ksh
Changing shell for harry.
Password:
Shell changed.
[harry@RHEL4 ~]$
```

23.6. schimbare utilizatori cu su

su la un alt utilizator

Comanda **su** permite ca un utilizator să ruleze un shell ca un alt utilizator.

```
[paul@RHEL4b ~]$ su harry
Password:
[harry@RHEL4b paul]$
```

su la root

Da de asemeni puteți scrie **su** pentru a deveni **root**, când știți **parola root**.

```
[harry@RHEL4b paul]$ su root
Password:
[root@RHEL4b paul]#
```

su ca root

Dacă nu sînteți intrat în sistem ca **root**, a executa un shell ca un alt utilizator cere să știți parola acelui utilizator. Utilizatorul **root** poate deveni orice utilizator fără a ști parola utilizatorului.

```
[root@RHEL4b paul]# su serena
[serena@RHEL4b paul]$
```

su -\$username

Prin default, comanda **su** menține același mediu shell. Pentru a deveni un alt utilizator și de asemeni de a obține mediul utilizatorului țintă, invocați comanda **su -** urmată de utilizatorul țintă.

```
[paul@RHEL4b ~]$ su - harry
Password:
[harry@RHEL4b ~]$
```

su -

Cînd nici un username nu este specificat comenzii **su** sau **su -**, comanda va înțelege că **root** este ținta.

```
[harry@RHEL4b ~]$ su -
Password:
[root@RHEL4b ~]#
```

23.7. executați un program ca un alt utilizator

despre sudo

Programul **sudo** permite unui utilizator să înceapă un program cu acreditările unui alt utilizator. Înainte ca asta să funcționeze, administratorul de sistem trebuie să seteze fișierul **/etc/sudoers**. Asta poate fi folositor pentru a delega sarcini administrative unui alt utilizator (fără a da parola de root).

Captura de ecran de mai jos arată folosirea lui **sudo**. Utilizatorul **paul** a primit dreptul de a executa **useradd** cu acreditările lui **root**. Asta îi permite lui **paul** să creeze noi utilizatori pe sistem fără a deveni **root** și fără a ști **parola root**.

```
paul@laika:~$ useradd -m inge
useradd: unable to lock password file
paul@laika:~$ sudo useradd -m inge
[sudo] password for paul:
paul@laika:~$
```

setuid pe sudo

Executabilul **sudo** are setat bitul **setuid**, astfel că orice utilizator poate să-l execute cu userid efectiv a lui root.

```
paul@laika:~$ ls -l `which sudo`
-rwsr-xr-x 2 root root 107872 2008-05-15 02:41 /usr/bin/sudo
paul@laika:~$
```

visudo

Vedeți pagina de manual a **visudo** înainte de a vă juca cu fișierul **/etc/sudoers**.

sudo su

Pe unele sisteme Linux ca Ubuntu și Kubuntu, utilizatorul **root** nu are o parolă setată. Asta înseamnă că nu este posibil să intrăm în sistem ca **root** (securitate extra). Pentru a face acțiuni ca **root**, primului utilizator îi sînt date toate **drepturile sudo** prin fișierul **/etc/sudoers**. De fapt toți utilizatorii care sînt membri a grupului **admin** pot folosi **sudo** pentru a executa toate comenzile ca **root**.

```
root@laika:~# grep admin /etc/sudoers
# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL
```

Rezultatul final este că utilizatorul poate să tasteze **sudo su** - și să devină **root** fără a trebui să scrie parola de **root**. Comanda **sudo** cere să tastați propria voastră parolă. De aceea promptul **password** din captura de ecran de mai jos este pentru **sudo**, nu pentru **su**.

```
paul@laika:~$ sudo su -
Password:
root@laika:~#
```

23.8 practică: utilizatori

1. Creați utilizatorii Serena Williams, Venus Williams și Justine Henin, toate cu parola setată la stargate, cu numele de utilizator (cu litere mici) ca prenume, și numele lor complet în comentariu. Verificați dacă utilizatorii și directoarele lor home sînt create în mod corespunzător.
2. Creați un utilizator numit **kornuser**, dați-i un shell Korn (/bin/ksh) ca shell-ul lui default. Intrați în sistem cu acest utilizator (pe o linie de comandă sau într-un tty).
3. Creați un utilizator numit **einstime** fără director home, dați-i shell-ul default de logon **/bin/date**. Ce se întîmplă cînd intrați în sistem cu acest utilizator? Vă puteți gîndi la un exemplu folositor din lumea reală schimbînd un shell de login a unui utilizator la o aplicație?
4. Încercați comenzile `who`, `whoami`, `who am i`, `w`, `id`, `echo $USER $UID`.
- 5a. Blocați contul de utilizator **venus** cu `usermod`.
- 5b. Folosiți **passwd -d** pentru dezabilita parola lui serena. Verificați linia serena în **/etc/shadow** înainte și după dezabilitare.
- 5c. Care este diferența dintre a bloca un cont de utilizator și a dezabilita parola contului unui utilizator?
6. Ca **root** schimbați parola lui **einstime** în stargate.
7. Acum încercați să schimbați parola lui serena în serena ca serena.
8. Asigurați-vă că fiecare utilizator nou trebuie să-și schimbe parola la fiecare 10 zile.
9. Setăți numărul zilelor de avertisment la 4 pentru kornuser.
- 10a. Setăți parola a doi utilizatori separați la stargate. Uitați-vă la parola criptată stargate în **/etc/shadow** și explicați.
- 10b. Faceți un backup ca root a fișierului **/etc/shadow**. Folosiți `vi` ca să copiați stargate criptat pentru un alt utilizator. Poate acum acest utilizator să intre în sistem cu stargate ca parolă?
11. Puneți un fișier în directorul skeleton și verificați dacă el e copiat în directorul home a utilizatorului. Cînd este copiat directorul skeleton?
12. De ce să folosim **vipw** în loc de **vi**? Care ar fi problema cînd utilizăm **vi** sau **vim**?
13. Folosiți `chsh` pentru a lista toate shell-urile, și comparați cu `cat /etc/shells`. Schimbați-vă shel-ul de login în shell-ul Korn, ieșiți din sistem și intrați din nou. Acum schimbați din nou shlell-ul în bash.
14. Care opțiune `useradd` vă permite să numiți un director home?
15. Cum puteți vedea dacă parola utilizatorului harry este blocată sau deblocată? Dați o soluție cu `grep` și o soluție cu `passwd`.

23.9. soluție: utilizatori

1. Creați utilizatorii Serena Williams, Venus Williams și Justine Henin, toate cu parola setată la stargate, cu numele de utilizator (cu litere mici) ca prenume, și numele lor complet în comentariu. Verificați dacă utilizatorii și directoarele lor home sînt create în mod corespunzător.

```
useradd -m -c "Serena Williams" serena ; passwd serena
useradd -m -c "Venus Williams" venus ; passwd venus
useradd -m -c "Justine Henin" justine ; passwd justine
tail /etc/passwd ; tail /etc/shadow ; ls /home
```

Păstrați numele de login cu litere mici!

2. Creați un utilizator numit **kornuser**, dați-i un shell Korn (/bin/ksh) ca shell-ul lui default. Intrați în sistem cu acest utilizator (pe o linie de comandă sau într-un tty).

```
useradd -s /bin/ksh kornuser ; passwd kornuser
```

3. Creați un utilizator numit **einstime** fără director home, dați-i shell-ul default de logon **/bin/date**. Ce se întîmplă cînd intrați în sistem cu acest utilizator? Vă puteți gîndi la un exemplu folositor din lumea reală schimbînd un shell de login a unui utilizator la o aplicație?

```
useradd -s /bin/date einstime ; passwd einstime
```

Poate fi folositor cînd utilizatorii trebuie să acceseze doar o singură aplicație pe server. Numai logîndu-i deschide aplicația pentru ei, și închizînd aplicația îi deloghează automat.

4. Încercați comenzile who, whoami, who am i, w, id, echo \$USER \$UID.

```
who ; whoami ; who am i ; w ; id ; echo $USER $UID
```

5a. Blocați contul de utilizator **venus** cu usermod.

```
usermod -L venus
```

5b. Folosiți **passwd -d** pentru dezabilita parola lui serena. Verificați linia serena în **/etc/shadow** înainte și după dezabilitare.

```
grep serena /etc/shadow ; passwd -d serena ; grep serena /etc/shadow
```

5c. Care este diferența dintre a bloca un cont de utilizator și a dezabilita parola contului unui utilizator?

Blocarea va preveni utilizatorul să intre în sistem cu parola lui (punînd un ! în fața parolei din /etc/shadow). Dezabilitarea cu passwd va șterge parola din /etc/shadow.

6. Ca **root** schimbați parola lui **einstime** în stargate.

```
Intrați în sistem ca root și tastați: passwd einstime
```

7. Acum încercați să schimbați parola lui serena în serena ca serena.

Intrați în sistem ca serena, apoi executați: `passwd serena`... ar trebui să eșueze!

8. Asigurați-vă că fiecare utilizator nou trebuie să-și schimbe parola la fiecare 10 zile.

Pentru un utilizator existent: `chage -M 10 serena`

Pentru toți utilizatorii noi: `vi /etc/login.defs` (și schimbați `PASS_MAX_DAYS` la 10)

9. Setati numărul zilelor de avertisment la 4 pentru kornuser.

`chage -W 4 kornuser`

10a. Setati parola a doi utilizatori separati la stargate. Uitati-vă la parola criptată stargate în `/etc/shadow` și explicați.

Dacă ați folosit `passwd`, atunci `salt` va fi diferit pentru cele două parole criptate.

10b. Faceți un backup ca root a fișierului `/etc/shadow`. Folosiți `vi` ca să copiați stargate criptat pentru un alt utilizator. Poate acum acest utilizator să intre în sistem cu stargate ca parolă?

Da.

11. Puneți un fișier în directorul `skeleton` și verificați dacă el este copiat în directorul `home` al utilizatorului. Când este copiat directorul `skeleton`?

Când creați un cont de utilizator cu un nou director `home`.

12. De ce să folosim **`vipw`** în loc de **`vi`**? Care ar fi problema când utilizăm **`vi`** sau **`vim`**?

`vipw` va da un mesaj de avertisment când altcineva deja utilizează acel fișier.

13. Folosiți `chsh` pentru a lista toate shell-urile, și comparați cu `cat /etc/shells`. Schimbați-vă shell-ul de login în shell-ul Korn, ieșiți din sistem și intrați din nou. Acum schimbați din nou shell-ul la `bash`.

Pe Red Hat Enterprise Linux: `chsh -l`

Pe Debian/Ubuntu: `cat /etc/shells`

14. Care opțiune `useradd` vă permite să numiți un director `home`?

`-d`

15. Cum puteți vedea dacă parola utilizatorului `harry` este blocată sau deblocată? Dați o soluție cu `grep` și o soluție cu `passwd`.

`grep harry /etc/shadow`

`passwd -S harry`

23.10. mediul shell

E frumos să avem aceste presetări, aliasuri și variabile la comandă, dar de unde vin ele? **Shell-ul** folosește un număr de fișiere de startup care sînt verificate (și executate) oricînd shell-ul este invocat. Ceea ce urmează este o trecere în revistă a scripturilor de startup.

/etc/profile

Ambele shell-uri **bash** și **ksh** vor verifica existența **/etc/profile** și-l va executa dacă el există.

Cînd citiți acest script, probabil că veți observa (cel puțin pe Debian Lenny și pe Red Hat Enterprise Linux 5) că el construiește variabila mediului PATH. Scriptul poate de asemenea să schimbe variabila PS1, să seteze HOSTNAME și să execute chiar mai multe script-uri ca **/etc/inputrc**.

Puteți folosi acest script pentru a seta aliasuri și variabile pentru fiecare utilizator pe sistem.

~/.bash_profile

Cînd acest fișier există în directorul home al utilizatorilor, atunci **bash** îl va executa. Pe Debian Linux el nu există în mod default.

RHEL5 folosește un scurt **~/.bash_profile** unde caută existența **~/.bashrc** și apoi îl execută. De asemenea el adaugă \$HOME/bin la variabila \$PATH.

```
[serena@rhel53 ~]$ cat ~/.bash_profile
```

```
# ~/.bash_profile
```

```
# Get the aliases and functions
```

```
if [ -f ~/.bashrc ]; then
```

```
    . ~/.bashrc
```

```
fi
```

```
# User specific environment and startup programs
```

```
PATH=$PATH:$HOME/bin
```

```
export PATH
```

~/.bash_login

Cînd **.bash_profile** nu există, atunci **bash** va căuta **~/.bash_login** și-l va executa.

Nici Debian nici Red Hat nu au acest fișier prin default.

~/.profile

Cînd nici **~/.bash_profile** și **~/.bash_login** nu există, atunci bash va verifica existența **~/.profile** și îl va executa. Acest fișier nu există prin default în Red Hat.

Pe Debian acest script poate executa **~/.bashrc** și de asemenea va adăuga \$HOME/bin

către variabila \$PATH.

```
serena@deb503:~$ tail -12 .profile
# if running bash
    if [ -n "$BASH_VERSION" ]; then
# include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
        . "$HOME/.bashrc"
    fi

fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi
```

~/**.bashrc**

Precum am văzut la punctele anterioare, scriptul **~/bashrc** poate fi executat de alte script-uri. Să vedem ceea ce face el în mod default.

Red Hat folosește un foarte simplu **~/bashrc**, căutînd **/etc/bashrc** și executîndu-l. El de asemeni lasă loc pentru aliasuri și funcții la comandă.

```
[serena@rhel53 ~]$ more .bashrc
# .bashrc
```

```
# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi
```

```
# User specific aliases and functions
```

Pe Debian acest script e un pic mai lung și configură \$PS1, niște variabile history și un număr de aliasuri active și inactive.

```
serena@deb503:~$ ls -l .bashrc
-rw-r--r-- 1 serena serena 3116 2008-05-12 21:02 .bashrc
```

~/**bash_logout**

Cînd ieșim din **bash**, el poate executa **~/bash_logout**. Și Debian și Red Hat utilizează această oportunitate pentru a curăța ecranul.


```
serena@deb503:~$ cat .bash_logout
# ~/.bash_logout: executed by bash(1) when login shell exits.
# when leaving the console clear the screen to increase privacy
    if [ "$SHLVL" = 1 ]; then
[ -x /usr/bin/clear_console ] && /usr/bin/clear_console -q
    fi
```

```
[serena@rhel53 ~]$ cat .bash_logout
# ~/.bash_logout
```

```
/usr/bin/clear
```

Debian

Mai jos este un tabel despre timpul cînd Debian execută oricare dintre aceste scripturi bash de startup.

Tabel 23.1 mediu de utilizator Debian

script	su	su -	ssh	gdm
~/bashrc	nu	da	da	da
~/profile	nu	da	da	da
/etc/profile	nu	da	da	da
/etc/bash.bashrc	da	nu	nu	da

RHEL5

Mai jos este un tabel despre timpul cînd Red Hat Enterprise Linux 5 execută oricare dintre aceste scripturi bash de startup.

Tabelul 23.2. mediu de utilizator Red Hat

script	su	su -	ssh	gdm
~/bashrc	da	da	da	da
~/bash_profile	nu	da	da	da
/etc/profile	nu	da	da	da
/etc/bashrc	da	da	da	da

Capitolul 24. grupuri

Conținut

24.1.	despre grupuri.	.211
24.2.	groupadd.	.211
24.3.	/etc/group.	.211
24.4.	usermod.	211
24.5.	groupmod.	.212
24.6.	groupdel.	.212
24.7.	groups.	.212
24.8.	gpasswd.	212
24.9.	vigr.	213
24.10.	practică: grupuri.	214
24.11.	soluție: grupuri.	.215

24.1. despre grupuri

Utilizatorii pot fi listați în **grupuri**. Grupurile vă permit să setați permisiuni la nivelul grupului în loc să setați permisiuni pentru fiecare utilizator individual. Fiecare distribuție Unix sau Linux are un utilitar grafic pentru managementul grupurilor. Utilizatorii novici sunt sfătuiți să utilizeze acel utilitar grafic. Utilizatorii mai experimentați pot folosi utilitare de linie de comandă pentru a face managementul utilizatorilor, dar fiți atenți: unele distribuții nu permit amestecul folosirii utilităților GUI (Graphical User Interface) și a CLI (Command Line Interface) pentru administrarea grupurilor (YaST în Novell SuSE). Administratorii avansați pot edita fișierele relevante direct cu **vi** sau **vigr**.

24.2. groupadd

Grupurile pot fi create cu comanda **groupadd**. Exemplul de mai jos arată crearea a cinci grupuri (goale).

```
root@laika:~# groupadd tennis
root@laika:~# groupadd football
root@laika:~# groupadd snooker
root@laika:~# groupadd formula1
root@laika:~# groupadd salsa
```

24.3 /etc/group

Utilizatorii pot fi un membru a mai multor grupuri. Grupul de membri este definit de fișierul **/etc/group**.

```
root@laika:~# tail -5 /etc/group
tennis:x:1006:
football:x:1007:
snooker:x:1008:
formula1:x:1009:
salsa:x:1010:
root@laika:~#
```

Primul câmp conține numele grupului. Al doilea câmp (criptat) este parola grupului (poate fi goală). Al treilea câmp este identificarea grupului sau **GID**. Al patrulea câmp este lista membrilor, aceste grupuri nu au membri.

24.4. usermod

Totalitatea de membri poate fi modificată cu comanda **useradd** sau **usermod**.

```

root@laika:~# usermod -a -G tennis inge
root@laika:~# usermod -a -G tennis katrien
root@laika:~# usermod -a -G salsa katrien
root@laika:~# usermod -a -G snooker sandra
root@laika:~# usermod -a -G formula1 annelies
root@laika:~# tail -5 /etc/group
tennis:x:1006:inge,katrien
football:x:1007:
snooker:x:1008:sandra
formula1:x:1009:annelies
salsa:x:1010:katrien
root@laika:~#

```

Fiți atenți când utilizați **usermod** pentru a adăuga utilizatori la grupuri. Prin default, comanda **usermod** va **șterge** utilizatorul din fiecare grup din care el este membru dacă grupul nu este listat în comandă! Folosind întrerupătorul **-a** (*append*, a adăuga la) previne acest comportament.

24.5. groupmod

Puteți schimba numele grupului cu comanda **groupmod**.

```

root@laika:~# groupmod -n darts snooker
root@laika:~# tail -5 /etc/group
tennis:x:1006:inge,katrien
football:x:1007:
formula1:x:1009:annelies
salsa:x:1010:katrien
darts:x:1008:sandra

```

24.6. groupdel

Puteți șterge permanent un grup cu comanda **groupdel**.

```

root@laika:~# groupdel tennis
root@laika:~#

```

24.7. groups

Un utilizator poate tasta comanda **groups** pentru a vedea o listă a grupurilor la care aparține.

```

[harry@RHEL4b ~]$ groups
harry sports
[harry@RHEL4b ~]$

```

24.8. gpasswd

Puteți delega controlul totalității grupului altui utilizator cu comanda **gpasswd**. În exemplul de mai jos delegăm permisiunile pentru a adăuga și a șterge membri de grup lui serena pentru grupul sports. Apoi devenim **su** la serena și îl adăugăm pe harry grupului sports.

```
[root@RHEL4b ~]# gpasswd -A serena sports
[root@RHEL4b ~]# su - serena
[serena@RHEL4b ~]$ id harry
uid=516(harry) gid=520(harry) groups=520(harry)
[serena@RHEL4b ~]$ gpasswd -a harry sports
Adding user harry to group sports
[serena@RHEL4b ~]$ id harry
uid=516(harry) gid=520(harry) groups=520(harry),522(sports)
[serena@RHEL4b ~]$ tail -1 /etc/group
sports:x:522:serena,venus,harry
[serena@RHEL4b ~]$
```

Administratorii de grupuri nu trebuie să fie un membru al grupului. Ei pot să se șteargă pe ei înșiși dintr-un grup, dar asta nu influențează abilitatea lor de a adăuga sau a șterge membri.

```
[serena@RHEL4b ~]$ gpasswd -d serena sports
Removing user serena from group sports
[serena@RHEL4b ~]$ exit
```

Informația despre administratorii de grup este păstrată în fișierul **/etc/gshadow**.

```
[root@RHEL4b ~]# tail -1 /etc/gshadow
sports:!:serena:venus,harry
[root@RHEL4b ~]#
```

Pentru a șterge toți administratorii de grup dintr-un grup, folosiți comanda **gpasswd** pentru a seta o listă goală de administratori.

```
[root@RHEL4b ~]# gpasswd -A "" sports
```

24.9. **vigr**

Similar cu **vipw**, comanda **vigr** poate fi utilizată pentru a edita manual fișierul **/etc/group**, pentru că ea va face o blocare proprie a fișierului. Doar administratorii seniori experimentați ar trebui să folosească **vi** sau **vigr** pentru a face managementul grupurilor.

24.10. practică: grupuri

1. Creați grupurile tennis, football și sports.
2. Cu o singură comandă, faceți pe venus un membru al tennis și sports.
3. Redenumiți grupul football în foot.
4. Folosiți vi pentru a adăuga serena la grupul tennis.
5. Folosiți comanda id pentru a verifica dacă serena este un membru a grupului tennis.
6. Faceți pe cineva responsabil de managementul totalității grupului foot și sports. Testați dacă funcționează.

24.11. soluție: grupuri

1. Creați grupurile tennis, football și sports.

```
groupadd tennis ; groupadd football ; groupadd sports
```

2. Cu o singură comandă, faceți pe venus un membru al tennis și sports.

```
usermod -a -G tennis,sports venus
```

3. Redenumiți grupul football în foot.

```
groupmod -n foot football
```

4. Folosiți vi pentru a adăuga serena la grupul tennis.

```
vi /etc/group
```

5. Folosiți comanda id pentru a verifica dacă serena este un membru a grupului tennis.

```
id (și după logoff logon serena ar trebui să fie membru)
```

6. Faceți pe cineva responsabil de managementul totalității grupului foot și sports. Testați dacă funcționează.

```
gpasswd -A (pentru a face un administrator)
```

```
gpasswd -a (pentru a adăuga un membru)
```

Partea VIII. securitatea fișierului

Capitolul 25. permisiuni fișier standard

Conținut

25.1. proprietarul fișierului.	.218
25.2. listă a fișierelor speciale.	.219
25.3. permisiuni.	219
25.4. practică: permisiuni fișier standard.	223
25.5. soluție: permisiuni fișier standard.	.224

25.1. proprietarul fișierului

proprietar fișier și proprietar grup

Utilizatorilor și grupurilor unui sistem li se pot face managementul local în `/etc/passwd` și `/etc/group`, sau ei pot fi într-un domeniu NIS, LDAP, sau Samba. Acești utilizatori și grupuri pot **deține** fișiere. În realitate, fiecare fișier are un **proprietar utilizator** și un **proprietar grup**, cum poate fi văzut în următoarea captură de ecran.

```
paul@RHELv4u4:~/test$ ls -l
total 24
-rw-rw-r-- 1 paul paul 17 Feb 7 11:53 file1
-rw-rw-r-- 1 paul paul 106 Feb 5 17:04 file2
-rw-rw-r-- 1 paul proj 984 Feb 5 15:38 data.odt
-rw-r--r-- 1 root root 0 Feb 7 16:07 stuff.txt
paul@RHELv4u4:~/test$
```

Utilizatorul paul deține trei fișiere, două dintre acestea sînt la fel deținute de către grupul paul; data.odt este deținut de grupul proj. Utilizatorul root deține fișierul stuff.txt, la fel ca grupul root.

chgrp

Puteți schimba deținătorul grup a unui fișier folosind comanda **chgrp**.

```
root@laika:/home/paul# touch FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root root 0 2008-08-06 14:11 FileForPaul
root@laika:/home/paul# chgrp paul FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root paul 0 2008-08-06 14:11 FileForPaul
```

chown

Utilizatorul proprietar a unui fișier poate fi schimbat cu comanda **chown**.

```
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root paul 0 2008-08-06 14:11 FileForPaul
root@laika:/home/paul# chown paul FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 paul paul 0 2008-08-06 14:11 FileForPaul
```

Puteți de asemenea folosi **chown** pentru a schimba și proprietarul utilizator și proprietarul grup.

```
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 paul paul 0 2008-08-06 14:11 FileForPaul
root@laika:/home/paul# chown root:project42 FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root project42 0 2008-08-06 14:11 FileForPaul
```

25.2. listă a fișierelor speciale

Cînd utilizați `ls -l`, pentru fiecare fișier puteți vedea 10 caractere înaintea proprietarului utilizator și a grupului. Primul caracter ne spune tipul de fișier. Fișierele regulate primesc un `-`, directoarele un `d`, legăturile simbolice sînt afișate cu un `l`, conductele primesc un `p`, dispozitivele caracter primesc un `c`, dispozitivele bloc un `b`, și soclurile un `s`.

Tabelul 25.1. fișiere speciale Unix

primul caracter	tipul de fișier
-	fișier normal
d	director
l	legătură simbolică
p	conductă
b	dispozitiv bloc
c	dispozitiv caracter
s	soclu

Mai jos este o captură de ecran a unui dispozitiv caracter (consola) și un dispozitiv bloc (hard-disk-ul).

```
paul@debian6lt~$ ls -ld /dev/console /dev/sda
crw----- 1 root root 5, 1 Mar 15 12:45 /dev/console
brw-rw---- 1 root disk 8, 0 Mar 15 12:45 /dev/sda
```

Și aici puteți vedea un director, un fișier regulat și o legătură simbolică.

```
paul@debian6lt~$ ls -ld /etc /etc/hosts /etc/motd
drwxr-xr-x 128 root root 12288 Mar 15 18:34 /etc
-rw-r--r-- 1 root root 372 Dec 10 17:36 /etc/hosts
lrwxrwxrwx 1 root root 13 Dec 5 10:36 /etc/motd -> /var/run/motd
```

25.3. permisiuni

rwX

Cele nouă litere care urmează tipului de fișier denotă permisiunile în trei tripleți. O permisiune poate fi `r` pentru acces de citire, `w` pentru acces de scriere, și `x` pentru executare. Trebuie să aveți permisiunea `r` pentru a lista (`ls`) conținutul unui director. Trebuie să aveți permisiunea `x` pentru a intra (`cd`) într-un director. Trebuie să aveți permisiunea `w` pentru a crea fișiere sau șterge fișiere dintr-un director.

Tabelul 25.2. permisiuni de fișiere standard Unix

permisiune	asupra unui fișier	asupra unui director
r (citește)	citește conținutul fișierului (<code>cat</code>)	citește conținutul directorului (<code>ls</code>)
w (scrie)	schimbă conținutul fișierului (<code>vi</code>)	crează fișiere (<code>touch</code>)
x (execută)	execută fișierul	intră în director (<code>cd</code>)

trei seturi de rwX

Știm deja că ieșirea `ls -l` începe cu zece caractere pentru fiecare fișier. Această captură de ecran arată un fișier regulat (pentru că primul caracter este un `-`).

```
paul@RHELv4u4:~/test$ ls -l proc42.bash
-rwxr-xr-- 1 paul proj 984 Feb 6 12:01 proc42.bash
```

Mai jos este un tabel care descrie funcția tuturor celor zece caractere.

Tabelul 25.3. poziția permisiunilor de fișier Unix

poziție	caractere	funcție
1	-	acesta e un fișier regular
2-4	rwx	permisiuni pentru proprietarul utilizator (user)
5-7	r-x	permisiuni pentru proprietarul grup (group)
8-10	r--	permisiuni pentru ceilalți (others)

Când sînteți **proprietarul utilizator** a unui fișier, atunci vi se aplică **permisiunile de utilizator (user)**. Restul permisiunilor nu au nici o influență asupra accesului vostru la fișier.

Când aparțineți **grupului** care este **proprietarul grup** a unui fișier, atunci vi se aplică **permisiunile proprietarului grup (group)**. Restul permisiunilor nu au nici o influență asupra accesului vostru la fișier.

Cînd nu sînteți **proprietarul utilizator** a unui fișier și nu aparțineți de **proprietarul grup**, atunci **permisiunile pentru ceilalți (others)** vi se aplică. Restul permisiunilor nu au nici o influență asupra accesului vostru la fișier.

exemple de permisiuni

Unele exemple de combinații asupra fișierelor și directoarelor sînt vizibile în această captură de ecran. Numele fișierului explică permisiunile.

```
paul@laika:~/perms$ ls -lh
total 12K
drwxr-xr-x 2 paul paul 4.0K 2007-02-07 22:26 ToțiIntrăUtilizatorCreazăȘterge
-rwxrwxrwx 1 paul paul  0 2007-02-07 22:21 ToțiControlDeplin.txt
-r--r----- 1 paul paul  0 2007-02-07 22:21 DoarProprietariCitesc.txt
-rwxrwx--- 1 paul paul  0 2007-02-07 22:21 ProprietarTot_CeilalțiNimic.txt
dr-xr-x--- 2 paul paul 4.0K 2007-02-07 22:25 IntrăUtilizatoriȘiGrup
dr-x----- 2 paul paul 4.0K 2007-02-07 22:25 DoarUtilizatorIntră
paul@laika:~/perms$
```

Pentru a face un sumar, primul triplet **rwx** reprezintă permisiunile pentru **proprietarul utilizator**. Al doilea triplet corespunde **proprietarului grup**; el specifică permisiunile pentru toți membrii ai acelui grup. Al treilea triplet definește permisiunile pentru toți **ceilalți** utilizatori care nu sînt proprietarul utilizator și nu sînt un membru al proprietarului grup.

setare permisiuni (chmod)

Permisiunile pot fi schimbate cu **chmod**. Primul exemplu dă proprietarului utilizator permisiuni de executare.

```
paul@laika:~/perms$ ls -l permissions.txt
-rw-r--r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod u+x permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxr--r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Acest exemplu scoate permisiunea de citire a proprietarilor grup.

```
paul@laika:~/perms$ chmod g-r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx---r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Acest exemplu scoate permisiunea de citire celorlalți.

```
paul@laika:~/perms$ chmod o-r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx----- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Acest exemplu dă tuturor permisiunea de scriere.

```
paul@laika:~/perms$ chmod a+w permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx-w--w- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Nu e necesar nici să scrieți a.

```
paul@laika:~/perms$ chmod +x permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx-wx-wx 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Puteți de asemeni seta permisiuni explicite.

```
paul@laika:~/perms$ chmod u=rw permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw--wx-wx 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Sînteți liber să faceți orice tip de combinare.

```
paul@laika:~/perms$ chmod u=rw,g=rw,o=r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw-rw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Chiar și combinaările alunecoase sînt acceptabile de chmod.

```
paul@laika:~/perms$ chmod u=rwx,ug+rw,o=r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxrw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

setarea permisiunilor octale

Cei mai mulți administratori Unix vor utiliza sistemul octal **de modă veche** pentru a vorbi despre el și a seta permisiuni. Priviți la tripleți din punct de vedere al bitului, echivalînd ecuația $r = 4$, $w = 2$, și $x = 1$.

Tabelul 25.4. Permisuni octale

binar	octal	permisiune
000	0	- - -
001	1	- - x
010	2	- w -
011	3	- w x
100	4	r - -
101	5	r - x
110	6	r w -
111	7	r w x

Asta face **777** egal cu **rw-rw-rw** și prin aceeași logică, **654** înseamnă **rw-r-xr--**. Comanda **chmod** va accepta aceste numere.

```
paul@laika:~/perms$ chmod 777 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxrwxrwx 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod 664 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw-rw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod 750 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxr-x--- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

umask

Cînd creăm un fișier sau director, sînt aplicate un set de permisiuni default. Aceste permisiuni default sînt determinate de **umask**. **umask** specifică permisiunile pe care nu le vreți setate prin default. Puteți afișa **umask** cu comanda **umask**.

```
[Harry@RHEL4b ~]$ umask
0002
[Harry@RHEL4b ~]$ touch test
[Harry@RHEL4b ~]$ ls -l test
-rw-rw-r-- 1 Harry Harry 0 Jul 24 06:03 test
[Harry@RHEL4b ~]$
```

După cum puteți vedea de asemeni, fișierul este neexecutabil prin default. Asta este o caracteristică de securitate generală în sistemele de operare tip Unix; fișierele nou create nu sînt niciodată executabile prin default. Trebuie în mod explicit să scrieți comanda **chmod +x** pentru a face fișierul executabil. Asta de asemeni înseamnă că 1 bit în **umask** nu are nici un înțeles – un **umask** de 0022 este la fel cu 0033.

mkdir -m

Cînd creați directoare cu **mkdir** puteți folosi opțiunea **-m** pentru a seta **mode**. Această captură de ecran explică.

```
paul@debian5~$ mkdir -m 700 MyDir
paul@debian5~$ mkdir -m 777 Public
paul@debian5~$ ls -dl MyDir/ Public/
drwx----- 2 paul paul 4096 2011-10-16 19:16 MyDir/
drwxrwxrwx 2 paul paul 4096 2011-10-16 19:16 Public/
```

25.4 practică: permisiuni fișier standard

1. Ca utilizator normal, creați un director ~/permissions. Creați un fișier deținut de voi acolo.
2. Copiați un fișier deținut de root din /etc/ în directorul permissions, cine deține acest fișier acum?
3. Ca root, creați un fișier în directorul ~/permissions a utilizatorilor.
4. Ca utilizator normal, priviți cine deține acest fișier creat de root.
5. Schimbați proprietarul tuturor fișierelor din ~/permissions pentru dvs.
6. Asigurați-vă că aveți toate drepturile asupra acestor fișiere, și că ceilalți pot doar citi.
7. Cu chmod, este 770 la fel cu rwxrwx---?
8. Cu chmod, este 664 la fel cu r-xr-xr--?
9. Cu chmod, este 400 la fel cu r-----?
10. Cu chmod, este 734 la fel cu rwxr-xr--?
- 11a. Afișați umask în formă octală și în formă simbolică.
- 11b. Setati umask la 077, dar folosiți formatul simbolic pentru a-l seta. Verificați dacă asta funcționează.
12. Creați un fișier ca root, dați celorlalți doar drepturi de citire. Poate un utilizator obișnuit să citească acest fișier? Testați scriind în acest fișier cu vi.
- 13a. Creați un fișier ca utilizator obișnuit, dați-i drepturi de citire doar celorlalți. Un alt utilizator normal poate să citească acest fișier? Testați scriind în acest fișier cu vi.
- 13b. Poate root să citească acest fișier? Poate root să scrie în acest fișier cu vi?
14. Creați un director care aparține unui grup, unde fiecare membru al acelui grup poate citi și scrie în fișiere, și să creeze fișiere. Asigurați-vă că oamenii aceia pot doar să șteargă propriile lor fișiere.

25.5. soluție: permisiuni fișier standard

1. Ca utilizator normal, creați un director ~/permissions. Creați un fișier deținut de voi acolo.

```
mkdir ~/permissions ; touch ~/permissions/myfile.txt
```

2. Copiați un fișier deținut de root din /etc/ în directorul permissions, cine deține acest fișier acum?

```
cp /etc/hosts ~/permissions/
```

Copia este deținută de dvs.

3. Ca root, creați un fișier în directorul ~/permissions a utilizatorilor.

(deveniți root) # touch /home/username/permissions/rootfile

4. Ca utilizator normal, priviți cine deține acest fișier creat de root.

```
ls -l ~/permissions
```

Fișierul creat de root este deținut de către root.

5. Schimbați proprietarul tuturor fișierelor din ~/permissions pentru dvs.

```
chown user ~/permissions/*
```

Nu puteți deveni proprietarul fișierului care aparține lui root.

6. Asigurați-vă că aveți toate drepturile asupra acestor fișiere, și că ceilalți pot doar citi.

```
chmod 644 (asupra fișierelor)
```

```
chmod 755 (asupra directoarelor)
```

7. Cu chmod, este 770 la fel cu rwxrwx---?

Da.

8. Cu chmod, este 664 la fel cu r-xr-xr--?

Nu.

9. Cu chmod, este 400 la fel cu r-----?

Da.

10. Cu chmod, este 734 la fel cu rwxr-xr--?

Nu.

11a. Afișați umask în formă octală și în formă simbolică.

```
umask ; umask -S
```


11b. Setati umask la 077, dar folosiți formatul simbolic pentru a-l seta. Verificați dacă asta funcționează.

```
umask -S u=rwx,go=
```

12. Creați un fișier ca root, dați celorlalți doar drepturi de citire. Poate un utilizator obișnuit să citească acest fișier? Testați scriind în acest fișier cu vi.

(deveniți root)

```
# echo hello > /home/username/root.txt
```

```
# chmod 744 /home/username/root.txt
```

(deveniți utilizator)

```
vi ~/root.txt
```

13a. Creați un fișier ca utilizator obișnuit, dați-i drepturi de citire doar celorlalți. Un alt utilizator normal poate să citească acest fișier? Testați scriind în acest fișier cu vi.

```
echo hello > file ; chmod 744 file
```

Da, ceilalți pot citi acest fișier.

13b. Poate root să citească acest fișier? Poate root să scrie în acest fișier cu vi?

Da, root poate citi și scrie în acest fișier. Permisunile nu se aplică lui root.

14. Creați un director care aparține unui grup, unde fiecare membru al acelui grup poate citi și scrie în fișiere, și să creeze fișiere. Asigurați-vă că oamenii aceia pot doar să șteargă propriile lor fișiere.

```
mkdir /home/project42 ; groupadd project42
```

```
chgrp project42 /home/project42 ; chmod 775 /home/project42
```

Nu puteți încă să faceți ultima parte a acestui exercițiu...

Capitolul 26. permisiuni fișier avansate

Conținut

26.1. sticky bit asupra unui director.	.227
26.2. setgid bit asupra unui director.	.227
26.3. setgid și setuid asupra fișierelor regulate.	.228
26.4. practică: biții sticky, setuid și setgid.	.229
26.5. soluție: biții sticky, setuid și setgid.	.230

26.1. sticky bit asupra unui director

Puteți seta **sticky bit** asupra unui director pentru a preveni utilizatorii să nu șteargă fișiere pe care ei nu le dețin ca proprietar utilizator. Sticky bit este afișat în aceeași locație ca permisiunea x pentru ceilalți. Sticky bit este reprezentat cu un **t** (însemnând că x este de asemeni acolo) sau un **T** (unde nu există x pentru ceilalți).

```
root@RHELv4u4:~# mkdir /project55
root@RHELv4u4:~# ls -ld /project55
drwxr-xr-x 2 root root 4096 Feb 7 17:38 /project55
root@RHELv4u4:~# chmod +t /project55/
root@RHELv4u4:~# ls -ld /project55
drwxr-xr-t 2 root root 4096 Feb 7 17:38 /project55
root@RHELv4u4:~#
```

Sticky bit poate fi de asemeni setat cu permisiuni octale, este binarul 1 în primul din cei patru tripleți.

```
root@RHELv4u4:~# chmod 1775 /project55/
root@RHELv4u4:~# ls -ld /project55
drwxrwxr-t 2 root root 4096 Feb 7 17:38 /project55
root@RHELv4u4:~#
```

Veți găsi în mod tipic **sticky bit** în directorul **/tmp**.

```
root@barry:~# ls -ld /tmp
drwxrwxrwt 6 root root 4096 2009-06-04 19:02 /tmp
```

26.2. setgid asupra unui director

setgid poate fi utilizat pe directoare pentru a fi siguri că toate fișierele din director sînt deținute de grupul proprietar al directorului. Bitul **setgid** este afișat în aceeași locație ca permisiunea x pentru proprietarul grup. Bitul **setgid** este reprezentat de un **s** (însemnând că x este de asemeni acolo) sau de un **S** (unde nu există x pentru proprietarul grup). Așa cum acest exemplu arată, chiar dacă **root** nu aparține grupului proj55, fișierele create de root în /project55 vor aparține lui proj55 de vreme ce **setgid** este setat.

```
root@RHELv4u4:~# groupadd proj55
root@RHELv4u4:~# chown root:proj55 /project55/
root@RHELv4u4:~# chmod 2775 /project55/
root@RHELv4u4:~# touch /project55/fromroot.txt
root@RHELv4u4:~# ls -ld /project55/
drwxrwsr-x 2 root proj55 4096 Feb 7 17:45 /project55/
root@RHELv4u4:~# ls -l /project55/
total 4
-rw-r--r-- 1 root proj55 0 Feb 7 17:45 fromroot.txt
root@RHELv4u4:~#
```

Puteți folosi comanda **find** pentru a găsi toate directoarele **setgid**.

```
paul@laika:~$ find / -type d -perm -2000 2> /dev/null
/var/log/mysql
/var/log/news
/var/local
. . .
```

26.3. setgid și setuid asupra fișierelor regulate

Aceste două permisiuni fac ca un fișier executabil să fie executat cu permisiunile **proprietarului fișierului** în loc de **proprietarul executabil**. Asta înseamnă că dacă orice utilizator execută un program care aparține **utilizatorului root**, și bitul **setuid** este setat pe acel program, atunci programul se execută ca **root**. Asta poate fi periculos, dar uneori e bine pentru securitate.

Luați exemplul parolelor; ele sînt stocate în **/etc/shadow** care poate fi citit numai de către **root**. (Utilizatorul **root** nu are nevoie de permisiuni oricum.)

```
root@RHELv4u4:~# ls -l /etc/shadow
-r----- 1 root root 1260 Jan 21 07:49 /etc/shadow
```

Schimbarea parolei cere o actualizare a acestui fișier, altfel cum poate un utilizator non-root să facă acest lucru? Să ne uităm la permisiuni în fișierul **/usr/bin/passwd**.

```
root@RHELv4u4:~# ls -l /usr/bin/passwd
-r-s--x--x 1 root root 21200 Jun 17 2005 /usr/bin/passwd
```

Cînd se execută programul **passwd**, îl executați cu acreditările **root**.

Puteți utiliza comanda **find** pentru a găsi toate programele **setuid**.

```
paul@laika:~$ find /usr/bin -type f -perm -04000
/usr/bin/arping
/usr/bin/kgrantpty
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/fping6
/usr/bin/passwd
/usr/bin/gpasswd
...
```

În cele mai multe cazuri, setînd bitul **setuid** pe executabile este suficient. Setînd bitul **setgid** va rezulta ca aceste programe să se execute cu acreditările proprietarului lor grup.

26.4. practică: biți sticky, setuid și setgid

1a. Setați un director, deținut de grupul sports.

1b. Membrii grupului sports ar trebui să poată să creeze fișiere în acest director.

1c. Toate fișierele create în acest director ar trebui să fie deținute de grupul sports.

1d. Utilizatorii ar trebui să fie capabili să șteargă doar fișierele deținute de ei înșiși.

1e. Testați dacă asta funcționează!

2. Verificați permisiunile pe **/usr/bin/passwd**. Ștergeți **setuid**, apoi încercați să vă schimbați parola ca utilizator obișnuit. Resetați permisiunile înapoi și încercați din nou.

3. Dacă timpul permite (sau dacă așteptați ca alți studenți să termine această practică), citiți despre atributele fișierelor în paginile de manual ale **chattr** și **lsattr**. Încercați să setați atributul i unui fișier și testați dacă funcționează.

26.5. soluție: biți sticky, setuid și setgid

1a. Setați un director, deținut de grupul sports.

```
groupadd sports
```

```
mkdir /home/sports
```

```
chown root:sports /home/sports
```

1b. Membrii grupului sports ar trebui să poată să creeze fișiere în acest director.

```
chmod 770 /home/sports
```

1c. Toate fișierele create în acest director ar trebui să fie deținute de grupul sports.

```
chmod 2770 /home/sports
```

1d. Utilizatorii ar trebui să fie capabili să ștergă doar fișierele deținute de ei înșiși.

```
chmod +t /home/sports
```

1e. Testați dacă asta funcționează!

Intrați în sistem cu utilizatori diferiți (membri grup și ceilalți și root), creați fișiere și vedeți permisiunile. Încercați să schimbați și să ștergeți fișiere...

2. Verificați permisiunile pe **/usr/bin/passwd**. Ștergeți **setuid**, apoi încercați să vă schimbați parola ca utilizator obișnuit. Resetați permisiunile înapoi și încercați din nou.

```
root@deb503:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
root@deb503:~# chmod 755 /usr/bin/passwd
root@deb503:~# ls -l /usr/bin/passwd
-rwxr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
```

Un user normal nu poate să schimbe parola acum.

```
root@deb503:~# chmod 4755 /usr/bin/passwd
root@deb503:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
```

3. Dacă timpul permite (sau dacă așteptați ca alți studenți să termine această practică), citiți despre atributele fișierelor în paginile de manual ale **chattr** și **lsattr**. Încercați să setați atributul i unui fișier și testați dacă funcționează.

```
paul@laika:~$ sudo su -  
[sudo] password for paul:  
root@laika:~# mkdir attr  
root@laika:~# cd attr/  
root@laika:~/attr# touch file42  
root@laika:~/attr# lsattr  
----- ./file42  
root@laika:~/attr# chattr +i file42  
root@laika:~/attr# lsattr  
----i----- ./file42  
root@laika:~/attr# rm -rf file42  
rm: cannot remove `file42': Operation not permitted  
root@laika:~/attr# chattr -i file42  
root@laika:~/attr# rm -rf file42  
root@laika:~/attr#
```

Capitolul 27. liste control acces

Conținut

27.1. acl în /etc/fstab.	.233
27.2. getfacl.	.233
27.3. setfacl.	.233
27.4. ștergeți o intrare acl.	234
27.5. ștergeți complet acl.	234
27.6. acl mask.	234
27.7. eiciel.	234

Permisunile standard Unix pot să nu fie îndeajuns pentru unele organizații. Acest capitol introduce **liste control acces** sau **acl** pentru a proteja și mai mult fișiere și directoare.

27.1. acl în /etc/fstab

Sistemele de fișier care suportă **liste control acces** sau **acl**, trebuie să fie montate cu opțiunea **acl** listată în **/etc/fstab**. În exemplul de mai jos, puteți vedea că sistemul de fișier root are suport **acl**, în timp ce /home/data nu.

```
root@laika:~# tail -4 /etc/fstab
/dev/sda1      /          ext3      acl,relatime    0 1
/dev/sdb2      /home/data auto      noacl,default 0 0
pasha:/home/r  /home/pasha nfs       defaults        0 0
wolf:/srv/data /home/wolf  nfs       defaults        0 0
```

27.2. getfacl

Citirea **acl** poate fi făcută cu **/usr/bin/getfacl**. Această captură de ecran arată cum să citim **acl** al fișierului **file33** cu **getfacl**.

```
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
group::r--
mask::rwx
other::r--
```

27.3. setfacl

Scrierea sau schimbările **acl** pot fi făcute cu **/usr/bin/setfacl**. Aceste capturi de ecran arată cum să schimbăm **acl** al fișierului **file33** cu **setfacl**.

Mai întâi adăugăm utilizatorul **sandra** cu permisiunile octale **7** la **acl**.

```
paul@laika:~/test$ setfacl -m u:sandra:7 file33
```

Apoi adăugăm grupul **tennis** cu permisiunile octale **6** la **acl-ul** aceluiași fișier.

```
paul@laika:~/test$ setfacl -m g:tennis:6 file33
```

Rezultatul este vizibil cu **getfacl**.

```
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
user:sandra:rwx
group::r--
group:tennis:rw-
mask::rwx
other::r--
```

27.4. ștergeți o intrare acl

Opțiunea **-x** a comenzii **setfacl** va șterge o intrare **acl** din fișierul țintă.

```
paul@laika:~/test$ setfacl -m u:sandra:7 file33
paul@laika:~/test$ getfacl file33 | grep sandra
user:sandra:rw-
paul@laika:~/test$ setfacl -x sandra file33
paul@laika:~/test$ getfacl file33 | grep sandra
```

Notați că omiterea **u** sau **g** când definim **acl** pentru un cont va deveni default pentru un cont de utilizator.

27.5. ștergeți complet acl

Opțiunea **-b** a comenzii **setfacl** va șterge **acl** din fișierul țintă.

```
paul@laika:~/test$ setfacl -b file33
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
group::r--
other::r--
```

27.6. acl mask

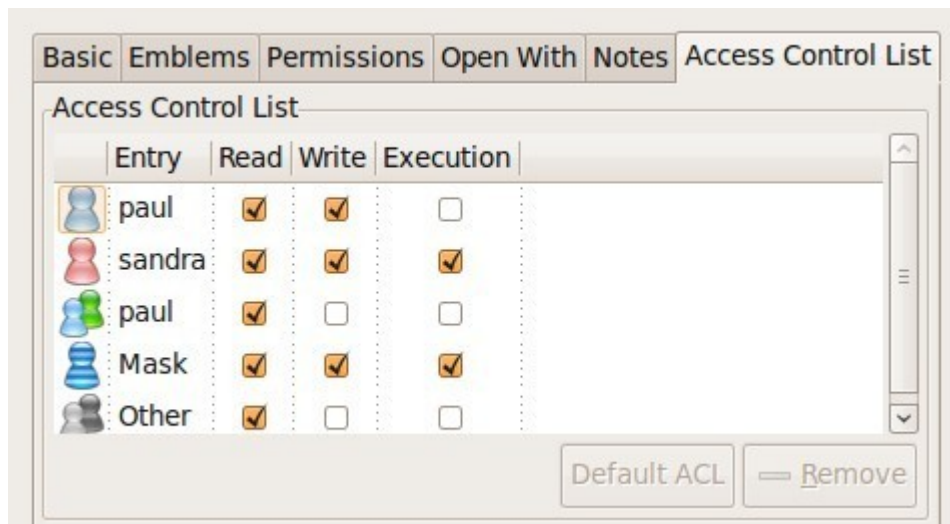
acl mask definește permisiunile efective maxime pentru orice intrare în **acl**. Acest **mask** este calculat de fiecare dată când executați comenzile **setfacl** sau **chmod**.

Puteți preveni calcularea utilizând întrerupătorul **--no-mask**.

```
paul@laika:~/test$ setfacl --no-mask -m u:sandra:7 file33
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
user:sandra:rw-   #effective:rw-
group::r--
mask::rw-
other::r--
```

27.7. eiciel

Utilizatorii desktop ar putea dori să folosească **eiciel** pentru a organiza **acl** cu un utilitar grafic.



Veți avea nevoie să instalați **eiciel** și **nautilus-actions** pentru a avea un extra tab în **nautilus** pentru a organiza **acl-uri**.

```
paul@laika:~$ sudo aptitude install eiciel nautilus-actions
```

Capitolul 28. legături fișier

Conținut

28.1. inod-uri.	237
28.2. despre directoare.	238
28.3. legături depline.	238
28.4. legături simbolice.	239
28.5. ștergere legături.	240
28.6. practică: legături.	241
28.7. soluție: legături.	242

Un computer obișnuit care folosește Linux are un sistem de fișier cu multe legături depline (**hard links**) și legături simbolice (**symbolic links**).

Pentru a înțelege legăturile într-un sistem de fișier, trebuie să înțelegeți ce anume este un **inod**.

28.1. inod-uri

conținuturi inod

Un **inod** este o structură de date care conține metadate despre un fișier. Când sistemul de fișiere stochează un nou fișier pe hard-disk, el stochează nu numai conținutul (datele) unui fișier, dar și extra proprietăți ca numele unui fișier, data creării, permisiunile lui, proprietarul fișierului, și mai mult. Toată această informație (cu excepția numelui fișierului și conținutul fișierului) este stocată în **inod-ul** fișierului.

Comanda **ls -l** va afișa câte ceva din conținutul inode, cum se poate vedea în această captură de ecran.

```
root@rhel53 ~# ls -ld /home/project42/
drwxr-xr-x 4 root pro42 4.0K Mar 27 14:29 /home/project42/
```

tabel inod

Tabelul inode conține toate **inod-urile** și este creat când creați sistemul de fișier (cu **mkfs**). Puteți folosi comanda **df -i** pentru a vedea cât de multe **inod-uri** sînt folosite și câte libere pe sisteme fișier montate.

```
root@rhel53 ~# df -i
Filesystem          Inodes      IUsed      IFree   IUse% Mounted on
/dev/mapper/VolGroup00-LogVol00
4947968      115326    4832642      3% /
/dev/hda1           26104         45    26059      1% /boot
tmpfs              64417          1    64416      1% /dev/shm
/dev/sda1          262144       2207    259937      1% /home/project42
/dev/sdb1           74400       5519    68881      8% /home/project33
/dev/sdb5              0           0         0      - /home/sales
/dev/sdb6          100744         11    100733      1% /home/research
```

În captura de ecran de deasupra cu **df -i** puteți vedea utilizarea **inod** pentru mai multe **sisteme de fișier** montate. Nu vedeți numere pentru **/dev/sdb5** pentru că el este un sistem de fișier **fat**.

număr inod

Fiecare **inod** are un număr unic (numărul inode). Puteți vedea numerele **inod** cu comanda **ls -li**.

```
paul@RHELv4u4:~/test$ touch file1
paul@RHELv4u4:~/test$ touch file2
paul@RHELv4u4:~/test$ touch file3
paul@RHELv4u4:~/test$ ls -li
total 12
817266 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file1
817267 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file3
paul@RHELv4u4:~/test$
```

Aceste trei fişiere au fost create unul după altul şi au primit 3 **inod-uri** diferite (prima coloană). Toată informaţia pe care o vedeţi cu comanda **ls** rezidă în **inod**, cu excepţia numelui de fişier (care este conţinut în director).

inod şi conţinutul fişierului

Să punem nişte date în unul din aceste fişiere.

```
paul@RHELv4u4:~/test$ ls -li
total 16
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 1 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
paul@RHELv4u4:~/test$ cat file2
It is winter now and it is very cold.
We do not like the cold, we prefer hot summer nights.
paul@RHELv4u4:~/test$
```

Datele care sînt afişate de către comanda **cat** nu este în **inod**, ci altundeva pe disk. **inod-urile** conţin indicatoare spre acele date.

28.2. despre directoare

un director este un tabel

Un **director** este un tip special de fişier care conţine un tabel care mapează numele de fişiere în inod-uri. Listarea directorului curent cu **ls -ali** va afişa conţinutul fişierului director.

```
paul@RHELv4u4:~/test$ ls -ali
total 32
817262 drwxrwxr-x 2 paul paul 4096 Feb 5 15:42 .
800768 drwx----- 16 paul paul 4096 Feb 5 15:42 ..
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 1 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
```

. şi ..

Puteţi vedea 5 nume, şi maparea la cele 5 inod-uri ale lor. **Punctul .** este o mapare de sine stătătoare, şi **punct punct ..** este maparea către directorul părinte. Celelalte 3 nume sînt mapări către inoduri diferite.

28.3. legături depline (hard links)

creare legături depline

Cînd creăm o **legătură deplină** la un fişier cu **ln**, o intrare extra este adăugată în director. Un nou nume de fişier este mapat la un inod existent.

```
paul@RHELv4u4:~/test$ ln file2 hardlink_to_file2
paul@RHELv4u4:~/test$ ls -li
total 24
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 2 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
817270 -rw-rw-r-- 2 paul paul 92 Feb 5 15:42 hardlink_to_file2
paul@RHELv4u4:~/test$
```

Ambele fișiere au același inod, astfel ele vor avea întotdeauna aceleași permisiuni și același proprietar. Ambele fișiere vor avea același conținut. În realitate, ambele fișiere sînt egale acum, însemnînd că puteți șterge în siguranță fișierul original, fișierul cu legătură deplină va rămîne. inode conține un numărător, numărînd numărul legăturilor depline în el însuși. Cînd numărătorul scade la zero, atunci inod-ul este golit.

găsire legături depline

Puteți folosi comanda **find** pentru a căuta fișiere cu un anumit inod. Captura de ecran de mai jos arată cum să căutăm toate numele de fișiere care țintesc la **inod 817270**. Țineți minte că un număr **inod** este unic partiției lui.

```
paul@RHELv4u4:~/test$ find / -inum 817270 2> /dev/null
/home/paul/test/file2
/home/paul/test/hardlink_to_file2
```

28.4. legături simbolice (soft links)

Legăturile simbolice (uneori numite **soft links**) nu fac legătură spre inoduri, ci crează un nume la maparea numelui. Legăturile simbolice sînt create cu **ln -s**. Așa cum puteți vedea mai jos, **legăturile simbolice** primesc un inod doar al lor.

```
paul@RHELv4u4:~/test$ ln -s file2 symlink_to_file2
paul@RHELv4u4:~/test$ ls -li
total 32
817273 -rw-rw-r-- 1 paul paul 13 Feb 5 17:06 file1
817270 -rw-rw-r-- 2 paul paul 106 Feb 5 17:04 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
817270 -rw-rw-r-- 2 paul paul 106 Feb 5 17:04 hardlink_to_file2
817267 lrwxrwxrwx 1 paul paul 5 Feb 5 16:55 symlink_to_file2 -> file2
paul@RHELv4u4:~/test$
```

Permiuniunile asupra unei legături simbolice nu au nici un înțeles, de vreme ce permiuniunile țintei se aplică. Legăturile depline sînt limitate la propriile lor partiții (pentru că țintesc la un inode), legăturile simbolice pot face legături oriunde (alte sisteme de fișiere, chiar și în rețea).

28.5. ștergerea legăturilor

Legăturile pot fi șterse cu `rm`.

```
paul@laika:~$ touch data.txt
paul@laika:~$ ln -s data.txt sl_data.txt
paul@laika:~$ ln data.txt hl_data.txt
paul@laika:~$ rm sl_data.txt
paul@laika:~$ rm hl_data.txt
```


28.6. practică: legături

1. Creați două fișiere numite `winter.txt` și `summer.txt`, puneți niște text în ele.
2. Creați o legătură deplină la `winter.txt` numită `hlwinter.txt`.
3. Afișați numerele inod ale acestor trei fișiere, legăturile depline ar trebui să aibă același inod.
4. Folosiți comanda `find` pentru a lista cele două fișiere cu legături depline.
5. Despre un fișier totul este în inode, cu excepția a două lucruri: numiți-le!
6. Creați o legătură simbolică către `summer.txt` numită `slsummer.txt`.
7. Găsiți toate fișierele cu numărul inod 2. Ce vă spune această informație?
8. Priviți directoarele `/etc/init.d/` `/etc/rc.d/` `/etc/rc3.d/` ... vedeți legăturile?
9. Priviți în `/lib` cu `ls -l` ...
10. Folosiți **find** pentru a căuta în directorul `home` fișiere regulate care nu (!) au o legătură deplină.

28.7. soluție: legături

1. Creați două fișiere numite winter.txt și summer.txt, puneți niște text în ele.

```
echo cold > winter.txt ; echo hot > summer.txt
```

2. Creați o legătură deplină la winter.txt numită hlwinter.txt.

```
ln winter.txt hlwinter.txt
```

3. Afișați numerele inod ale acestor trei fișiere, legăturile depline ar trebui să aibă același inod.

```
ls -li winter.txt summer.txt hlwinter.txt
```

4. Folosiți comanda find pentru a afișa cele două fișiere cu legături depline.

```
find . -inum xyz
```

5. Despre un fișier totul este în inod, cu excepția a două lucruri: numiți-le!

Numele fișierului este într-un director, și conținutul este undeva pe disk.

6. Creați o legătură simbolică către summer.txt numită slsummer.txt.

```
ln -s summer.txt slsummer.txt
```

7. Găsiți toate fișierele cu numărul inod 2. Ce vă spune această informație?

Vă spune că există mai mult decât un singur tabel inode (unul pentru fiecare partiție formatată + sisteme de fișier virtuale).

8. Priviți directoarele /etc/init.d/ /etc/rc.d/ /etc/rc3.d/ ... vedeți legăturile?

```
ls -l /etc/init.d
```

```
ls -l /etc/rc.d
```

```
ls -l /etc/rc3.d
```

9. Priviți în /lib cu ls -l ...

```
ls -l /lib
```

10. Folosiți **find** pentru a căuta în directorul home fișiere regulate care nu (!) au o legătură deplină.

```
find ~ ! -links 1 -type f
```

Partea IX. Apendice

Apendice A. certificări

A.1. Certificare

LPI: Linux Professional Institute

LPIC Nivel 1

Acesta este certificarea de nivel junior. Trebuie să treceți examenele 101 și 102 pentru a dobândi **certificarea LPIC 1**. Pentru a trece nivelul unu, trebuie să știți linia de comandă Linux, managementul utilizatorilor, backup și restaurare, instalare, networking, și să aveți aptitudini de bază de administrare a sistemului.

LPIC Nivel 2

Aceasta este certificarea de nivel avansat. Trebuie să aveți certificarea LPIC 1 și să luați examenele 201 și 202 pentru a obține **certificarea LPIC 2**. Pentru a trece la nivelul doi, trebuie să fiți capabil să administrați rețele Linux de mărime medie, incluzând Samba, poștă, știri, proxy, firewall, web și servere ftp.

LPIC Nivel 3

Aceasta este certificarea de nivel senior. Conține un examen de bază (301) care testează aptitudinile avansate în principal în ldap. Pentru a obține acest nivel trebuie de asemeni să aveți LPIC Level 2 și să luați un examen special (302 sau 303). Examenul 302 se focalizează în principal pe Samba, și 303 pe securitate avansată. Mai multe informații la <http://www.lpi.org>.

Ubuntu

Cînd aveți certificarea LPIC Level 1, puteți lua un examen LPI Ubuntu (199) și să deveniți certificat de Ubuntu.

Inginer Certificat de Red Hat

Marea diferență cu multe alte certificări este că nu există alegeri de răspuns multiple pentru **RHCE**. Inginerii Certificați Red Hat trebuie să ia un examen pe viu constînd în două părți. Prima dată, trebuie să depaneze și să mențină un setup existent dar stricat (avînd ca valoare cel puțin 80 la sută), și apoi trebuie să instaleze și să configureze o mașină (avînd ca valoare cel puțin 70 la sută).

MySQL

Există două căi pentru certificare MySQL; Dezvoltator (CMDEV) MySQL 5.0 și Certificat MySQL 5.0 DBA (CMDDBA). **CMDEV** este focalizat asupra dezvoltatorilor aplicație, și **CMDDBA** spre administratori de baze de date. Ambele căi cer fiecare două examene. Certificarea MySQL cluster DBA cere certificarea CMDDBA și trecerea examenului CMCDDBA.

Novell CLP/CLE

Pentru a deveni **Profesionist Linux Certificat Novell**, trebuie să luați un examen live. Aceasta este o sesiune VNC pentru a seta servere reale SLES. Trebuie să faceți mai multe sarcini și sînteți liber să alegeți metoda (linia de comandă sau YaST sau ...). Nu sînt implicate alegeri multiple.

Solaris Sun

Sun folosește formula clasică a examenelor cu alegeri multiple pentru certificare. A lua două examene pentru un sistem de operare vă dă titlul de Administrator Solaris Certificat pentru Solaris X.

Alte certificări

Există multe alte certificări mai puțin cunoscute ca Certificarea Comunității Europene Ethical Hacker, CompTIA Linux +, și Sair Linux GNU.

Apendice B. setări tastatură

B.1. despre interfața tastaturii

Mulți oameni (ca cetățeni americani USA) preferă interfața tastaturii default US-qwerty. Așa că atunci când nu sînteți din USA și vreți o interfață a tastaturii locale, atunci cea mai bună practică este să selectați această tastatură în timpul instalării. Apoi interfața tastaturii va fi mereu corectă. De asemenea, oricînd folosiți ssh pentru a administra de la distanță un sistem Linux, interfața tastaturii locale va fi folosită, independent de configurarea tastaturii server. Astfel nu veți găsi multă informație despre schimbarea interfeței tastaturii din zbor pe Linux, pentru că nu mulți oameni au nevoie de ea. Mai jos sînt cîteva sfaturi care să vă ajute.

B.2. interfața tastaturii X

Aceasta este porțiunea relevantă în /etc/X11/xorg.conf, prima pentru azerty belgian, apoi pentru US-qwerty.

```
[paul@RHEL5 ~]$ grep -i xkb /etc/X11/xorg.conf
Option          "XkbModel" "pc105"
Option          "XkbLayout" "be"
```

```
[paul@RHEL5 ~]$ grep -i xkb /etc/X11/xorg.conf
Option          "XkbModel" "pc105"
Option          "XkbLayout" "us"
```

Cînd sînteți în Gnome sau KDE sau orice alt mediu grafic, uitați-vă la meniul grafic din preferințe, va exista o secțiune a tastaturii pentru care să alegeți interfața. Folosiți meniul grafic în loc de a edita xorg.conf.

B.3. interfața tastaturii shell

Cînd sînteți în bash, uitați-vă în fișierul /etc/sysconfig/keyboard. Mai jos este un exemplu a configurării US-qwerty, urmat de o configurare belgiană azerty.

```
[paul@RHEL5 ~]$ cat /etc/sysconfig/keyboard
KEYBOARDTYPE="pc"
KEYTABLE="us"
```

```
[paul@RHEL5 ~]$ cat /etc/sysconfig/keyboard
KEYBOARDTYPE="pc"
KEYTABLE="be-latin1"
```

Mapările de tastatură pot fi găsite în /usr/share/keymaps sau /lib/kbd/keymaps.

```
[paul@RHEL5 ~]$ ls -l /lib/kbd/keymaps/  
total 52  
drwxr-xr-x 2 root root 4096 Apr 1 00:14 amiga  
drwxr-xr-x 2 root root 4096 Apr 1 00:14 atari  
drwxr-xr-x 8 root root 4096 Apr 1 00:14 i386  
drwxr-xr-x 2 root root 4096 Apr 1 00:14 include  
drwxr-xr-x 4 root root 4096 Apr 1 00:14 mac  
lrwxrwxrwx 1 root root    3 Apr 1 00:14 ppc -> mac  
drwxr-xr-x 2 root root 4096 Apr 1 00:14 sun
```

Apendice C. hardware

C.1. busuri

despre busuri

Componentele hardware comunică cu **Unitatea Centrală de Procesare** sau **cpu** printr-un **bus**. Cele mai comune busuri astăzi sînt **usb**, **pci**, **agp**, **pci-express** și **pcmcia** de asemeni cunoscut ca **pc-card**. Toate acestea sînt busuri **Plug and Play**.

Computerele mai vechi **x86** aveau deseori busuri **isa**, care pot fi configurate folosind întrerupătoare **jumpers** sau **dip**.

/proc/bus

Pentru a lista busurile recunoscute de kernel-ul Linux pe computer, vedeți conținutul directorului **/proc/bus** (capturi de ecran din Ubuntu 7.04 și RHEL4u4 mai jos).

```
root@laika:~# ls /proc/bus/  
input pccard pci usb
```

```
[root@RHEL4b ~]# ls /proc/bus/  
input pci usb
```

Puteți ghici care dintre aceste două capturi de ecran a fost făcută pe un laptop?

/usr/sbin/lshusb

Pentru a lista toate dispozitivele usb conectate la sistem, puteți citi conținutul **/proc/bus/usb/devices** (dacă el există) sau ați putea folosi ieșirea mult mai lizibilă a **lsusb**, care este executată aici pe un sistem SPARC cu Ubuntu.

```
root@shaka:~# lsusb  
Bus 001 Device 002: ID 0430:0100 Sun Microsystems, Inc. 3-button Mouse  
Bus 001 Device 003: ID 0430:0005 Sun Microsystems, Inc. Type 6 Keyboard  
Bus 001 Device 001: ID 04b0:0136 Nikon Corp. Coolpix 7900 (storage)  
root@shaka:~#
```

/var/lib/usbutils/usb.ids

Fișierul **/var/lib/usbutils/usb.ids** conține o listă gzip a tuturor dispozitivelor usb cunoscute.


```
paul@barry:~$ zmore /var/lib/usbutils/usb.ids | head
-----> /var/lib/usbutils/usb.ids <-----
#
# List of USB ID's
#
# Maintained by Vojtech Pavlik <vojtech@suse.cz>
# If you have any new entries, send them to the maintainer.
# The latest version can be obtained from
# http://www.linux-usb.org/usb.ids
#
# $Id: usb.ids,v 1.225 2006/07/13 04:18:02 dbrownell Exp $
```

/usr/sbin/lspci

Pentru a obține o listă cu toate dispozitivele pci conectate, puteți vedea **/proc/bus/pci** sau să executați **lspci** (ieșire parțială mai jos).

```
paul@laika:~$ lspci
...
00:06.0 FireWire (IEEE 1394): Texas Instruments TSB43AB22/A IEEE-139...
00:08.0 Ethernet controller: Realtek Semiconductor Co., Ltd. RTL-816...
00:09.0 Multimedia controller: Philips Semiconductors SAA7133/SAA713...
00:0a.0 Network controller: RaLink RT2500 802.11g Cardbus/mini-PCI
00:0f.0 RAID bus controller: VIA Technologies, Inc. VIA VT6420 SATA ...
00:0f.1 IDE interface: VIA Technologies, Inc. VT82C586A/B/VT82C686/A...
00:10.0 USB Controller: VIA Technologies, Inc. VT82xxxxx UHCI USB 1....
00:10.1 USB Controller: VIA Technologies, Inc. VT82xxxxx UHCI USB 1....
...
```

C.2. întrerupători

despre întrerupători

O **cerere de întrerupere** sau **IRQ** este o cerere de la un dispozitiv la CPU. Un dispozitiv solicită o întrerupere cînd el cere atenția CPU (poate fi din cauză că dispozitivul are date gata de a fi citite de către CPU).

De cînd s-a introdus pci, irq-uri pot fi împărțite între dispozitive.

Întrerupătorul 0 este întotdeauna rezervat pentru timer, întrerupătorul 1 pentru tastatură. IRQ 2 este folosit ca un canal pentru IRQ-urile 8 pînă la 15, și astfel este la fel ca IRQ 9.

/proc/interrupts

Puteți vedea o listare a întrerupătorilor pe sistem în **/proc/interrupts**.

```
paul@laika:~$ cat /proc/interrupts
          CPU0           CPU1
0: 1320048             555    IO-APIC-edge    timer
1:   10224              7    IO-APIC-edge    i8042
7:         0              0    IO-APIC-edge    parport0
8:         2              1    IO-APIC-edge    rtc
10:    3062            21    IO-APIC-fasteoi    acpi
12:     131             2    IO-APIC-edge    i8042
15:   47073             0    IO-APIC-edge    ide1
18:         0              1    IO-APIC-fasteoi    yenta
19:   31056             1    IO-APIC-fasteoi    libata, ohci1394
20:   19042             1    IO-APIC-fasteoi    eth0
21:   44052             1    IO-APIC-fasteoi    uhci_hcd:usb1, uhci_hcd:usb2,...
22: 188352             1    IO-APIC-fasteoi    ra0
23: 632444             1    IO-APIC-fasteoi    nvidia
24:   1585             1    IO-APIC-fasteoi    VIA82XX-MODEM, VIA8237
```

dmesg

De asemeni puteți folosi **dmesg** pentru a afla irq-urile alocate în timpul butării.

```
paul@laika:~$ dmesg | grep "irq 1[45]"
[ 28.930069] ata3: PATA max UDMA/133 cmd 0x1f0 ctl 0x3f6 bmdma 0x2090 irq 14
[ 28.930071] ata4: PATA max UDMA/133 cmd 0x170 ctl 0x376 bmdma 0x2098 irq 15
```

C.3. porturi io

despre porturi io

Comunicarea în cealaltă direcție, de la CPU la dispozitiv, se petrece prin **porturi IO**. CPU scrie date sau coduri de control spre portul IO al dispozitivului. Dar asta nu este comunicare într-o singură direcție, CPU poate de asemeni să folosească un port IO al dispozitivului să citească informație status despre dispozitiv. Spre deosebire de întrerupători, porturile nu pot fi împărțite!

/proc/ioproports

Puteți vedea o listare a porturilor IO a sistemului prin **/proc/ioproports**.

```
[root@RHEL4b ~]# cat /proc/ioproports
0000-001f : dma1
0020-0021 : pic1
0040-0043 : timer0
0050-0053 : timer1
0060-006f : keyboard
0070-0077 : rtc
0080-008f : dma page reg
00a0-00a1 : pic2
00c0-00df : dma2
00f0-00ff : fpu
0170-0177 : ide1
02f8-02ff : serial
...
```

C.4. dma

despre dma

Unui dispozitiv care are nevoie de multe date, întrerupătorii și porturile pot pune o încărcătură grea asupra cpu. Cu **dma** sau **Direct Memory Access** un dispozitiv poate obține acces (temporar) la o rază de acțiune specifică a memoriei **ram**.

/proc/dma

Privind la **/proc/dma** s-ar putea să nu vă dea informația pe care o vreți, pentru că ea conține doar canalele curente desemnate **dma** pentru dispozitivele **isa**.

```
root@laika:~# cat /proc/dma
1: parport0
4: cascade
```

Dispozitivele **pci** care folosesc dma nu sînt listate în **/proc/dma**, în acest caz **dmesg** poate fi folositor. Captura de ecran de mai jos arată că în timpul butării portul paralel a primit canalul 1 dma, și portul Infrared a primit canalul 3 dma.

```
root@laika:~# dmesg | egrep -C 1 'dma 1|dma 3'
[ 20.576000] parport: PnPBIOS parport detected.
[ 20.580000] parport0: PC-style at 0x378 (0x778), irq 7, dma 1...
[ 20.764000] irda_init()
--
[ 21.204000] pnp: Device 00:0b activated.
[ 21.204000] nsc_ircc_pnp_probe() : From PnP, found firbase 0x2F8...
[ 21.204000] nsc-ircc, chip->init
```

Apendice D. Licență (în limba română)

Licența GNU Pentru Documentația Liberă

Versiunea 1.3, 3 Noiembrie 2008

Drepturi de autor 2000, 2001, 2002, 2007, 2008 Fundația Soft-ului Liber, Inc.

Oricui îi este permisă copierea și distribuirea de copii identice ale acestui document, dar fără modificarea lui.

0. PREAMBUL

Scopul acestei Licențe este de a conferi „gratuitate” unui manual, colecții de texte, sau altui document funcțional și folositor, în sensul libertății: de a asigura tuturor permisiunea de copiere și redistribuire, cu sau fără modificări, în scopuri comerciale și necomerciale. Ca scop secundar, această Licență rezervă autorului și editorului dreptul de a fi creditați pentru munca lor, atât timp cât nu sunt responsabili pentru modificările efectuate de către alții.

Această Licență conferă un fel de „obligatii”, ceea ce înseamnă că lucrările derivate dintr-un document trebuie să fie și ele libere, la rîndul lor. Această Licență este inspirată de Licența Publică Generală GNU, care este o licență similară, concepută pentru a acoperi softul liber.

Am creat această Licență pentru a fi de folos manualelor pentru softul liber, deoarece un soft liber necesită o documentație liberă: un program trebuie însoțit de manuale care oferă aceeași libertate de folosire ca și softul. Această Licență nu este limitată, însă, la manualele pentru soft; ea poate fi folosită pentru textul oricărei lucrări, indiferent de subiect sau de modul de publicare. Această Licență este recomandată în principal pentru lucrări care servesc drept referință sau au fost scrise în scop de instruire.

1. APLICABILITATE ȘI DEFINIȚII

Această Licență se aplică oricărei lucrări sau manual, în orice mediu, care conține o notă, inclusă de către deținătorul dreptului de autor, care permite distribuția în termenii acestei Licențe. Această notă conferă dreptul universal, fără indemnizație și nelimitat ca durată de a folosi lucrarea în condițiile de față. Termenul Document, de mai jos, se referă la un astfel de manual sau lucrare. Orice membru din public este un beneficiar al acestei Licențe și va fi desemnat prin termenul „dumneavoastră”. Se consideră, în mod automat, că ați acceptat termenii acestei Licențe, în urma copierii, modificării sau distribuirii unei lucrări într-un mod care necesită permisiunea autorului, în condițiile legii drepturilor de autor.

O „Versiune Modificată” a Documentului este orice lucrare conținând Documentul sau o porțiune de-a lui, copiată identic sau cu modificări și/sau tradusă într-o altă limbă.

O „Secțiune Secundară” este o anexă cu titlu, sau o secțiune menționată în cuprins care are ca scop exclusiv descrierea relației editorilor sau a autorilor Documentului cu subiectul Documentului (sau cu aspecte conexe) și care nu conține referiri directe la subiectul Documentului. (Astfel, dacă Documentul este în parte manual de matematică, o Secțiune Secundară nu poate conține deloc explicații matematice.) Poate exista doar o conexiune istorică cu subiectul și cu problemele

înrudite cu subiectul, ori pot fi prezentate puncte de vedere legale, comerciale, filozofice, etice sau politice legate de acesta.

„Secțiunile Neschimbabile” sunt anumite Secțiuni Secundare ale căror titluri sunt specificate ca fiind acele titluri de Secțiuni Neschimbabile din nota ce permite distribuția Documentului sub acoperirea acestei Licențe. Dacă o secțiune nu este conformă cu definiția de mai sus a unei Secțiuni Secundare atunci ea nu poate fi desemnată ca fiind Neschimbabilă. Documentul poate să nu conțină Secțiuni Neschimbabile. Dacă Documentul nu specifică vreo Secțiune Neschimbabilă atunci se consideră că nu există nici una.

„Textele De Copertă” sînt anumite pasaje scurte de text care sînt listate ca Texte Pentru Coperta I sau ca Texte Pentru Coperta IV în nota care specifică distribuirea Documentului sub acoperirea acestei Licențe. Un Text Pentru Coperta I poate avea cel mult 5 cuvinte, iar un Text Pentru Coperta IV poate avea cel mult 25 de cuvinte.

O copie „Transparentă” a Documentului este o copie în format electronic, reprezentată într-un format ale cărui specificații sunt disponibile publicului, fiind ușor de modificat cu ajutorul unui editor de text generic sau (pentru imagini compuse din pixeli) cu un editor grafic generic ori (pentru desene) cu un editor larg răspândit de grafică vectorială, și care poate fi folosit ca intrare în procesoarele de text sau de transformare automată în diverse formate adecvate ca intrare pentru procesoarele de text. O copie făcută într-un format de fișier Transparent dar care, prin prezența sau absența anumitor elemente specifice formatului, descurajează sau împiedică modificările ulterioare, nu reprezintă o copie Transparentă. Un format de imagine nu este Transparent dacă este folosit pentru a reprezenta o cantitate substanțială de text. O copie care nu este „Transparentă” se numește „Opacă”.

Exemple de formate compatibile cu copiile Transparente: textul ASCII fără marcate, formatul de intrare Texinfo, formatele de intrare LaTeX, SGML și XML folosind un DTD public, HTML simplu și standard, fișierele PostScript și PDF modificabile. Exemple de formate Transparente pentru imagine: PNG, XCF și JPG. Formatele Opace includ formate de text ce pot fi citite și editate doar de procesoare de text proprietare, SGML și XML pentru care DTD-ul și/sau uneltele de procesare nu sunt disponibile, HTML generat automat, documentele PostScript și PDF produse de diverse procesoare de text doar în scopul printării/afișării.

„Pagina de Titlu” înseamnă, pentru o carte tipărită, pagina cu titlul și paginile următoare, necesare pentru a prezenta, lizibil, materialul care trebuie tipărit, conform acestei Licențe, pe Pagina de Titlu. Pentru lucrări care nu au o pagină cu titlu propriu-zisă, „Pagina de Titlu” este textul aflat lângă principala apariție a titlului lucrării, precedînd începutul corpului Documentului.

„Editorul” reprezintă orice persoană sau entitate care distribuie copii ale documentului pentru public.

O secțiune „Numită XYZ” este o subunitate a Documentului, al cărei titlu este, fie XYZ, fie conține XYZ în paranteze, după textul care traduce XYZ în altă limbă. (Aici XYZ înlocuiește nume specifice ce vor fi menționate mai jos, ca de exemplu „Mulțumiri”, „Dedicații”, „Giruri” sau „Istorie”.) Pentru a „Păstra Titlul” unei astfel de secțiuni atunci cînd modificați Documentul înseamnă că va rămîne o secțiune „Numită XYZ”, conform acestei definiții.

Documentul poate include Limitări de Responsabilitate atașate notificării care afirmă că această Licență se aplică Documentului. Acestea se consideră a fi incluse

prin referință în această Licență, dar numai cu privire la limitările de responsabilitate: orice alte implicații pe care aceste Limitări de Responsabilitate le-ar putea avea sunt nule și nu au nici un efect asupra înțelesului acestei Licențe.

2. COPII IDENTICE

Puteți copia și distribui Documentul pe orice mediu, fie comercial sau necomercial, atât timp cât această Licență, notificările de drepturi de autor și notificarea de licență care spune că această Licență se aplică acestui Document, sunt reproduse în toate copiile, și atâta timp cât nu adăugați nici un fel de altă condiție în afară de cele prezente în această Licență. Nu aveți dreptul să luați măsuri tehnice de a obstrucționa sau controla citirea sau recopiarea copiilor pe care le faceți sau le distribuiți. Aveți totuși dreptul să acceptați compensații în schimbul copiilor. Dacă distribuiți un număr suficient de mare de copii, atunci trebuie să respectați și condițiile din secțiunea 3.

Aveți, de asemenea, dreptul să împrumutați copii în aceleași condiții ca cele de mai sus, și aveți dreptul să afișați copii.

3. COPIEREA ÎN CANTITĂȚI MARI

Dacă publicați copii tipărite (sau copii în medii care folosesc de obicei coperti tipărite) ale Documentului, în număr mai mare de 100 și dacă notificarea de licență a Documentului cere Texte de Copertă, trebuie să includeți copiile pe coperti care să conțină, clar și lizibil, toate aceste Texte de Copertă: Textele Pentru Coperta I pe coperta I și Texte Pentru Coperta IV pe coperta IV. Ambele coperti trebuie de asemenea să vă identifice în mod clar și lizibil ca editor al respectivelor copii. Coperta I trebuie să prezinte titlul în întregime, cu toate cuvintele din titlu la fel de vizibile și proeminente. Puteți adăuga alte materiale pe copertă în plus. Copierea cu modificările limitate la coperti, atâta timp cât satisfac aceste condiții, pot fi tratate în toate celelalte aspecte ca și copii identice.

Dacă textele necesare pentru oricare dintre coperti sînt prea voluminoase pentru a încăpea în mod lizibil, trebuie să puneți primele rînduri (atâtea cît încap în mod rezonabil) pe coperta efectivă și să continuați cu restul pe pagini adiacente.

Dacă publicați sau distribuiți copii Opace ale Documentului în număr mai mare de 100, trebuie ori să includeți cîte o copie Transparentă în format electronic împreună cu fiecare copie Opacă, ori să specificați în sau împreună cu fiecare copie Opacă o locație din rețeaua electronică la care publicul general care folosește rețeaua să aibă acces pentru a descărca, folosind un protocol standard public, copii complete, Transparente ale documentului, fără adăugarea oricărui material adițional. Dacă folosiți a doua opțiune trebuie să faceți demersuri rezonabil de prudente ca atunci cînd începeți distribuirea copiilor Opace, să vă asigurați că această copie Transparentă va rămîne accesibilă, în acest fel, la locația respectivă timp de cel puțin un an după distribuția ultimei copii Opace (în mod direct sau prin agenți ori distribuitori) a respectivei ediții pentru public.

Se cere, dar nu în mod necesar, să contactați autorii Documentului cu o perioadă bună înainte de a distribui orice cantitate mare de copii, pentru a le da ocazia să vă pună la dispoziție o versiune actualizată a Documentului.

4. MODIFICĂRI

Puteți copia și distribui o Versiune Modificată a Documentului în condițiile secțiunilor 2 și 3 de mai sus, cu condiția de a acoperi Versiunea Modificată sub

exact această Licență, cu Versiunea Modificată ținând locul Documentului, astfel licențind distribuirea și modificările Versiunii Modificate oricui intră în posesia unei copii ale acesteia. În plus, trebuie să faceți următoarele lucruri în Versiunea Modificată:

- * A. Folosiți în Pagina de Titlu (și pe coperti, dacă există) un titlu diferit de cel al Documentului, și de versiunile sale anterioare (care trebuie, dacă există, să fie listate în secțiunea de Istorie a Documentului). Puteți folosi același titlu ca o versiune anterioară dacă editorul original al acelei copii vă dă permisiunea.
- * B. Listați pe Pagina de Titlu, ca autori, una sau mai multe dintre persoanele sau entitățile responsabile în calitate de autori pentru modificările Versiunii Modificate, împreună cu cel puțin cinci dintre autorii principali ai Documentului (toți autorii principali, dacă are mai puțin de cinci), în afară de cazul că aceștia vă eliberează de această obligație.
- * C. Includeți pe Pagina de Titlu numele editorului Versiunii Modificate în calitate de editor.
- * D. Păstrați toate notificările de drepturi de autor ale Documentului.
- * E. Adăugați o notificare de drepturi de autori relevantă pentru modificările dumneavoastră adiacent celorlalte notificări de drepturi de autor.
- * F. Includeți, imediat după notificările de drepturi de autor, o notificare de licență dând permisiune publică de a folosi Versiunea Modificată în condițiile acestei Licențe, sub forma prezentată în Apendicele de mai jos.
- * G. Păstrați în acea notificare de licență lista integrală a Secțiunilor Neschimbabile și Textele de Copertă necesare, date în notificarea de licență a Documentului.
- * H. Includeți o copie nealterată a acestei Licențe.
- * I. Păstrați secțiunea Numită „Istorie”, păstrați-i Titlul și adăugați-i un element care să indice măcar titlul, anul, noii autori și editorul Versiunii Modificate așa cum este dat pe Pagina de Titlu. Dacă nu există o secțiune numită „Istorie” în Document, creați una în care indicați titlul, anul, autorii și editorul Documentului așa cum este dat pe Pagina de Titlu al acestuia și apoi adăugați un element care să descrie Versiunea Modificată așa cum a fost cerut în fraza precedentă.
- * J. Păstrați locația de rețea, dacă există, dată în Document pentru acces public la o copie Transparentă a Documentului, cât și locațiile de rețea date în Document pentru versiunile mai vechi pe care s-a bazat acesta. Acestea pot fi incluse în secțiunea Numită „Istorie”. Puteți omite locația de rețea a unei lucrări care a fost publicată cu cel puțin patru ani înainte de Documentul în sine, sau dacă editorul original al versiunii la care se referă vă dă permisiunea.
- * K. Pentru orice secțiune numită „Mulțumiri” sau „Dedicații” păstrați Titlul secțiunii și păstrați în secțiunile respective toată substanța și tonul mulțumirilor și/sau dedicațiilor fiecărui contribuitor.
- * L. Păstrați toate Secțiunile Neschimbabile ale Documentului, nealterate ca text și ca titluri. Numerotarea secțiunilor sau echivalentul numerotării nu sunt considerate ca făcând parte din titlurile secțiunilor.
- * M. Ștergeți orice secțiune numită „Giruri”. O astfel de secțiune nu poate fi inclusă în Versiunea Modificată.
- * N. Nu modificați titlul nici unei secțiuni existente pentru a fi numită „Giruri” sau pentru a intra în conflict cu vreo Secțiune Neschimbabilă.
- * O. Păstrați toate Limitările de Responsabilitate.

Dacă Versiunea Modificată include secțiuni noi incluse în titlu sau anexe care se califică drept Secțiuni Secundare și nu conțin material copiat din Document, aveți dreptul la alegerea dumneavoastră să numiți unele sau toate acestea ca fiind Neschimbabile. Pentru a face aceasta, adăugați-le titlurile la lista de Secțiuni Neschimbabile în notificarea de licență a Versiunii Modificate. Aceste titluri trebuie să fie distincte față de toate celelalte titluri de secțiune.

Puteți adăuga o secțiune Numită „Giruri” doar dacă aceasta conține numai girurile a diverse entități asupra Versiunii Modificate – de exemplu recenzii sau faptul că textul a fost aprobat de o organizație ca fiind o definiție autoritară a unui standard.

Puteți adăuga un pasaj de cel mult cinci cuvinte ca Text Pentru Coperta I și un pasaj de cel mult 25 de cuvinte ca Text Pentru Coperta IV la sfârșitul Textelor De Copertă în Versiunea Modificată. Numai un singur pasaj poate fi adăugat la Textul Pentru Coperta I și unul la Textul Pentru Coperta IV de către (sau prin aranjament cu) orice entitate. Dacă Documentul conține deja texte de copertă pentru coperta respectivă, adăugat în prealabil de dumneavoastră sau prin aranjament cu aceiași entitate în numele căreia acționați, atunci nu puteți adăuga un altul, însă puteți să-l înlocuiți pe cel vechi numai cu permisiunea explicită a editorului anterior care l-a adăugat pe cel vechi.

Autorul (autorii) și editorul (editorii) Documentului nu vă dau prin această Licență permisiunea de a le folosi numele pentru publicitate sau pentru a pretinde sau implica vreo girare a oricărei Versiuni Modificate.

5. COMBINAREA DOCUMENTELOR

Puteți combina Documentul cu alte documente acoperite de această Licență sub termenii definiți în secțiunea 4 de mai sus pentru versiuni modificate, cu condiția să includeți în versiunea combinată toate Secțiunile Neschimbabile ale tuturor documentelor originale, nemodificate, și să le listați pe toate ca Secțiuni Neschimbabile ale versiunii combinate în notificarea de licență, cât și să păstrați toate Limitările de Responsabilitate.

Versiunea modificată nu trebuie să conțină decît o singură copie a acestei Licențe, iar duplicatele identice ale Secțiunilor Neschimbabile pot fi înlocuite cu o singură copie. Dacă există Secțiuni Neschimbabile cu nume identice și conținut diferit, schimbați-le numele adăugând la sfârșitul titlului, în paranteză, ori numele autorului sau al editorului original al acelei secțiuni dacă acesta este cunoscut, ori un număr unic. Faceți aceleași modificări respective titlurilor secțiunilor în lista de Secțiuni Neschimbabile din notificarea de licență a versiunii combinate.

În versiunea combinată trebuie să combinați și toate secțiunile Numite „Istorie” din diversele documente originale, creînd o secțiune unică Numită „Istorie”; la fel trebuie să combinați și toate secțiunile Numite „Mulțumiri” cît și cele numite „Dedicații”. Trebuie să ștergeți toate secțiunile numite „Giruri”.

6. COLECȚII DE DOCUMENTE

Puteți crea o colecție formată din Document și alte documente acoperite de această Licență și să înlocuiți copiile individuale ale acestei Licențe din diversele documente cu o singură copie care să fie inclusă în colecție cu condiția să urmați regulile acestei Licențe pentru copii identice pentru fiecare document în toate celelalte privințe.

Puteți să extrageți un document dintr-o astfel de colecție și să-l distribuiți individual sub această Licență cu condiția de a include o copie a acestei Licențe în documentul extras și să urmați condițiile acestei Licențe în toate celelalte privințe în legătură cu copiile identice ale acelui document.

7. AGREGAREA CU LUCRĂRI INDEPENDENTE

O compilație a Documentului sau a unui derivat al său cu orice document sau lucrare separată independentă, în sau pe un volum de stocare sau distribuire se numește „agregat” dacă drepturile de autor rezultate în urma compilării nu sunt folosite pentru a limita drepturile legale ale utilizatorilor compilației mai mult decât permit lucrările individuale. Când Documentul este inclus într-un agregat, această Licență nu se aplică celorlalte lucrări din agregat care nu sunt ele însele rezultate derivate ale Documentului.

Dacă cerințele legate de Textele de Copertă din secțiunea 3 se aplică acestor copii ale Documentului, atunci dacă Documentul este mai puțin de jumătate din întregul agregat atunci Textele de Copertă ale Documentului pot fi puse pe coperti care să separe Documentul în cadrul agregatului, sau pe un echivalent electronic al acestora, dacă Documentul se prezintă în format electronic. Altfel ele trebuie să apară pe copertile tipărite care îmbracă întreg agregatul.

8. TRADUCERE

Traducerea este considerată o formă de modificare, drept care puteți distribui traduceri ale Documentului sub cerințele secțiunii 4. Înlocuirea Secțiunilor Neschimbabile cu traduceri ale acestora necesită permisiune specială din partea celor care dețin drepturile de autor, însă puteți include traduceri ale unora dintre sau tuturor Secțiunilor Neschimbabile împreună cu variantele originale ale acestora. Puteți include o traducere a acestei Licențe cât și toate notificările de licență din Document, cât și Limitările de Responsabilitate atâta timp cât includeți și versiunea originală în limba engleză a acestei Licențe, plus versiunile originale ale respectivelor notificări de licență și limitări de responsabilitate. În cazul apariției oricăror discrepanțe între versiunea tradusă și versiunea originală a acestei Licențe, a vreunei notificări de licență sau a vreunei limitări de responsabilitate, versiunea originală are prioritate.

Dacă vreo secțiune din Document este Numită „Mulțumiri”, „Dedicații” sau „Istorie” cerința (din secțiunea 4) de a-i păstra Titlul (secțiunea 1) va necesita în mod normal schimbarea titlului în sine.

9. REZILIERE

Nu puteți copia, modifica, sublicenția sau distribui Documentul decât în condițiile specificate explicit în această Licență. Orice copiere, modificare sau redistribuire a Documentului în vreo altă condiție este nulă și vă va anula în mod automat drepturile conferite de această Licență.

Cu toate acestea, dacă încetați orice încălcare a acestei Licențe, licența din partea titularului dreptului de autor este reinstaurată (a) cu titlu provizoriu, cu excepția cazului când titularul dreptului de autor încetează în mod explicit și în cele din urmă licența, și (b) permanent, în cazul în care titularul dreptului de autor nu vă anunță încălcarea, prin mijloace rezonabile, în termen de 60 de zile de la încetare.

În plus, licența de la titularul particular al dreptului de autor este repusă permanent în cazul în care titularul dreptului de autor vă anunță de încălcare prin mijloace rezonabile, și este prima dată când ați primit o notificare de încălcare a acestei Licențe (pentru orice lucrare), din partea titularului dreptului de autor, și ați încetat încălcarea cu 30 de zile înainte de primirea notificării. Încetarea drepturilor dumneavoastră, în conformitate cu această secțiune, nu încetează licențele părților care au primit copii sau drepturi de la dumneavoastră

sub această Licență. Dacă drepturile dumneavoastră au fost terminate și nu s-au repus permanent, primirea unei copii ale acelui material nu vă dă nici un drept să-l folosiți.

10. VERSIUNI VIITOARE ALE ACESTEI LICENȚE

Fundația Free Software Foundation poate publica, din când în când, versiuni noi, revizuite ale acestei Licențe GNU pentru Documentația Liberă. Aceste noi versiuni vor păstra spiritul acestei versiuni dar pot diferi în privința detaliilor, cu scopul de a se adresa unor noi probleme reale sau potențiale. A se vedea <http://www.gnu.org/copyleft/>.

Fiecărei versiuni ale acestei Licențe îi este asociat un număr de versiune distinct. Dacă Documentul specifică un anumit număr de versiune „sau orice versiune ulterioară” al acestei Licențe, aveți de ales între a vă conforma termenilor și condițiilor ori ale versiunii specificate explicit sau ale oricărei variante ulterioare publicate (nu ca variantă preliminară) de către Free Software Foundation. Dacă Documentul nu specifică un număr de versiune al acestei Licențe atunci puteți alege orice versiune publicată (nu ca variantă preliminară) de către Free Software Foundation.

11. RELICENȚIEREA

„Site-ul de Colaborare Masivă a Multiautorilor” (sau „MMC Site”) înseamnă orice server www care publică lucrări posibil de a fi supuse drepturilor de autor și, de asemenea, oferă facilități proeminente pentru oricine editează aceste lucrări. Un server public wiki este un exemplu în care oricine poate edita lucrări scrise. Un „Site de Colaborare Masivă a Multiautorilor” (sau „MMC Site”) în conținutul său înseamnă o mulțime de lucrări susceptibile de a fi supuse licențelor supuse site-ului MMC.

„CC-BZ-SA” înseamnă licență a Creative Commons Attribution-Share Alike 3.0 publicată de Corporația Creative Commons, o corporație nonprofit cu sediul principal în San Francisco, California, și de asemenea viitoare variante de „obligații” a acestei licențe publicată de aceeași organizație.

„Încorporarea” înseamnă publicarea ori republicarea unui Document, în întregime sau în parte, ca parte a unui alt document.

Un MMC este „eligibil pentru reautorizare” dacă este licențiat sub această Licență, și în cazul în care toate lucrările care au fost publicate mai întâi în această Licență în altă parte decât acest MMC, și, ulterior, au fost încorporate în totalitate sau în parte în MMC, (1) nu a avut texte de copertă sau secțiuni invariante, și (2) au fost astfel incluse până la 1 noiembrie 2008.

Operatorul unui site MMC poate republica un MMC conținut în site sub CC-BY-SA în același loc, în orice moment înainte de 1 august 2009, cu condiția ca MMC să fie eligibil pentru reautorizare.

Apendice E. Licență (în limba engleză)

GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document "free" in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of "copyleft", which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The "Document", below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as "you". You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A "Modified Version" of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A "Secondary Section" is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document's overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The "Invariant Sections" are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above

definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The "Cover Texts" are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A "Transparent" copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not "Transparent" is called "Opaque".

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The "Title Page" means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, "Title Page" means the text near the most prominent appearance of the work's title, preceding the beginning of the body of the text.

The "publisher" means any person or entity that distributes copies of the Document to the public.

A section "Entitled XYZ" means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as "Acknowledgements", "Dedications", "Endorsements", or "History".) To "Preserve the Title" of such a section when you modify the Document means that it remains a section "Entitled XYZ" according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies,

and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- * A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.
- * B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together

- with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.
- * C. State on the Title page the name of the publisher of the Modified Version, as the publisher.
 - * D. Preserve all the copyright notices of the Document.
 - * E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
 - * F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
 - * G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
 - * H. Include an unaltered copy of this License.
 - * I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
 - * J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
 - * K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
 - * L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
 - * M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.
 - * N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.
 - * O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties – for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements".

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an "aggregate" if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled "Acknowledgements", "Dedications", or "History", the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License "or any later version" applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that

version for the Document.

11. RELICENSING

"Massive Multiauthor Collaboration Site" (or "MMC Site") means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A "Massive Multiauthor Collaboration" (or "MMC") contained in the site means any set of copyrightable works thus published on the MMC site.

"CC-BY-SA" means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

"Incorporate" means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is "eligible for relicensing" if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.